Offline AI NPC 1.1 - documentation

English · 1.1

Contents

1. Offline AI NPC — documentation
2. Is this for your game?
3. Getting started
4. Upgrading from Offline AI NPC 1.0
5. Release history
6. Beginner's guide
7. Recipes
8. The interview demo
9. Dialogue: wiring a talking character
10. Internal state
11. Points of interest
12. Structured actions
13. Which languages does this support?
14. Using the current local models
15. Qwen speech: language, voice and style
16. Agent tools (MCP)
17. Limitations
18. Hardware tiers
19. Licensing
20. Troubleshooting
21. Offline AI NPC — architecture
22. Third-party notices — Offline AI NPC
23. Changelog — Offline AI NPC
24. Voice licences and provenance
25. native-build-manifest.json
26. The espeak boundary
27. Source pins — recorded 2 October 2026
28. GPL-3.0.txt
29. GCC-RUNTIME-EXCEPTION.txt
30. ggml-LICENSE.txt
31. GPL-3.0.txt
32. MinGW-w64-runtime.txt
33. qwentts-LICENSE.txt

- Documentation languages
- Start here
- Building with it
- Before you ship
- Reference
- Check your game
- What is not included

Offline AI NPC — documentation

With local engines: fully offline conversational NPCs for Unity, using a local language model and local speech recognition, local speech synthesis, per-character memory and state. No cloud, no API keys, no per-token cost, no telemetry. The one exception lives in the editor and is yours to choose: the optional MCP server passes project data to the coding agent you connect (Agent tools).

Documentation languages

Language	Available guides
English	Complete documentation
Русский	Complete documentation
简体中文	Complete documentation
Español	Complete documentation

Choose **Editor UI language** in `Tools ▸ Offline AI NPC ▸ Welcome` or `Tools ▸ Offline AI NPC ▸ Check my setup`. The choice also selects the translated guides. It does not change an NPC's language. Every window of the plugin — Welcome, Check my setup, Model Manager, Presets, Live verification, Prompt debugger and Agent tools — and its inspectors are translated, including the results of the built-in checks. Console messages — apart from those of **Advanced ▸ Speak one line (Piper)** — and the descriptions the MCP tools give an agent stay in English.

Start here

Is this for your game?	**Read first.** Suitable game scenarios and hardware/setup requirements
Getting started	Import to a talking character
Upgrading from 1.0	Keep your working project, speech setup and custom subtitle code when updating

Release history	Versions 1.1 and 1.0; links to upgrade guidance
Beginner's guide	The long version: empty project to a build you can hand to somebody, assuming nothing
Recipes	Five shapes this is good at, and roughly what each costs to build
The Interview	A finished demo built on the plugin, and which parts it exercises

Building with it

Dialogue	Wiring a talking character — the complete controller
State	Moods, needs, relationships. **Opens with the ten-line version**
Perception	The character knowing what is actually around it
Actions	The character doing, not only saying
Languages	How many languages, honestly — and how to add one
Current model usage	Qwen text settings, Piper protocols and Whisper audio/language/lifecycle
Qwen speech usage	TTS language, voice and style; CustomVoice limits and ASR distinctions
Agent tools (MCP)	An AI coding agent inspecting and testing your characters through the editor

Before you ship

Limitations	Every limit, with its mitigation where one exists
Hardware tiers	What each tier means, and what happens below the lowest
Licensing	What you may ship and what shipping it obliges

Troubleshooting	Real error strings mapped to fixes

Reference

Architecture	Assembly boundary, seams, folder layout
Third-party notices	Full inventory
Changelog	

Check your game

Run **Tools** ▸ **Offline AI NPC** ▸ **Live verification** in the Hello NPC sample, then in your scene. Test the first reply, a longer conversation, your chosen voice and background NPC activity while the game renders. This window measures to synthesized audio; also check what the player hears.

What is not included

Named so you do not go looking: RAG or document retrieval, lip sync, per-word prosody control in speech synthesis (a delivery hint per line is what Qwen3-TTS takes), streaming (as opposed to per-utterance) speech recognition, and mobile, web or console support.

- The one thing to understand first
- Fits well
- Does not fit
- Fits with care
- What you also need
- Still unsure?

Is this for your game?

Read this before you buy. It is written to talk the wrong buyer out of the purchase, because a refund and a one-star review cost more than one sale is worth — and because the shape of this plugin genuinely suits some games and genuinely ruins others.

The one thing to understand first

Budget generation alongside the rest of your game. Characters can share an LLMUnity server or use separate hosts. Concurrent requests on a shared server can queue; the server configuration and available hardware determine concurrency.

Give the player's conversation priority. In a tavern, generate replies for the selected or nearby NPCs, and use recorded chatter for the room. For a companion, generate comments between encounters and use authored audio for an immediate combat alert. Test your chosen model and voice while the game is rendering.

Fits well

A small cast. One to a handful of characters. A companion, a shopkeeper, a suspect, a roommate. One conversation at a time is the design centre.

Player-initiated, turn-shaped conversation. The player speaks or types, the character answers. This is a dialogue system, not a barking system.

Genres built around talking. Visual novels. Life and social sims. Interrogation and detective scenes. Companion characters. VR social experiences. Immersive sims with a few deep NPCs rather than many shallow ones.

Games with a defined hardware budget. Plan memory and generation time alongside rendering. A small model can run on CPU; GPU use depends on your chosen setup. See Hardware tiers.

Teams who work with an AI coding agent. A character who answers differently every time is tiring to test by hand: press Play, walk over, talk, read, stop, change a number, repeat. The package ships an MCP server for the editor, off until you switch it on, through which Claude Code, Cursor or Codex can ask the character something, read exactly what the model was told, see why the door is or is not in its context, change it and ask again — see Agent tools.

Offline and privacy as requirements. With local engines: no API keys, no per-call billing, no rate limits, no network at all. Nothing of the player's leaves the machine, and there is no telemetry in this package — not even anonymous. If your game must work on a plane, in a school, or under a privacy policy that forbids sending player speech to a third

party, that is the case this was built for. In the editor, the optional MCP server is the one exception, and yours to choose: it hands project data to the coding agent you connect, and an agent that runs its model in the cloud sends it on to its provider — see Agent tools.

Does not fit

Plainly, so nobody is surprised after paying.

Mobile, web, or console. This package targets desktop games. Windows has been exercised; the complete macOS/Linux path is not verified with matching native engines and a built player. Do not infer verified support from a dependency's platform list. Mobile, web and consoles are not currently supported.

Unbounded background generation. Keep ambient requests within a budget and pause them for the player. A populated tavern can combine generated conversations with recorded room chatter; measure how many active conversations your setup can sustain.

Immediate combat barks. Use authored audio for an alert that must play as soon as an enemy spots the player. Generated speech can supply a companion's remarks between fights; measure its complete input-to-playback path.

Tightly authored narrative. The grammar constrains the *shape* of a reply — that an action payload is well-formed — never its content. If a line must be delivered word for word because the plot turns on it, a dialogue tree is the right tool. **A dialogue system and this plugin are complements, not competitors:** author the beats that matter, and use this for the space between them.

Low-spec target audiences. See Hardware tiers. Below the lowest tier the plugin degrades to no language model at all, which is a working state but not the product you are buying.

Anything needing guaranteed output. A language model produces a different reply every time, occasionally a bad one. There is a resilience layer so the game never crashes, but if your design cannot tolerate an off-key line, this is the wrong tool.

Fits with care

Many characters. Six residents do not require six simultaneous requests. Activate the conversation the player is having and schedule the others. If your design needs simultaneous generation, test it with the actual server configuration, model, voice and game workload.

Games leaning on the deterministic half. State axes, thresholds and events are exact every time and cost nothing; the language model is the expensive, probabilistic part. A design that drives most of its behaviour from state and reaches for the model sparingly gets the best of both. See State.

A first project. It works, but you will be integrating an LLM, a speech recogniser and a speech synthesiser, each with its own failure modes. Budget time for that rather than expecting a drop-in NPC.

What you also need

Two external dependencies, neither bundled, both free:

- **LLM for Unity** (LLMUnity), for the language model.
- **whisper.unity**, for speech recognition. Optional if you only want typed input.

And a model file — 1 to 5 GB depending on tier, which you or your player downloads. The package ships a catalog of licence-checked options and a download manager, but not the weights themselves.

Speech synthesis for the built-in language definitions is Qwen3-TTS (Apache-2.0 weights, MIT engine, ten languages, nothing copyleft): its Windows x64 CPU library comes with the package, and you download its two model files in Model Manager. Piper is a second engine, an external program you install. Only Windows has been exercised for either. If you choose Piper and redistribute it, note that it links espeak-ng under GPL-3.0 — see Licensing, which explains exactly what that obliges and how to avoid it.

Still unsure?

Open the Hello NPC sample from Welcome, press Play, and read your own numbers: `Tools ▸ Offline AI NPC ▸ Live verification` times real turns on *your* machine — to synthesized audio, with the tier and the layers on the GPU. It needs Play mode; outside it, **Advanced ▸ Speak one line (Piper)** plays a fixed Piper sentence with no model. "Will this run acceptably here" is the one question no cloud-based competitor can answer for you, and the only honest way to answer it is on your hardware.

- Choose a speech setup for your game
- 0. The window that opens by itself
- 1. Dependencies
- 2. A model
  - Optional: use your own GGUF with Hello NPC
  - Including models in a build
- 3. Check the text setup
- 4. Your first typed answer
- 5. Optional: speech output
  - Optional: connect a local Piper voice
- 6. Optional: speech recognition
- 7. Your own character
- Adding your own assembly
- Where to go next

Getting started

From import to the first typed answer; add voice and microphone input afterwards. Read Is this for your game? first if you have not — it takes two minutes and it is the page most likely to save you a refund.

Required path: install compatible LLM for Unity → obtain and configure one suitable language model → open Hello NPC → type a question and read its answer. Speech output and microphone input are optional steps after that first answer.

Choose a speech setup for your game

Start with typed dialogue. Add Whisper when players need a microphone, then choose a ready voice:

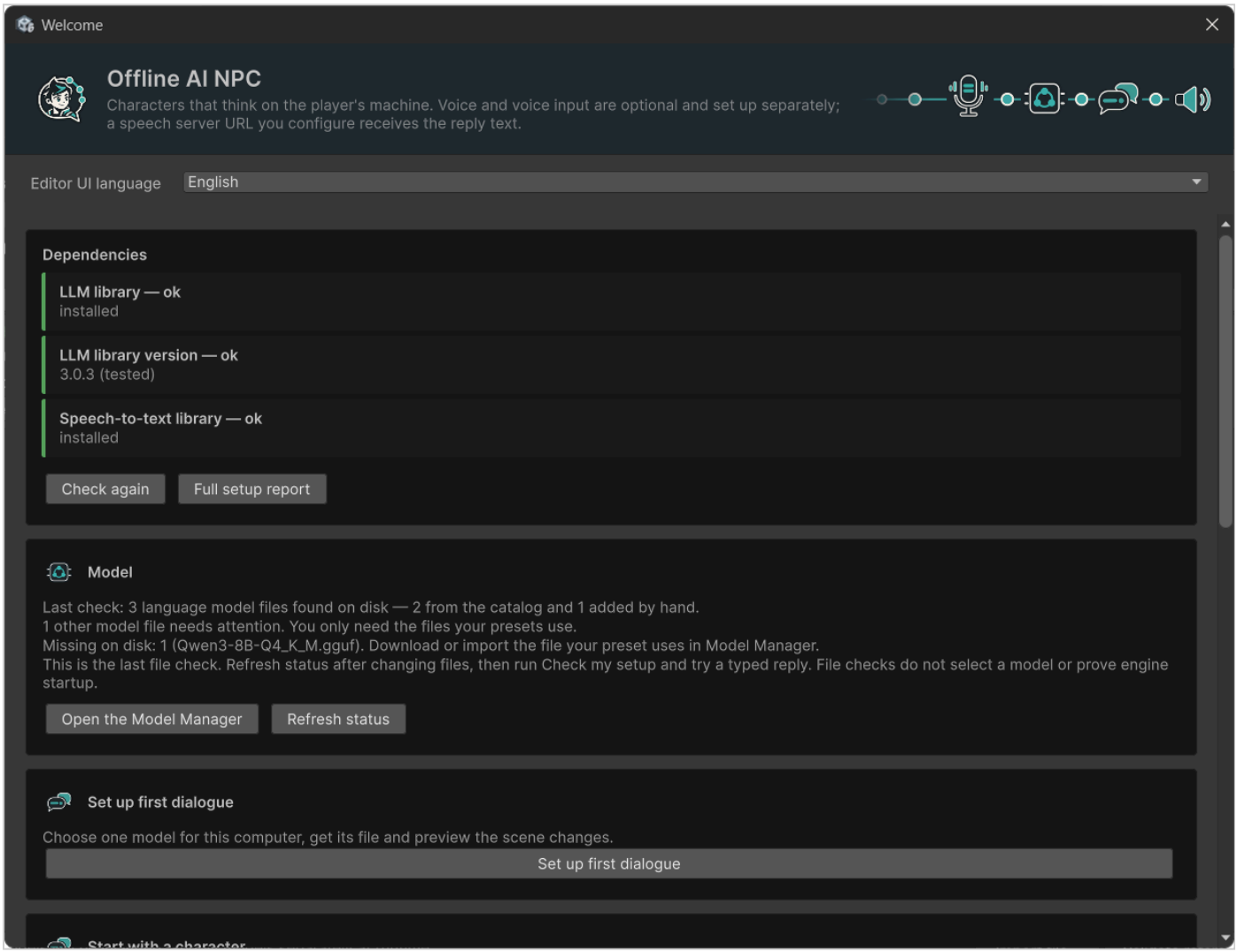
Your priority	Start with
A quick reply and a suitable voice for your language	Piper with that voice's `.onnx` / `.onnx.json` pair
One voice engine across several supported languages	Qwen3-TTS with a named CustomVoice speaker
No spoken replies	Keep text dialogue; speech output is optional

The built-in Qwen speech presets use the 0.6B CustomVoice Q4 talker and Q8 codec. New Qwen engine components use the bundled CPU DLL with **External server URL** and **Style** empty, and the existing 1.5-second lead buffer. Serena (`serena`) is the default for every built-in Qwen language and preset. You can choose another of the nine ready speakers; creating a voice from a description (VoiceDesign) is not part of this version. Instructions for delivery style require a separately checked 1.7B CustomVoice setup. Check the language, voice terms and response time on your players' hardware; see Languages, Hardware tiers and Licensing.

Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no separately started HTTP server, Python environment, speech executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.

0. The window that opens by itself

The first editor load after import opens **Welcome** — `Tools ▸ Offline AI NPC ▸ Welcome` if you want it back. It runs the checks on this page for you and links to the two windows below.



It shows **once** on a machine, on the first load, and once more after an update to a new release such as 1.1 — not after a recompile, not in the next project. It changes nothing in your project on its own: no Project Settings, no scene edits, no files. Everything it can do is behind a button you press.

The cards follow the order of this page: dependencies, the language model, a character that answers in text, **What works now** — typed replies, voice and voice input, each ready, below the intended quality, blocked or not measured yet — then the optional voice setup, **Hear a character speak**, and documentation and support. **Open Hello NPC sample** opens the already-wired example directly, asking you to save your current scene first; without LLM for Unity the card names the version to install, and **How to install LLM for Unity** takes the button's place. Download missing files in the Model Manager, then use **Check again** to refresh the status. Start with typed dialogue; microphone input

and speech output can be configured afterwards. **Create demo scene** is the alternative for wiring your own character from perception and state; it makes a new, unsaved scene. Updating an existing project? Open the migration guide from this window too. You can close Welcome and work from the pages; nothing later depends on having used it.

---

1. Dependencies

Two external, free packages. Install them separately; this plugin includes optional text source fixes with their original licences and modification notices.

	Package id	Needed for
LLM for Unity	`ai.undream.llm`	the language model. Required; API minimum 3.0.3
whisper.unity	`com.whisper.unity`	speech input. Optional — typed input works without it

The bundled compatibility fixes target **LLM for Unity 3.0.3** and **whisper.unity 1.4.0** at the revisions below. For that route, use Package Manager → **Add package from git URL**:

```
https://github.com/undreamai/LLMUnity.git#9931a9ea46906b1f43adaf9498c05ccf227b5d0b
```

For optional microphone input:

```
https://github.com/Macoron/whisper.unity.git?
path=/Packages/com.whisper.unity#529a628a915a97799e89e061af9cb7c71407124d
```

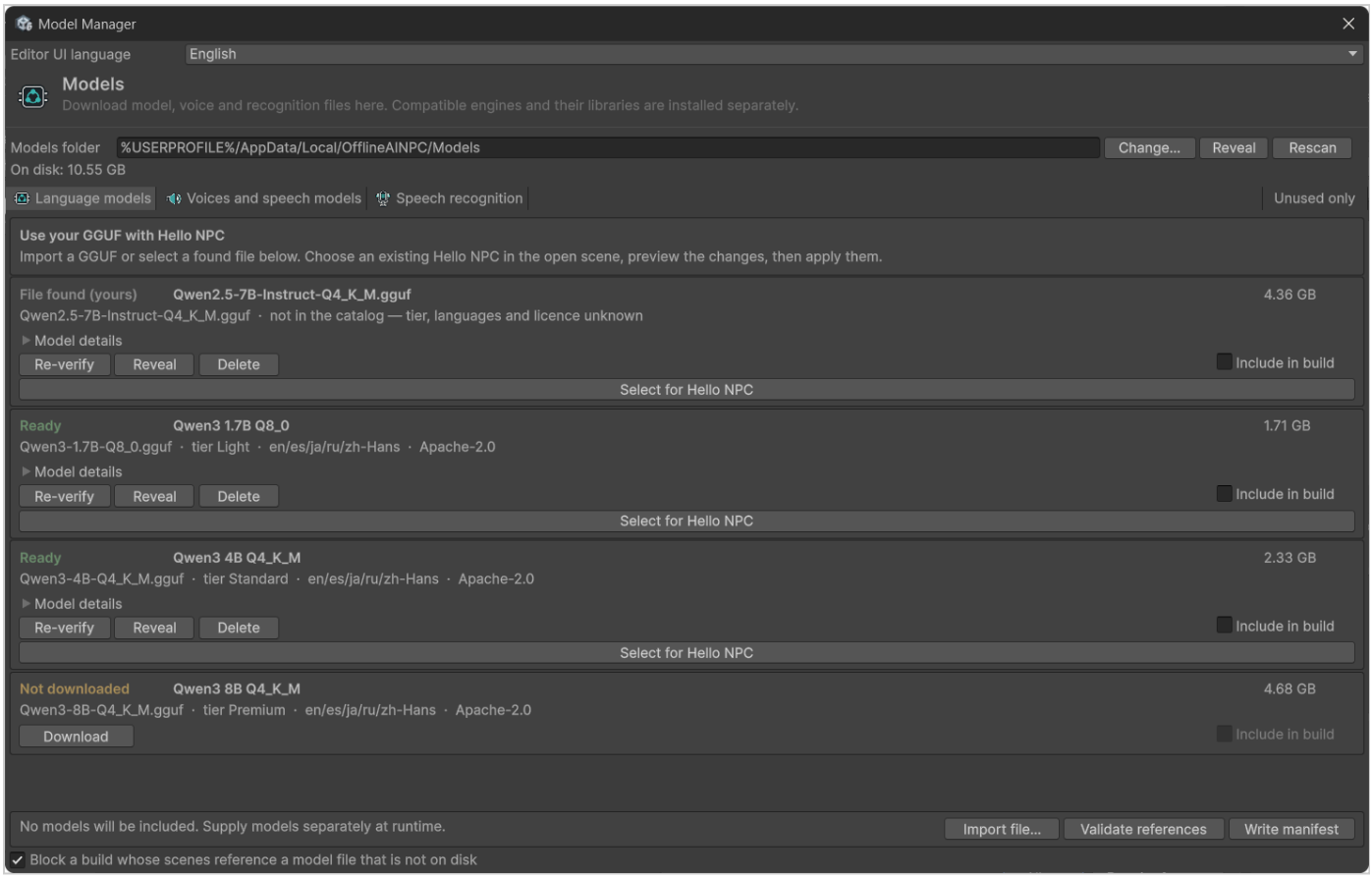
After installation, the optional **Tools ▸ Offline AI NPC ▸ Advanced ▸ Compatibility fixes** window offers **Preview compatibility changes** → **Apply compatibility changes**. Preview shows the files to change and keeps existing project tuning in other source files. Apply keeps script GUIDs and the installed native libraries. For a package installation, it selects a local copy in `OfflineAINPCDependencies` inside this project; retain that folder with the project. For an Assets or embedded package installation, it updates only the listed files and saves the previous bytes. **Undo compatibility changes** restores the recorded dependency. An unknown version or an edited patch target is preserved; use the installation guide before choosing a different dependency. A working installation does not need to be reinstalled. No manual changes in `Library` are needed.

The plugin compiles without either dependency: their features stay switched off. `Check my setup` identifies a missing dependency or an LLM for Unity older than 3.0.3. The plugin reads its own JSON (`OfflineAINPC.Core.Serialization`); its core does not require Newtonsoft.Json. LLM for Unity declares Newtonsoft for package installs; for an Asset Store folder, add `com.unity.nuget.newtonsoft-json` through Package Manager.

Both upstream packages use **Unity UI** (`com.unity.ugui`) without declaring it. If your project template omitted it, add *Unity UI* in Package Manager to resolve their `UnityEngine.UI` and `EventSystems` compile errors.

2. A model

`Tools ▸ Offline AI NPC ▸ Model Manager`.



Each **catalog** entry states its tier, its languages and its licence before you download anything. **Ready** means the file is present and no size or known checksum mismatch was found. A periodic scan does not hash the whole file or prove that its model will start. Expand **Model details** to inspect a GGUF header; use **Re-verify** to check a catalog checksum when one is available.

A file you dropped in yourself shows as **File found (yours)**. Its tier, languages and licence are unknown unless it belongs to the catalog. Being found does not make it the active model or prove that it works with Hello NPC. The Welcome window counts these files separately. Choose the optional direct-assignment route below or configure a matching manifest/host, then check your setup.

Welcome refreshes after a Model Manager operation succeeds, and when you come back to it after its model files changed. If files changed outside the Editor, use **Refresh status** on Welcome's Model card or **Rescan** in the Model Manager. A failed refresh keeps the last results; refreshing never runs a model or starts a download.

For tier startup, choose a preset for your language and tier and download one compatible language model. A preset recommends files; it does not rewrite the scene manifest or select an active LLM host model. Leave voice and speech-model downloads for the optional speech step. The manager checks the declared size and verifies the hash when the preset carries one, so an incomplete file is caught before publication; the Console says "downloaded and verified" only when a SHA-256 was checked. A download that stalls for 60 seconds or loses its connection, or that the server answers with 408, 429 or a 5xx error, is retried automatically up to three times — after 2, 4 and 8 seconds, or after the server's

`Retry-After` up to two minutes — and continues the partial file; after that, **Resume** continues it. Presets carry their licence, and the default line-up is chosen without a separate visible model-credit requirement. Keep the licence texts and applicable notices when redistributing the files; see Licensing.

Cancelling during checksum verification keeps the partial and leaves the installed file unchanged. A late verification cannot install a cancelled attempt. Verified bytes replace an existing file atomically; if replacement is unsupported or blocked, the old file and partial are kept. **Re-verify** hashes on a worker and can be cancelled; closing the window cancels that verification too. No new model is downloaded by Re-verify.

For the matching manifest/host route, use **Import file...** in Model Manager to copy an existing `.gguf` into the right subfolder. To copy it manually, place it at `<models root>/llm/<filename>`; the window shows the models root. Use the exact filename the manifest expects. The locator searches that path, then `<persistentDataPath>/llm/<filename>`, then `StreamingAssets` (including its subfolders). Configure the matching manifest/host before relying on this search; finding a file does not bind it to Hello NPC. The optional direct-assignment route below uses an absolute LLM path instead.

For that locator route, Play Mode includes the machine-local folder selected in Model Manager. The `OFFLINE_AI_NPC_MODELS` environment variable still takes priority. Your editor preference is not embedded in a player build; players use their environment/default root and the two game locations above. Catalog download links accept Hugging Face file pages as well as repository pages, and preserve URL query parameters when resolving a direct file.

Optional: use your own GGUF with Hello NPC

Use this route for a local `.gguf` you want to assign directly to an existing Hello NPC. Finding or importing a file does not select an active model. Work outside Play mode, with compatible LLM for Unity installed and the character's Agent and local LLM already assigned.

1. In Model Manager, use **Select for Hello NPC** on the found-file row. For a file elsewhere, use **Import file...**, wait for completion, then **Select imported GGUF for Hello NPC**. Import copies the file; it preserves the original and refuses an existing destination. Selection alone leaves scene bindings unchanged.
2. Choose the existing Hello NPC in the open scene. This route requires a dedicated Agent/LLM binding. Shared references, ambiguous bindings and unsupported startup owners are refused; it does not create another Agent or host for you.
3. Press **Preview application** and read the current file and proposed changes. Preview reads the configuration only: it does not start or download a model, change voice/STT settings or edit the scene.
4. Press **Apply to this Hello NPC** only when that preview is the change you want. Apply writes the absolute file path to `LLM._model`, enables that LLM, and disables only its scoped linked `TierBootstrap` for manual startup. Other settings and references, presets, catalogs, manifests, defaults and voice/STT stay as configured. Save the scene when ready. Unity **Undo** restores the previous scene configuration.

A preview is used once. Relevant file or scene changes, rebinding, Undo or closing the window can invalidate it; create a fresh preview after resolving the reported problem. The file check uses a bounded GGUF prefix and file identity, length and timestamps. It is not a whole-file checksum or proof that the backend can load the tensors. Changes that retain the same identity, length and timestamps may escape that version check. Preview and Apply do not prove native compatibility or startup; check the actual LLM startup and a typed reply in Play mode.

The applied absolute path is **machine-local**. Configure the model path for your player and build inclusion separately, then check both in the built player on another machine. **Include in build** copies the file; it does not rewrite `LLM._model` to a player path. The matching manifest/host route above remains an alternative when you want the plugin's tier selection and model-location search.

Including models in a build

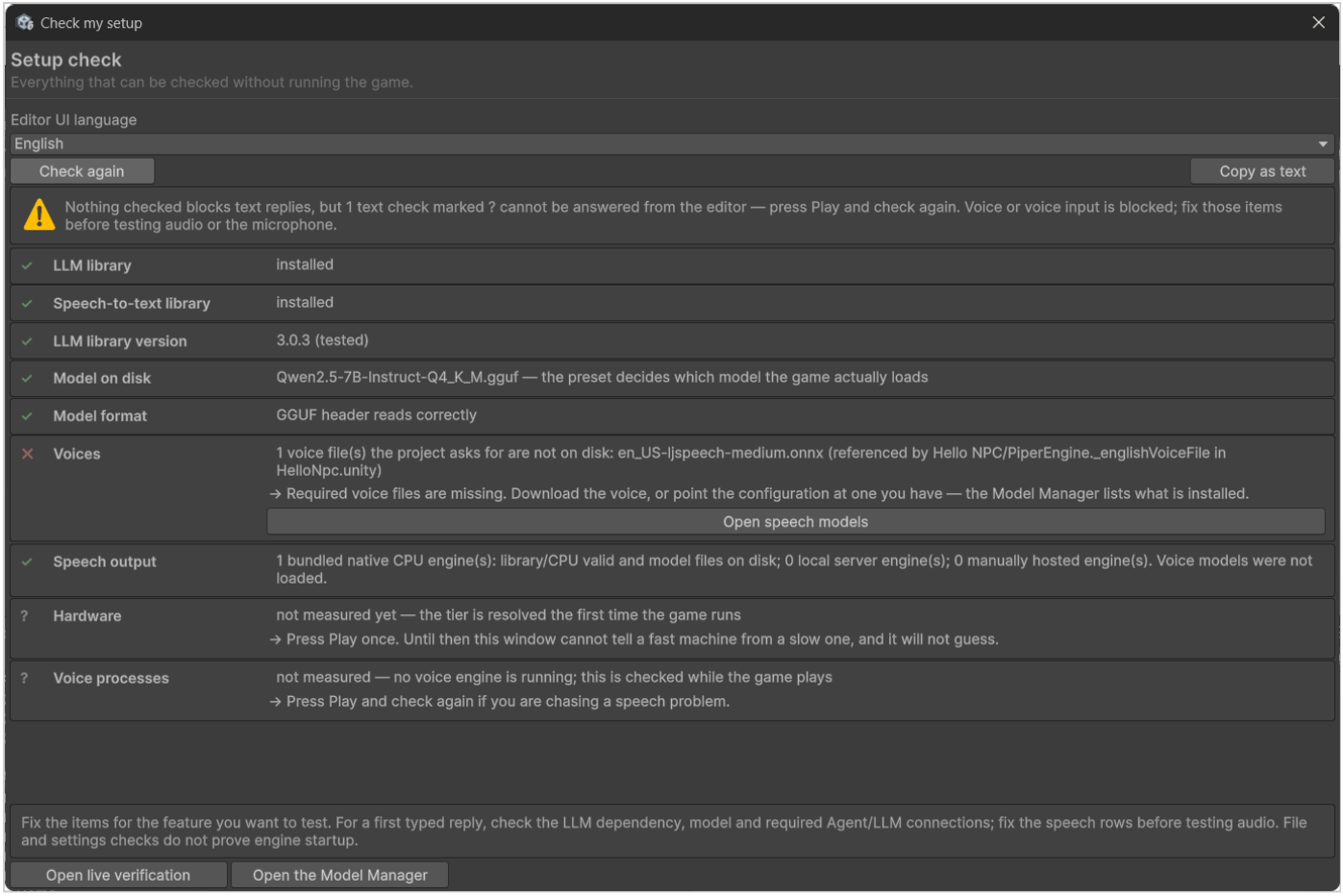
Model Manager's inclusion list stages required files into `Assets/StreamingAssets/OfflineAINPC` only for the build. A Piper `.onnx` voice includes its adjacent `.onnx.json`; both must exist. The size preview counts this same file set. For Piper, this copy alone does not configure its fixed runtime voice path: provide the pair in `Assets/StreamingAssets/piper/voices/` for the built-in driver. Files placed there ship with Unity's StreamingAssets independently of this inclusion list. A journal in `Library/OfflineAINPC` identifies unchanged copies that cleanup may remove. Existing or modified files and metadata are preserved, including manually placed models; conflicting destinations fail with a reason instead of being overwritten. After a failed or cancelled build, the next build recovers only confirmed owned copies. Do not delete the whole staging folder to recover: it may also contain your files. Inspect the named conflict or changed copy before retrying.

3. Check the text setup

In **Welcome**, press **Open Hello NPC sample** and save your current scene if prompted. This opens the already-wired `Samples/HelloNpc/HelloNpc.unity`; you do not need to create a controller or wire a microphone.

`Tools ▸ Offline AI NPC ▸ Check my setup`.

Open the sample before this check so the scene report includes its actual connections. Use **Check again** after opening it. Hello NPC requires **Agent** and the agent's **LLM**. Missing **Audio** or **Engine** connections appear in **Optional scene wiring** as warnings; complete them when you want speech output. A connection check does not confirm native generation.



The report checks dependencies, models and optional speech features separately. For the first typed answer, fix the red items that affect the language model: the compatible LLM for Unity dependency, the selected model and its references. Read each row's reason and suggested action.

A red speech output, voice or codec row is a real error for speech output. It does not prevent a configured language model from answering in text. Fix those rows before testing audio; a missing or invalid Piper `.onnx.json` is reported and that voice request is refused.

4. Your first typed answer

Press Play, type a question into the sample's text box and press Enter. The first success is an answer in the subtitle. With no ready speech engine, the character still answers in text. A voice warning tells you what to fix if you want audio next.

The first model load and reply can be slower than later turns. If no text answer arrives, check the Console and the LLM/model rows in the setup report, then see Troubleshooting. Continue to the optional steps after this works.

5. Optional: speech output

The package includes the Windows x64 CPU Qwen speech library. Its target CPU must support **AVX2, BMI2, FMA, F16C and SSE4.2**. macOS and Linux libraries are not included.

1. In **Tools → Offline AI NPC → Model Manager**, download the **0.6B CustomVoice Q4 talker** and **Q8 codec** under *Voices and speech models* (about 0.9 GB together).

2. Use the scene's **Qwen3-TTS Engine** through **TTS Engine Manager** and its language binding. The sample supplies this wiring; a custom character must select that engine.
3. Leave **External server URL** and **Style** empty. Use `serena` in **Default voice** to keep the same voice across languages. A character's `SpeechRequest` or **Speaker voices** entry can override it. Set the language separately; choosing a voice does not translate the text.
4. In Play, type a short prompt and check the reply's subtitle and speech in the chosen language. Repeat this check in your built Windows player with the model files available.

For a Windows x64 build, the plugin stages `native-build-manifest.json` and five licence notices in `OfflineAINPCQwenNative-Licenses` beside the Qwen DLL. Keep that folder with your player distribution. This step does not include model weights; configure model inclusion separately. A build whose output is not a Player `.exe` — **Create Visual Studio Solution**, for one — is not staged this way: the Console says which files to copy next to the DLL by hand.

The default route loads the DLL in Unity and streams PCM to the existing speech session. It does not start `tts-server.exe`. The native profile is CPU-only with eight threads. **StopAsync** waits for native generation and worker settlement before releasing the context. Cancellation requests cooperative cancellation; it is not an instant-stop guarantee.

A separately launched server and **External server URL** / `AttachTo(url)` are optional advanced alternatives. Keep that server's own model, backend, version and distribution requirements separate.

For a new Piper setup, install Windows `piper.exe` and its required companion files manually in `Assets/StreamingAssets/piper/`. Keep an existing working setup and custom paths. The following window does not download or install Piper.

Optional: connect a local Piper voice

Use this Windows Editor route outside Play mode to make a local voice available to one existing scene engine. It does not connect a character or prove that speech works.

1. In Model Manager, use an existing ONNX voice row; import your local pair first if needed. Press **Connect selected Piper voice**. **Choose voice file (.onnx)** can choose another `.onnx`; its matching `.onnx.json` must be beside it. Import alone copies the pair into `ModelsRoot`, without selecting the runtime voice.
2. Under **Existing scene Piper engine**, select an existing ordinary scene `PiperEngine` that has not started. Leave working, shared or prefab engines as configured.
3. Press **Preview**. Read both source → destination paths and the proposed engine field. Copy puts a permanent pair in `Assets/StreamingAssets/piper/voices/`; matching files already there are reused without ownership or deletion. These files enter builds independently of **Include in build**; selecting both routes can duplicate the pair. Preview does not copy files, change the scene or start Piper.
4. Press **Apply to this engine** only for the preview you want. Apply changes only the eligible English/Spanish fallback field; it does not rewire the character, change its language or rewrite catalog, router, flags or secondary-cast settings. Save the scene deliberately. **Undo** restores engine fields, not the permanent voice copy.

If setup refuses: use the reported reason. This route is for an unused, non-prefab scene engine with at most one loaded component reference. A custom router/path, initialized pool or catalog selecting another voice is preserved. New fallback assignments require the cached engine language to be English or Spanish; an existing catalog selecting this exact file can use it without changing a field. Other cases remain on the manual route. Changed files or settings require another preview. **Cancel** requests cancellation; wait for IO/cleanup to settle before retrying. Changed files or files whose ownership cannot be confirmed are preserved.

Manual route and verification: you can still place `name.onnx` and `name.onnx.json` in `Assets/StreamingAssets/piper/voices/`, configure `piper` and the voice file in the language definition, and assign `PiperEngine` to Hello NPC's **Engine** field yourself. Keep custom routing and paths if they already work. See Languages for the pair rule and your own engine. License, provenance and voice language remain **Unknown** in this window: review the exact voice's evidence separately. Successful Apply and file checks do not prove character binding, native startup or audio; other platforms remain unverified.

Run **Check my setup** again, fix the speech rows, then ask another typed question and verify audio as well as the subtitle. File checks do not prove that a native engine will start.

6. Optional: speech recognition

Only add microphone input after the typed path works. The package ships push-to-talk and a Whisper recognizer; a recognized line goes to the character the way a typed one does.

1. Install **whisper.unity**: *Window ▸ Package Manager ▸ + ▸ Install package from git URL*, then paste `https://github.com/Macoron/whisper.unity.git?path=/Packages/com.whisper.unity`. See Dependencies.
2. In Model Manager, open **Speech recognition** and download the model your preset names: for English `ggml-tiny.en.bin` (Light) or `ggml-base.en.bin` (Standard); for English Premium and every other language the multilingual `ggml-base.bin` or `ggml-small.bin`. An `.en` model recognizes English only.
3. Select the Hello NPC character — the object with *Hello Npc Character* — and add **Add Component ▸ Offline AI NPC ▸ Push To Talk** and **Add Component ▸ Offline AI NPC ▸ Whisper Speech Recognition**. Push To Talk adds its own microphone recorder and uses the recognizer on the same object. Whisper Speech Recognition loads the file the preset names for the current language and tier, or the one you type into *Model File Name*. With a Tier Bootstrap in the scene it waits until the tier is settled; a press made before that is not sent, and the screen says why. For runtime English/Russian switching, download a multilingual model such as `ggml-small.bin` and set **Model File Name** to it before entering Play. The loaded model stays in use for the session; a Language Definition's **STT model id** does not replace it.
4. Press Play, click outside the text box, hold **V**, speak and release. The screen shows that it is listening, then recognizing, then what it heard — sent as if typed — or why that press was not sent.

The key is ignored while an IMGUI text field has keyboard focus. For a TextMeshPro or uGUI input field, connect its focus state: `talk.IsInputBlocked = () => inputField.isFocused;`. For a controller button, untick *Listen To Key* and call `Begin()` and `End()` from your own input. In your own controller, subscribe to `PushToTalk.Heard` — or wire the *On Heard* event — and pass the line to the method a typed line goes to; `Dropped` reports why a press produced no line (`VoiceInputDrop`). Another recognizer plugs in as a component implementing `ISttEngine` next to Push To Talk.

The recorder uses the system microphone by default. For a microphone selector in your game, list inputs with `SharedMicLoop.GetAvailableDevices()` and assign an exact name to `talk.Recorder.SelectedMicDevice`; an empty name restores the system default. If a named input is unavailable, capture reports an error instead of silently using another device. Changing the input discards an unfinished take. For an input meter, use `talk.Recorder.TryGetInputLevel(out float rms)`; `CaptureProblem` explains capture failures.

You can also configure an existing component in code:

```
talk.Key = KeyCode.V;
talk.LanguageCodeOverride = "en";
talk.Heard += SubmitQuestion;
```

An empty `LanguageCodeOverride` follows the game's language. An override changes the recognition request for this control; it does not change the game's language, download weights or turn an English-only model into a multilingual one. `Recognizer`, `Recorder`, `ListensToKey` and `MinimumHoldSeconds` are also configurable in code. Call `Discard()` when opening a pause menu to cancel this control's recording or recognition. The `RecordingStarted` and `RecordingEnded` events drive recording indicators; the latter reports `Released` or `Discarded`, before recognition finishes. Unsubscribe your handlers when their owner is disabled. `IsInputBlocked` applies to the automatic key shortcut; your button handler must check its own eligibility before calling `Begin()`.

For player-facing feedback, `VoiceInputFeedback.GetMessage(drop, languageCode)` supplies short messages in English, Russian, Spanish and Simplified Chinese, with English as the fallback. `NoSignal` means the captured samples were all zero; the message suggests checking the selected microphone, mute and access settings without claiming a specific cause. Quiet audio is reported separately as `TooQuiet`.

Two things that are not obvious and will cost you an evening each:

Silence is transcribed as words. Whisper hallucinates on an empty recording — "Thank you for watching" and similar. Without a gate, a stray key press starts a conversation. Push To Talk runs two: `UtteranceGate` checks the recording's energy before recognition is paid for and the transcript against the known phrases after it (with the recording's RMS, an audible "okay" gets through); `TranscriptCleaner` strips the bracketed sound-words recognition adds. Both are pure C#, for your own input path too — see Troubleshooting.

Any word can be a trigger. If you route on keywords, match whole words. In a script without spaces — Japanese, Chinese — a `\b` boundary does not exist, so compare against the exact token the recogniser returned rather than a substring.

7. Your own character

After the sample answers, use `Samples/HelloNpc/HelloNpcCharacter.cs` as the canonical controller to customize. Preserve the way it refuses a turn while another runs, its callback ownership and its cleanup on disable. Dialogue explains the guarded wiring; its backend and UI methods are explicit integration placeholders.

An `AudioSource` is for optional audio. Add `NpcStateComponent` and `PerceptionQuery` when you need state or perception, rather than to obtain the first answer.

Give the character a persona: who it is, how it speaks, a handful of facts. Keep it short. Everything in the system prompt is re-read on every turn, and a small model does better with five clear facts than with two paragraphs.

Adding your own assembly

If your game code lives in its own assembly definition, reference `OfflineAINPC.Runtime`:

```
{
    "name": "MyGame.Characters",
    "references": [ "OfflineAINPC.Runtime" ],
    "autoReferenced": true
}
```

For a test assembly alongside it:

```
{
    "name": "MyGame.Characters.Tests",
    "references": [ "MyGame.Characters", "OfflineAINPC.Runtime",
                    "UnityEngine.TestRunner", "UnityEditor.TestRunner" ],
    "includePlatforms": [ "Editor" ],
    "overrideReferences": true,
    "precompiledReferences": [ "nunit.framework.dll" ],
    "defineConstraints": [ "UNITY_INCLUDE_TESTS" ],
    "autoReferenced": false
}
```

`OfflineAINPC.Runtime` references **nothing but the engine** — not your game, not LLMUnity, not whisper.unity, and no third-party library. That is what makes it portable between projects, and it is why an import cannot fail on something you have not installed.

Where to go next

You want	Page
A character who notices the room	Perception
Moods, needs, relationships	State — start at the short path
A character who *does* things	Actions
Another language	Languages
To know what will bite you	Limitations
An AI coding agent testing the character with you	Agent tools (MCP)

- Defaults and your existing project
- The short path
- One reply without speech
- Stop buttons and pending generation
- Optional: assign your own GGUF to Hello NPC
- New MCP entry points
- Reviewed MCP changes
- Reply analysis context
  - Qwen's local route changes in 1.1
- Optional: connect a selected Piper voice
- Keep Piper, or configure Qwen3-TTS deliberately
- If the project has compile errors after importing
- Custom speech and subtitle code
- Existing saves and model inclusion settings
- Speech callbacks and the starter sample
- Custom verification adapters
- Editor language and diagnostic reports
- Service menu items moved under Advanced
- Optional features and platform checks
- Before replacing your working copy
- Published utility APIs
- Compatibility of 1.0 editor helpers
- Custom preset catalog and downgrade
- Build selections and dependency discovery
- Microphone recordings and stop listeners
- Prompt inspection, object identity and custom model inputs
- Check custom state and trait numbers
- Language of the sample screens

Upgrading from Offline AI NPC 1.0

Upgrade guide for version 1.1. This page describes updating the published 1.0 package to 1.1. Read the changelog for the version you install and keep a backup of your customized project.

Defaults and your existing project

New speech presets use the same 0.6B CustomVoice Q4 talker and Q8 codec at every LLM tier. Leave **External server URL** empty to use the bundled Qwen CPU DLL. Larger talkers and external HTTP/GPU setups are advanced alternatives to check on your target hardware. All built-in Qwen TTS presets use `serena`, so switching the dialogue language keeps the same speaker.

The default non-thinking reply profile allows up to 128 generated tokens and carries a one-sentence limit. The limit does not require a 128-token reply. The token ceiling is written to every agent of the LLM when the model starts. The sentence limit is applied by the code that delivers a reply: a **Hello NPC Character** applies it when its **Reply Limits** is **Reply Profile** or **Current Language**, and as shipped speaks every sentence within the token ceiling. Custom presets remain yours. Sampling values typed on an `LLMAgent` take effect in Play only with **Override agent sampling** turned off on the LLM host; it is on by default.

Whisper supplies the optional microphone path in 1.1. Use a multilingual Whisper model when dialogue switches between languages. VoiceDesign is not part of this version.

The short path

1. Save your project and make a version-control commit or a complete backup. Keep copies of your own scenes, prefabs, language definitions, axis catalogs and action handlers, especially anything you edited inside the package folder. Preserve existing player memory/state data and NPC ids so you can check them after the upgrade.
2. Import the update into a copy of your working project first. Review which package assets will be replaced. Keep your working speech engine and voice files for this pass.
3. Use **LLM for Unity 3.0.3 or newer**. This was the documented minimum in 1.0; the newer integration now enforces it for recognized installations. Versions 3.0.1 and 3.0.2 are below that floor too. An unrecognized version is not proof of compatibility.
4. Open your own character scene and run **Tools ▸ Offline AI NPC ▸ Check my setup**. Fix the language-model items required for typed dialogue, then test a typed reply. Microphone input and voice output are optional for this first check. Keep speech errors visible and resolve them before testing speech; preserve a working Piper setup.
5. Check an ordinary reply, an interrupted reply, each language you use, and a restart with existing memory/state. Then make and test a player build with your model inclusion settings and the speech executable/libraries you plan to distribute.

Welcome reports files from its last scan, not proof of active selection or native startup. Found files, missing files, incomplete downloads, established defects and unavailable verification are separate states. The language-model recovery buttons open the LLM tab and clear its unused-file filter; they do not choose a model or start a download. Refresh the status after changing files.

One reply without speech

Use `HelloNpcCharacter.SayText(string)` for a text-only turn, or `TrySayText(string)` to keep input when submission is rejected. For an assigned `hello`:

```
if (hello.TrySayText(playerLine))
    playerLine = "";
```

`true` means submitted, not completed. Blank input, required model setup, an inactive character or a busy/still-settling turn can reject it. Subtitles, actions, `Finished` and ordinary UI conversation history use the existing pipeline. This turn does not check speech readiness, create a speech session/clip or request synthesis. Existing `Say(string)`/`TrySay(string)` and UnityEvent bindings remain supported. The component's `Start` can still

inspect/start an assigned engine: leave Engine empty if the scene should have no assigned speech engine. A typed or spoken turn and an MCP `say` take turns on the same character — while one runs, the others are refused — and a cancelled reply keeps the character busy until the model's running task has actually finished.

Stop buttons and pending generation

`HelloNpcCharacter.CancelReply()` and `StopSpeaking()` are additive; `Say(string)`, `TrySay(string)` and existing event signatures remain supported. Both Stop methods cancel the whole reply while keeping the component enabled and the borrowed engine running. Wait for `!IsBusy` before retrying a kept question: a cancelled generation or repair keeps the character busy until its task has actually finished. Cancellation does not roll back actions that already ran. Optional speech readiness and listener failures are contained; a missing AudioSource stays subtitle-only without a clip or audio-onset claim, while the existing synthesis iterator is still driven.

Optional: assign your own GGUF to Hello NPC

This is an optional editor route, not a required scene migration. Updating the package, finding/importing a GGUF, selecting it or previewing it does not automatically rebind an existing scene. Keep a working setup until you explicitly choose another one.

In Model Manager, select the found or imported file for an existing Hello NPC, use **Preview application**, then explicitly **Apply to this Hello NPC**. Apply sets `LLM._model` to an absolute file path, enables that LLM and disables only its scoped linked `TierBootstrap` for manual startup. Other settings and references, presets, catalogs, manifests, defaults and voice/STT are preserved. Preview does not edit the scene, start/download a model or change voice. Save the scene deliberately; Unity **Undo** restores its previous configuration. Shared/ambiguous bindings and stale previews are refused.

The prefix and file-version checks are not a whole-file checksum or proof of native tensor compatibility. The applied path is machine-local: configure the player model path and build inclusion separately. Including the file does not rewrite `LLM._model`. Check real startup and a typed reply in Play and in the built player. Follow the own-GGUF steps for requirements and refusals. An explicit matching manifest/host remains the alternative for tier startup.

New MCP entry points

Start a new agent session with `project_overview`, select one exact character and use `inspect_character` for its configuration and declared bindings. `setup_plan` explains the first typed dialogue without starting Play or a model. Default reads do not probe model/voice readiness or scan files. Request `probe_runtime` or `check_files` explicitly when you want those checks; runtime getters may initialize a manager or restore flags. Projected/filtered snapshots report exclusions and unknown evidence, and continuation keeps the captured snapshot. Existing tools, unique selectors and custom character interfaces remain supported; implementing the passive inspection source is optional.

Hello NPC now implements `ILiveVerifiable` directly. Its screen and MCP use the same assigned Agent, take turns on the character, and run the same actions and presentation. `say` with `speak:false` does not create a speech session or read voice readiness. Cancelling an MCP operation cannot cancel a later reply from the screen; the character stays busy until the model's running task has actually finished. Verification history/reset is in memory; saved history files are

preserved. Language sweeps are refused because this sample has no persona/voice switch. Dispatcher calls are observed; durable world mutation is not. Native startup and actual replies still require verification. Voice and microphone are optional for the first typed check; keep working Piper files.

Reviewed MCP changes

The four additive tools `preview_poi_change`, `preview_state_change`, `apply_change` and `verify_change` provide a reviewable one-target path. Existing `set_state` and `mark_point_of_interest` remain immediate calls; their `dry_run` option is still unsupported. Select an existing exact ref, review a complete preview and apply its `plan_id` once. State preview is passive and refuses an uninitialized model instead of loading its store. Delta keeps Game trait sensitivity; an absolute value does not scale with sensitivity.

Apply reports the value it wrote separately from the state seen after callbacks ran. Retrying a token that was already applied returns the first receipt and never writes again. If an exception escapes the apply, whether the write happened is unknown — there is no receipt and nothing is rolled back. Call `verify_change` with the same `plan_id` and read `get_state` before deciding on another delta: repeating a delta after an unknown outcome can apply it twice. Relevant changes, scene/model replacement, reset and expiry can invalidate plans. POI uses Unity Undo outside Play and does not save a scene; State is model-only without Unity Undo. A State Store can flush persisted values outside Play too. Neither apply nor verify proves durable saving, and neither calls `SaveNow`. Read Agent tools for limits and recovery.

Reply analysis context

`check_reply` adds `analysis_context` while preserving existing arguments and result fields. It checks the schema at analysis time, not the original generation prompt or grammar. A perceiver uses its current cached targets without refreshing the world. An earlier reply may fail after a quest stage, permission or target changes. Equal target-registry counters still mean `unknown`; `changed` only reports that counter's change during this call. Filters and permissions are not snapshotted. Accept the new JSON object and keep gameplay checks in your action handler. See Agent tools.

Qwen's local route changes in 1.1

An empty **External server URL** now selects the included Windows x64 CPU native DLL. You no longer need to build or copy a Qwen `tts-server.exe` for this ordinary route. Download the talker and codec in Model Manager as before. A nonempty URL and `AttachTo(url)` retain the advanced HTTP route and the separately launched server's ownership.

Existing `Say` / `TrySay`, serialized engine references and public `StartAsync` remain. **An existing Piper scene keeps Piper**: updating does not replace its executable, voice pair, casting or binding. New components use the bundled CPU DLL with External server URL and Style empty; existing serialized settings are preserved. Cancellation awaits actual native work before context free. Test this DLL, model pair, language and Unity backend in Editor and your built Player before shipping your game.

Optional: connect a selected Piper voice

This is an optional Windows Editor route, not a required migration. Updating the plugin or importing a pair leaves a working voice and its scene references in place. In Model Manager, use **Connect selected Piper voice**, select an unused ordinary scene engine, then **Preview** and explicitly **Apply to this engine**. The window can copy the local pair

permanently to `StreamingAssets/piper/voices` or reuse a matching pair there, and configure only that engine's default voice. It does not wire Hello NPC, change the character's language, install Piper or prove readiness.

Undo restores engine fields, not the permanent file copy. **Include in build** is a separate staging choice and can add a duplicate pair. Shared/prefab engines, custom routers/paths and overriding catalogs stay on their existing manual route; new fallback assignments are limited to the cached English/Spanish language. An exact catalog voice may need no field change. Keep working settings and resolve the reason for a refusal. Follow the four connection steps for manual installation, cancellation and file-preservation limits, then check actual character wiring and speech separately. License, provenance and voice language remain Unknown; names or valid files do not establish distribution permission.

Keep Piper, or configure Qwen3-TTS deliberately

Piper remains a supported engine. Updating the plugin does not require converting a working Piper integration to Qwen3-TTS. Keep both the `.onnx` voice and its `.onnx.json`, check that the language definition still selects `piper`, and verify the engine binding in your scene. The built-in language definitions now select Qwen3-TTS; in 1.0 they selected Piper, and VOICEVOX for Japanese. A Language Set of your own keeps the engines it names. A 1.0 scene keeps its speech manager's binding rows; without rows, every language Piper has a voice for still goes to its Piper field, and the Console says which. The development samples use Qwen3-TTS; they are examples, not replacements for your own character scene.

For a local voice, **Model Manager ▸ Import file** copies the `.onnx` and its adjacent `.onnx.json` together. Keep both originals in the same source folder. The import checks both before copying and refuses existing destination files; it does not replace a working voice or attach another voice's config. Read the error if an import fails: changed files or files whose ownership cannot be verified are preserved for inspection. Copying and verification run in the background; watch the inline progress or use **Cancel import**. Cancellation waits for ownership-safe cleanup. Closing the window requests cancellation too; a normally completed cleanup can report preserved paths in the Console. Wait for an operation to settle before retrying a conflicting download, import or deletion. On Windows, choose an ordinary drive or UNC path for these operations. Device/extended namespace paths and components ending in a dot or space are refused as ambiguous aliases.

Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no separately started HTTP server, Python environment, speech executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.

An existing custom `IPiperVoiceRouter` continues to own casting. The legacy manager retains its published English, Spanish and Russian Piper routes even with empty voice fields. Configure an explicit binding or Piper voice for additional languages; the router interface has no query that proves support for every language. Voice readiness and its actual files still need checking before speech.

The Qwen client sends the requested language separately from its speaker. If you use the optional external HTTP route, the server build must support per-request `language`; older servers ignore it and keep their process-wide setting. Check short replies and a language switch with that server. Updating this Unity package does not rebuild or replace a manually installed speech server.

If the project has compile errors after importing

Errors about `LLMAgent` or `LLMClient` in `OfflineAINPC.LLMUnity` can mean an old loose-folder LLM for Unity install is still paired with the `LLMUNITY_PRESENT` scripting define set by 1.0. With compilation already broken, Unity may not load the updated detector that would remove it.

Update LLM for Unity to 3.0.3 or newer. If you are recovering before that update, remove only `LLMUNITY_PRESENT` from **Project Settings ▸ Player ▸ Other Settings ▸ Scripting Define Symbols** for the affected target and let Unity compile. Keep your other symbols. The optional integration stays unavailable until the dependency is compatible. Restore the dependency and verify references on the character before continuing.

Custom speech and subtitle code

Your integration	What to check
`ITTSEngine.Synthesize(string, int, Action<AudioData>)`	The original one-buffer contract still works. Streaming is an additional interface, not a required rewrite.
Existing `StreamingSpeechSession` constructor	The original signatures remain available. Your call can stay as it is.
Subtitle callback also appends a transcript or counts lines	It can now receive the same text to extend its deadline. Use the new overload's `onSubtitleExtended(seconds)` callback for deadline-only updates. Durations start at the callback and use unscaled seconds.
Interruption checks only `AudioSource.isPlaying`	Use `session.IsSpeaking` as well: a paused streaming reply is still interruptible. Call `Cancel()` to stop the whole reply.
You stop the hosting coroutines yourself	Dispose the session too, so its current clip and pending work are released.
A custom streaming engine	Deliver ordered chunks with one sample rate/channel count per request and honor cancellation. Test pauses, partial failures and scene changes.

See Dialogue for wiring and lifetime details. Recheck code that assumes every subtitle callback is a new sentence, even if it previously worked with a one-buffer engine.

Existing saves and model inclusion settings

Keep the same NPC ids, state keys and `memory_` keys. This update does not require converting the existing state or conversation-memory JSON formats. A custom state store can keep its `IStateStore` implementation. Nested `SaveNow()` calls made by its callback are coalesced; changes made during that write remain pending for the next debounce or pause/disable/quit flush. A nested request gets one deferred flush after the current read/write returns, even if its callback disabled the character. Continuing callback mutations remain pending without an endless synchronous loop. Changes smaller than the presentation-event threshold count as pending saves too. A callback cannot write over an unfinished read, and a superseded read cannot replace a newer store selection or explicit `Import`.

If your synchronous memory compactor or key-value store callback appends or clears the same character's memory, the requests now run in order after the current mutation. Queued appends keep the rolling cap without calling the compactor again in that batch. Inside such a callback, a queued request has not reached the store yet; inspect the final memory after the outer operation returns. Keep summarization cheap and synchronous.

Custom callbacks must not create unbounded memory changes. While one outer call runs, the callbacks it triggers can append or clear memory up to 128 times in all, across every store and character, with at most 64 different character memories nested at once. Further requests are refused with one warning; requests for the same character still run in order. The next ordinary call starts with a fresh budget. Existing memory keys and JSON are unchanged.

Older per-user `OfflineAINPC.IncludeInBuild.<assetId>` choices migrate to `ProjectSettings/OfflineAINPCModels.asset`. Commit that project-settings file once you have verified the intended inclusion choices. If saving it throws, the old key and previous in-memory value remain available for retry; fix the write problem and select the setting again. Check the resulting file and model inclusion in a player build.

Regenerating `required-models.json` now refuses an unreadable file or a schema from a newer plugin instead of replacing its metadata. Preserve the file and use a compatible plugin, or deliberately recover it from your backup before regeneration. The original unversioned manifest and understood versions remain accepted. Native import and the resulting player build still need checking after the file has been saved.

`required-models.json` is now saved to `ProjectSettings/OfflineAINPC/`, next to the other model choices; inside the plugin folder, a remove-and-reimport of the plugin deleted it. A 1.0 manifest at the old place is still read until a new one is saved; after that the old copy is no longer read and can be deleted. Commit the new file.

Speech callbacks and the starter sample

Existing speech constructor signatures and callback types remain unchanged. Subtitle and subtitle-extension callbacks that throw are reported without ending the voice session. Call `Cancel()` or `Dispose()` explicitly when you want to interrupt a reply. Cancelling from `OnFirstAudio` no longer shows that reply's stale subtitle. Disposing an unfinished playback iterator cancels its pending synthesis before `OnFinished`. A completed session's later disposal cannot stop the next reply on the same source.

Hello NPC can answer in text when its Engine field is empty. Assign a speech engine to add voice; an absent speech engine no longer blocks a ready language model. Setup check offers direct buttons to the relevant model window or setup guide beside supported problems. Opening these helpers does not change settings or start downloads.

`HelloNpcCharacter.Say(string)` keeps its void signature, including assignments to `Action<string>`. The optional `TrySay(string)` reports whether input was accepted; the sample HUD keeps a rejected question while the model starts or another reply runs. Acceptance is not completion: use the existing reply events for the eventual result.

A timed-out startup measurement is retired and its token-limit override restored once, but the agent remains reserved until the original generation and tokenization settle. Hello NPC and the supplied live adapter refuse generation while that reservation is pending. Cancellation alone cannot prove native completion. Custom `ILlmHost` code keeps its existing interface; implementing the additional `ILlmMeasurementLifetime` allows the cascade to retire a pending task. Code of your own that calls `LLMAgent.Chat` directly must refuse too while `LlmMeasurementAdmission.IsPending(agent)` is true. Native teardown waits for the host measurement to settle; an indefinitely pending backend is not declared ready.

TTS switch observers are contained per listener. A nested `SwitchForHost` request joins the switch already running instead of starting another engine; its own iterator does not wait for that switch. Check the final engine and language readiness when a custom observer queues a change, and dispose an abandoned host switch iterator so it releases the switch and its paired notification.

A running switch carries out at most 128 further switches that observers ask for; an endless chain is refused with one warning. A burst of pending requests still ends at the latest choice, and the next switch from outside starts afresh. Old cached benchmark speeds and layer-growth suggestions are ignored when their measurement is out of date; layer counts known to work and markers of unfinished attempts are kept.

The startup speed is now the number of tokens that arrive after the first non-empty batch of streamed text, divided by the time from that batch to the end of the reply. When the reply arrives in one batch, tokens cannot be counted or the interval cannot be seen, no speed is reported rather than an estimate. Speeds recorded before this change are not directly comparable.

Custom verification adapters

Existing verification rows retain their JSON-array format. A real legacy measurement without a configuration fingerprint remains stale. A read failure is shown as unknown and retried; it is not an empty history that a new result may overwrite. Unsupported or unreadable whole files are preserved. If saving or forgetting results fails, resolve the reported file access problem and retry. A reply may have played even when its result was not saved; live verification reports that distinction. None of these checks substitutes for listening to your actual character after the upgrade.

`ILiveVerifiable` remains compatible with existing adapters. A warm latency run needs a conversation of its own: implement `IConversationIsolation.TryIsolateConversation(out IIsolatedConversation conversation, out string problem)`. Set the agent's history, any rolling summary and its save setting aside, start an empty conversation that is never saved, generate the run's turns through `conversation.GenerateAsync`, and refuse every other generation until the originals are back. Put them back only after every turn the run started has finished, including after a cancelled run. A turn of the run may add only the player's line and one reply; anything else — a reply generated on the agent directly, during a turn or between them — was written by someone else, so leave the history as it is, report the conflict and refuse the run's later turns. Start a run only while the character is idle. For an `LLMAgent`, `AgentConversationIsolation.TryBegin(agent, generate, out problem)` sets the conversation aside, keeps the run unsaved, detects the conflict and puts the originals back; it needs Play mode. It does not refuse other

generations itself: while its `Active` is true, your character refuses them, as the supplied adapters do. Without isolation the runner reports an inconclusive run before generation and does not touch the conversation. The runner no longer calls `ResetConversation` or `IConversationResetVerifier`.

`SetLanguage(code, onProblem)` should apply the selected language setting before its first yield, then wait for the voice. The supplied adapters already follow this order. On cancellation, verification can restore the setting immediately and hand the remaining voice switch to a surviving cleanup host. One probe owns the project language until that rollback completes. Cleanup is bounded to 45 seconds and reports failures; an editor reload or quit defers voice startup to the next session. A custom adapter that delays its setting change until after a yield cannot guarantee synchronous rollback during reload; check its ordering and handle the reported restoration failure.

Editor language and diagnostic reports

Choose English, Russian, Simplified Chinese or Spanish in **Welcome** or **Check my setup**. This is an editor preference: it does not change an NPC's language, presets, model selection or a working Piper setup. Available translated guides open in the same language; pages without a translation fall back to English.

Built-in setup results can be displayed in the selected editor language. Existing `CheckResult` fields, check ids, severity and the copied plain-text report remain in their original English format. Custom messages without translation data are displayed as written. Existing code that consumes the report needs no change for this feature.

`CheckResult.Presentation` is optional read-only view data. Unity's public-field JSON shape is unchanged. If your serializer also includes public properties, exclude this property when you need to preserve the old five-field JSON contract.

Model Manager can show `VerificationUnavailable` when a previously checked file's current version cannot be read, for example while another program locks it. The file is preserved; resolve the access problem and re-verify. It is not treated as a usable model or as proven corruption. Existing `ModelAssetState` values 0–6 keep their numbers; custom state switches should also handle this new value 7.

For LLM `.gguf` files, the scan also checks up to 64 KiB of the structural header, including files named in your own catalog. Empty files and proven truncated or invalid headers are excluded as damaged. Unsupported versions or byte order, unreadable files and files changed during checking are unavailable for verification, not proof of corruption. Vocabulary-only containers without LLM tensors are also excluded without being declared damaged. The check supports little-endian GGUF v2/v3, retains stable results for the editor session and does not repeatedly hash the whole file. A passing prefix is not a checksum or proof that the native backend can load all tensors; test actual loading before shipping. Finding a manual file still does not select it for your character.

When the tensor directory fits inside the read budget, the check accounts for `general.alignment` (32 when omitted), rejects padding without weight data and checks tensor offsets against the aligned data area. Ambiguous or unsupported alignment is unavailable for verification, not proof of corruption. The prefix is read through one held file handle. On Windows, sharing rules block concurrent writes or replacement while that handle is open, including when a rescan reuses a cached prefix result. This does not protect against later edits, including edits to the same file that restore its original length and timestamps.

Diagnostic ids and severities remain stable, but human-readable repair wording can be corrected. Use the id for programmatic routing rather than matching a full repair sentence.

Model lookup rechecks a cached path and periodically refreshes missing nested StreamingAssets files, so moving or adding a file need not require a Play restart. Lookup is not a native loading check. Build-scene validation scans enabled build scenes even when other disabled, unlisted or unsaved scenes are open; the explicit open-scene check remains available for those scenes. Prefab and asset checks retain their existing scope.

Service menu items moved under Advanced

Tools ▸ Offline AI NPC keeps the windows at the top: Welcome, Model Manager, Presets, Check my setup, Live verification, Prompt debugger and Agent tools (MCP) with its start and stop items. Everything else is one level down, under **Advanced**: Check my setup (log to console), Live verification — verify everything (console), Validate model references, Check for unaddressable scripts, Re-check LLMUnity installation, Reset to the embedded catalog, Show welcome again after the next reload, Logging and Speak one line (Piper). A script that runs one of them by its menu path needs the new path, and a shortcut bound to one of them in the Shortcuts window needs binding again.

Optional features and platform checks

The new MCP tools run in the Unity editor and start disabled. Enable them only if you want an MCP client to work with the plugin; upgrading a game does not require configuring them.

MCP write calls reject ambiguous NPC selectors and unsupported arguments such as `dry_run`. `set_state` distinguishes refusal from a valid unchanged or clamped value. Keep existing NPC ids/save keys and resolve scene ambiguity before retrying; see Agent tools.

MCP scene references are optional additive selectors: `character_ref` for `inspect_prompt`/`say`, `state_ref` for `get_state`/`set_state`, `perceiver_ref` for `perception`/`check_reply` and `poi_ref` for editing an existing POI. Obtain them from `list_characters` or `list_points_of_interest` and pass a reference instead of the corresponding legacy selector, never both (including explicit nulls). Old unique selectors still work. References are temporary exact-component handles, not save keys; relist after removal/replacement, scene changes or editor/server reset. Historical inspection continuation does not renew them. Memory `npc_id` and `check_reply.character` retain their store-key and speaker meanings. Existing ids and saved data stay unchanged.

MCP consumers must handle unknown readiness explicitly: a failing getter returns `null` instead of a guessed `false`. Read the accompanying status and `readiness_diagnostics`. Known values retain their booleans; an absent voice stays distinct from an unreadable binding. `say` reports model readiness failure promptly and preserves text when only the requested voice cannot be checked. This does not change the public runtime interfaces.

MCP consumers must also follow bounded inspection pages. For `list_characters` and `inspect_prompt`, check `page.complete`; while false, call the same tool with only `cursor=page.next_cursor`. Reassemble marked fragments before treating long values as complete. Continuations describe one captured snapshot, not the current scene. An expired or unavailable cursor requires a fresh first call; it never silently refreshes the data. `max_bytes` defaults to 65536 (8192–262144) for the encoded result including text and structured copies, excluding the protocol envelope. Small results keep their old fields; text keeps the captured summary before a blank line and page JSON, and that summary also counts toward the budget. Continuations repeat it without rereading providers; older clients must learn pagination for large results. Other/custom tool outputs and runtime/save interfaces are unchanged. See Agent tools.

macOS is not verified for the complete upgraded plugin by the publisher. Test the actual native engine build, Unity backend and player on the target platform before promising support for your game. A compiling C# assembly alone is not that validation.

Before replacing your working copy

Confirm that your customized assets, NPC ids and saved memory/state survive; your languages select the intended engines and voices; interruption does not resume stale speech; and the built player locates its included or downloaded models. Keep the backup until these checks pass in your own project. No universal save-data conversion has been verified yet.

Published utility APIs

`ActionParams.TryGetNumber(name, out var value)` still infers a double, and `out _` remains valid. For a float, use `TryGetFloat(name, out float value)` or `GetNumber(name)`. Only code written against the unpublished float overload needs that accessor renamed; published 1.0 handlers require no such change. The one-argument state/perception `EstimateTokens` and two-argument `HardwareProfile.CacheKey` signatures remain present, so old method-group assignments to delegates compile too.

`ActionIdentifiers.ToRuleFragment(string)` keeps its 1.0 mapping and signature. `LatencyMeasurement.Turns` keeps its `string[]` type, including `.Length` and array assignment. Existing integrations using these members do not need a source change.

`ActionTurn` now coordinates grammar ownership for turns using the same `IGrammarSink` instance. Keep that instance shared for one backend grammar slot, and always end or dispose a turn. An old turn cannot clear a newer owner after a reentrant callback. Separate wrappers around the same native slot are not automatically identified. Recursive schema filters and stale state notifications are contained; reset and restore do not emit an obsolete change after a callback replaces the value. The JSON writer refuses cyclic or excessively nested values with an `InvalidOperationException`; overflowing JSON numbers are refused during parsing.

Compatibility of 1.0 editor helpers

These deprecated entry points keep existing source compiling:

- `OfflineAINPC.Core.ExternalProcessLifecycle.Install()` still installs the quit hook. Hooks are automatic; remove the call when convenient. Calling it again does not stop registered hosts.
- `OfflineAINPC.EditorTools.AddressableScriptCheck.RunFromMenu()`, `Scan(string root)` and its public `Problem` fields and `Describe()` remain available, backed by the current scanner. The menu is **Tools ▸ Offline AI NPC ▸ Advanced ▸ Check for unaddressable scripts**.
- `OfflineAINPC.Editor.Packaging.DocumentationAssets.Folder` and `.Images` remain constant strings with their original 1.0 values. The script keeps its path and GUID, so importing 1.1 replaces it. For a relocated package, resolve paths relative to its actual folder.

The public publisher operations shipped in 1.0 are removed from the buyer package: `PackageExport.Export()`, `CollectShippingPaths()` and `IsExcluded(string path)`, and `DocumentationAssets.ApplyImageSettings()`, `FloatDocWindows()` and `Count()`. If your build pipeline called them, select your own game assets explicitly and use

`UnityEditor.AssetDatabase.ExportPackage(paths, destination, options)`; filter and count that same path list. Configure your screenshots through `TextureImporter`, and open your own editor windows with `EditorWindow.GetWindow<T>(utility: true)`. Publisher menus stay in the development repository.

Importing 1.1 over 1.0 does not remove files absent from the new package. The old publisher helper `Assets/OfflineAINPC/Editor/Packaging/PackageExport.cs` can therefore keep its **Tools ▸ Offline AI NPC ▸ Export .unitypackage...** menu. If you no longer need the original 1.0 helper, remove that old file. Keep the imported `DocumentationAssets.cs` compatibility constants and your own editor wrappers.

Custom preset catalog and downgrade

Tools ▸ Offline AI NPC ▸ Advanced ▸ Reset to the embedded catalog now replaces official entries and retains your understood user entries. Keep the preset cache in your backup alongside customized assets. If a newer plugin has written a schema this version cannot understand, cache writes, update and reset are refused with the file path; keep that file and use a compatible plugin rather than deleting it. A malformed cache may be set aside under a unique recovery name. The public `ClearCache()` API still explicitly deletes the whole cache, including user presets; use `TryResetToEmbedded` when you want the preserving reset behavior.

To use the new Serena defaults for all built-in Qwen presets with a saved official catalog, choose **Reset to the embedded catalog**. This retains your user entries. Voice choices in your own Language Set or character requests remain your overrides.

A superseded model is still usable when its file is present. A Piper voice needs its nonempty `.onnx` and readable `.onnx.json`; a green file status does not validate native startup, the scene, redistribution rights or a target player.

Build selections and dependency discovery

An explicit **Include in build** choice is retained when its model becomes missing, corrupted, incomplete or temporarily unverifiable. The build now stops with the selected file and its reason instead of silently omitting it. This also applies to a deleted manual model or a removed catalog entry whose id is still selected in project settings. Restore the model, finish its download/verification, or deliberately remove the choice in Model Manager. An unavailable selected row can be unchecked; a disappeared row has a **Remove from build** action below the list. The broken-reference warning switch does not permit omitting an explicitly selected model. Files that are not selected do not block this inclusion check. The preview uses the same selection rules as build preparation.

No selection means that you must supply models separately for your game's runtime. The plugin does not automatically add a first-run downloader to your game.

Loose LLM for Unity discovery still has a disk fallback when Unity's asset index cannot be read. An unrelated project's `package.json` or `VERSION` is no longer evidence of the dependency's version. A recognized incompatible dependency is still refused; an unknown loose version is still unknown, not a claim that it meets the documented minimum. No automatic dependency installation or model replacement is part of this recovery.

Microphone recordings and stop listeners

`PersistentMicRecorder.StartRecord()` starts a logical take; `IsRecording` is not proof that a microphone is available. The next explicit `StopRecord()` still completes once with a non-null, possibly empty PCM array when capture is unavailable. A capture restart, device loss or component disable invalidates that take instead of mixing positions from different microphone buffers. Disabling/destroying the component does not emit a stop event by itself. If you start a new take after re-enabling, the invalid old take is replaced. Treat an empty chunk as no speech; do not leave your dialogue locked waiting for PCM. Each stop listener is isolated so one throwing listener cannot suppress later listeners. Public recording methods and the `MicChunk` event payload remain compatible.

Prompt inspection, object identity and custom model inputs

Prompt debugger now uses the same English, Russian, Simplified Chinese and Spanish editor preference as Welcome. Changing that preference redisplays the captured prompt; it does not call your prompt provider again or change the NPC language, the test line, model, voice or generation settings. Section names, prompt text and custom error details remain exactly as supplied. Token totals are estimates, not backend measurements.

The character picker selects an instance, even when several characters have the same id. Refreshing retains that instance after a rename or list reorder while it still exists. A failing id getter no longer hides the other characters. An unavailable or failed inspection clears the previous totals. The public `ILiveVerifiable` / `InspectPrompt` contract is unchanged: return null if inspection is unavailable, or a `PromptSnapshot` with a non-null `Sections` list. An empty list is valid. A null optional exclusions list is displayed as empty; it does not require a migration change.

`PerceivedPoi.Id` is captured when that item is created. Renaming a live point of interest does not rewrite an older `PerceptionSnapshot`; the next recompute notices the changed id. `PerceptionSnapshot.Items` remains `IReadOnlyList<PerceivedPoi>` and protects its item list from later array edits. Use the declared read-only interface instead of casting it to a mutable array. The referenced `Poi` remains a live object, not a frozen copy of the scene. Nested registry callbacks keep the newest initialization or enable/disable intent.

A custom manifest can list several models for one tier. Installed candidates are tried in manifest order before moving to a lower tier; `ModelCatalog.ForTier` still returns the first declared entry. An unknown `TierPolicy.maxTier` produces a detected Fallback verdict with a reason; without an explicit tier override, the host is not configured. The existing explicit override priority and supported tier values 0–3 are unchanged.

Custom catalog JSON must use a supported whole-number `schemaVersion`. A fractional version is not silently truncated. Sampling numbers that overflow their destination numeric type are rejected rather than becoming Infinity. Missing fields retain their existing defaults. The cache recovery and preservation rules above still apply; keep your cache backup.

Check custom state and trait numbers

Existing state and memory saves need no format conversion. Keep the same NPC ids and relationship keys. Check your custom axis catalogs and trait presets for `NaN` or infinity after upgrading, including definitions that your code changes at runtime.

The catalog inspector and validation identify nonfinite axis fields. Trait asset validation reports affected fields in the Console. Fix them at the source. Runtime fallbacks keep state finite: an invalid baseline becomes clamped zero, an invalid trait shift becomes zero and a nonfinite trait multiplier becomes one. Nonfinite axis rates disable ticking; nonfinite model delta limits disable model writes. Nonfinite hysteresis uses `0.08`, and bands with nonfinite boundaries are unavailable.

For a broken axis range, new game/model/tick writes are refused. Loading a finite saved relationship or need preserves its number without applying that broken clamp and without replaying change or threshold events. With a repaired valid range, later ordinary writes use the normal clamp; editing the definition alone does not reclamp loaded values. Finite tuning values keep their existing policy. Arithmetic overflow is refused before storing a nonfinite result. See State for the recovery details. Test a restart with your existing player data after repairing the settings.

The NPC State component inspector also checks its assigned catalog and trait preset. Select a single character or the referenced asset when the selection contains different assets and the result is undetermined.

Language of the sample screens

Hello NPC, Living Character and World Aware now have their own four-language selector for screen labels: English, Russian, Spanish and Simplified Chinese. The serialized `_uiLanguage` field defaults to `en`; the toolbar changes only this sample component. It does not use the editor language preference or change the NPC's dialogue language, persona, language catalog or voice. Your existing `Say`/`TrySay` calls and event bindings need no migration for this addition.

Entered questions, subtitles, provider errors, authored grammar and object IDs remain as supplied. Context previews still use the samples' existing English context calls. World Aware's last-action message and Living Character's stored log lines keep the language used when they were created; switching the selector does not retranslate that history. If you copied a sample script into your own project, merge the optional display changes deliberately. Check fonts, layout, input focus and serialized values in your own scene before shipping.

- 1.1
- Released — 1.0

Release history

A short record of what each release includes. For individual fixes and API details, read the technical changelog.

1.1

Changes in version 1.1 compared with the published 1.0 package.

- **A clearer start.** Welcome opens the wired Hello NPC sample directly. The guides reach a typed answer before optional voice output and microphone input. The sample keeps your question when the model is starting or another reply is running.
- **Qwen3-TTS speech.** The built-in language definitions use Qwen3-TTS and Serena (`serena`) across supported languages. The package includes the Windows x64 CPU native DLL; download the talker and codec in Model Manager. An external HTTP server is optional. Existing Piper setups can stay.
- **Voice input.** Push To Talk with Whisper Speech Recognition: hold a key, speak and release, and the line reaches the character as if typed. It needs whisper.unity and a Whisper model; a press that produced nothing says why.
- **Microphone selection.** Custom game settings can select a named input and display its signal level. An unavailable input reports an error without switching devices.
- **Safer character saves.** Failed saves remain pending for retry, small persisted changes are tracked, and stale reads cannot replace a newer restored state.
- **Safer conversation history.** Memory storage errors are contained without overwriting unread history; nested requests to add or clear messages keep their order.
- **Builds on Windows.** The Windows Editor releases native file handles between build operations under Mono. Build journal updates retry transient access conflicts and preserve changed files.
- **Model-file tools.** Local imports run in the background with progress and cancellation. File reports distinguish missing files, established defects and checks that could not be completed; finding a file does not prove the engine will start.
- **Editor and guide languages.** Every window and inspector of the plugin, including the built-in setup reports, supports English, Russian, Spanish and Simplified Chinese, and the guides open in the chosen editor language. This setting does not change the NPC's language.
- **Optional editor MCP tools.** A coding agent can inspect the project and an exact character reference, and get a plan for the first typed conversation. Overview tools read passive information by default; runtime probes and file checks require an explicit request. Supported POI and State changes can be previewed, explicitly applied and checked on the same component. This does not promise an atomic transaction or a durable save. The MCP server stays disabled until you enable it.

Before updating a working 1.0 project, follow Upgrading from 1.0. Keep your working Piper setup while checking the update. See Agent tools for MCP setup and its inspection and change limits.

Released — 1.0

The first release included:

- Local dialogue through the external **LLM for Unity** dependency, with **Qwen3** language-model presets. Models are downloaded separately.
- Optional voice input through **whisper.unity** and speech output through **Piper**. These dependencies, engines and model files are installed separately.
- Character state, traits and relationships; bounded conversation memory using storage you supply; perception of marked scene objects; and structured actions sent to game handlers you implement.
- **Hello NPC**, **Living Character** and **World Aware** examples, plus model management, setup diagnostics and buyer documentation.

Start with Getting started, or return to the documentation index. For the complete change record, use the technical changelog.

- Choose a speech setup for your game
- 0. What this is, in one paragraph
- 1. Before you start
- 2. Make a project
- 3. Install LLM for Unity — required
- 4. Import Offline AI NPC
- 5. Set up the first text dialogue
- 6. Check the text setup
- 7. Your first typed conversation
  - Normal, or broken?
- 8. Optional: add a voice
- 9. Optional: add microphone input
- 10. What is in the plugin, and what each part is for
  - The runtime, by area
  - Parts worth knowing about early
  - The editor windows
  - Every item in the menu
  - What the plugin deliberately does not do
- 11. Your own character
  - The four checks to preserve
- 12. The names you will actually type
  - Put on a GameObject
  - Implement in your own code
  - Call
  - Author as assets
  - Configure in the inspector or in code
- 13. Making a build
  - Before you send it to anyone
- 14. When it does not work
- 15. Words you will meet

Beginner's guide

Start here if you have not installed a Unity package with external dependencies before. The first goal is one typed question and one text answer in Hello NPC. After that, add voice, microphone input or your own character, then make a player build. Download and setup time depend on your connection, hardware and the features you choose.

After installation: follow the first text dialogue steps in one setup window.

Choose a speech setup for your game

Start with typed dialogue. Add Whisper when players need a microphone, then choose a ready voice:

Your priority	Start with
A quick reply and a suitable voice for your language	Piper with that voice's `.onnx` / `.onnx.json` pair
One voice engine across several supported languages	Qwen3-TTS with a named CustomVoice speaker
No spoken replies	Keep text dialogue; speech output is optional

The built-in Qwen speech presets use the 0.6B CustomVoice Q4 talker and Q8 codec. New Qwen engine components use the bundled CPU DLL with **External server URL** and **Style** empty, and the existing 1.5-second lead buffer. Serena (`serena`) is the default for every built-in Qwen language and preset. You can choose another of the nine ready speakers; creating a voice from a description (VoiceDesign) is not part of this version. Delivery instructions require a separately checked 1.7B CustomVoice setup. Check the language, voice terms and response time on your players' hardware; see Languages, Hardware tiers and Licensing.

Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no manually started HTTP server, executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.

0. What this is, in one paragraph

With the default local configuration, your character talks. The language model, the speech recognition and the speech synthesis all run **on the player's own machine** — no cloud, no API keys, no per-token bill, no telemetry, and it works with the network cable pulled out. For text dialogue you install a model runner, one model file and this plugin. Voice and microphone input add their own optional engines and files. One editor-only exception is yours to choose: the optional MCP server hands project data to the coding agent you connect, and a cloud-based agent sends it on to its provider — see Agent tools.

First milestone: compatible LLM for Unity → **Set up first dialogue** → one model → Play → a typed question and a text answer. Voice and microphone input are optional next steps.

---

1. Before you start

Unity	6000.5.7f1 is what this is developed and tested on. 2022.3 LTS or newer should work, but only 6000.5 is verified

Render pipeline	any for the runtime, which references no rendering package. The samples are made in URP: World Aware's five props use URP's Lit material, and outside URP the scene's Pipeline Material Fallback object gives them the running pipeline's default material instead of magenta. Built-in and HDRP are not verified
Platform	**Windows x64 CPU is the bundled Qwen target.** See the speech setup for its required CPU instructions. macOS/Linux native libraries are not included. Mobile, web and consoles are not supported by this package. Linux and macOS are not verified.
Disk	10 GB free is comfortable
Graphics	Choose a model for the available RAM/VRAM and the memory your game needs. **First dialogue** shows catalog options under the current hardware policy; see Hardware tiers
Internet	needed for downloads; the configured local engines can run offline. A deliberately selected remote endpoint needs its own connection

If any of those are a problem, read Is this for your game? before installing the dependencies or downloading large files.

---

2. Make a project

New project, any template. Nothing here depends on which.

You can also use an existing project. Importing the package adds its assets; opening the sample may ask you to save the current scene. Setup changes and build inclusion are explicit actions. Keep your existing scene and model/voice bindings when adapting the sample.

---

3. Install LLM for Unity — required

This is the thing that actually runs the language model. Without it your character cannot form a sentence.

- Package id: `ai.undream.llm`, version **3.0.3** or newer

- Free, and not bundled here — it is somebody else's package under its own licence, and freezing a copy inside this one would pin you to whatever version shipped that day
- Install it the way its own documentation says

How you know it worked: *Window ▸ Package Manager* lists **LLM for Unity**.

If Unity was already open, it may take one recompile before this plugin notices. You do not need to restart — `Tools ▸ Offline AI NPC ▸ Advanced ▸ Re-check LLMUnity installation` forces the check.

---

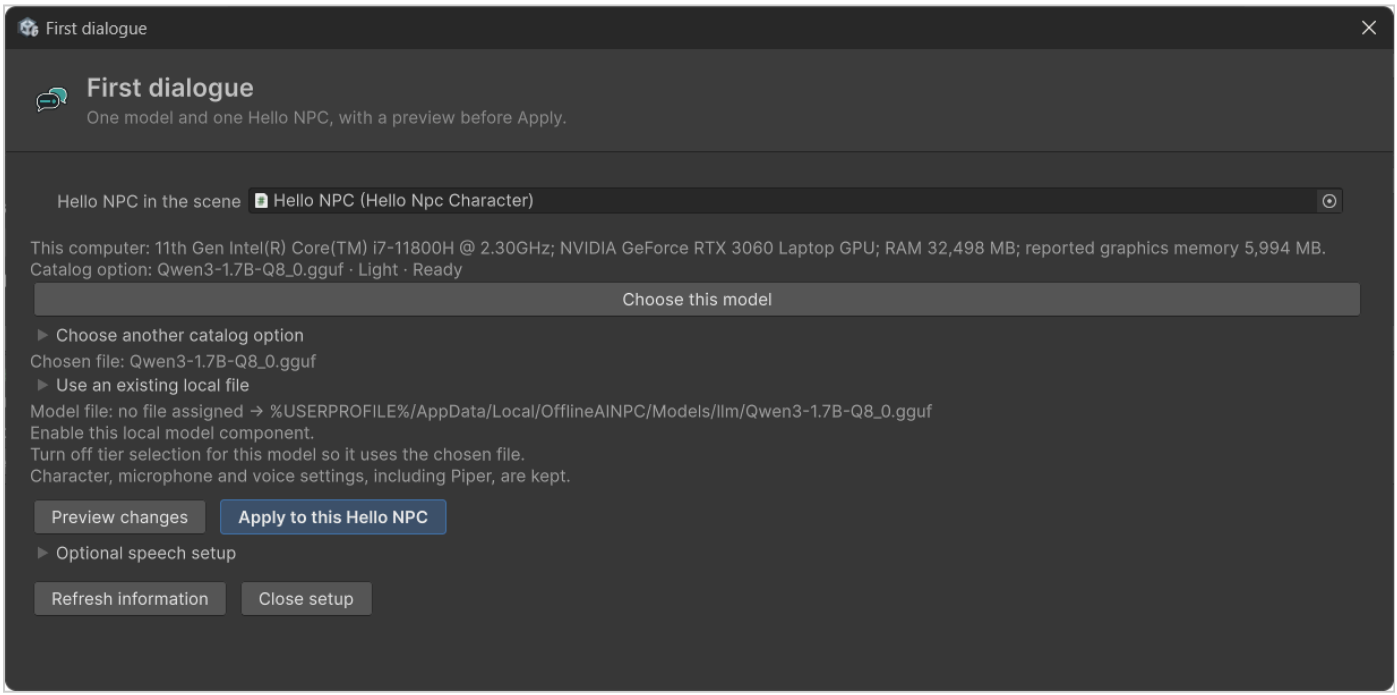
4. Import Offline AI NPC

Import the package. In **Welcome**, choose **Set up first dialogue**. You can reopen Welcome with `Tools ▸ Offline AI NPC ▸ Welcome`, or go directly to `Tools ▸ Offline AI NPC ▸ Set up first dialogue`.

---

5. Set up the first text dialogue

1. In **First dialogue**, if **Hello NPC in the scene** is empty, click **Open Hello NPC sample**. Otherwise keep or select the character you want to configure.
2. Click **Choose this model**. If its file is missing, Model Manager opens for that model. Download it there, then click **Return to first dialogue**.
3. Click **Preview changes**, read the proposed assignment, then **Apply to this Hello NPC**. You can restore the previous assignment with Unity **Undo**.
4. Save the scene, press Play and wait for startup. Type a question, press Enter and read the answer in the subtitle.



Already have a GGUF? Expand **Use an existing local file** and select it. To import a file, click **Open Model Manager**; see the own-GGUF steps.

Choose another catalog option lets you filter the model catalog by language and configuration size. The character's dialogue language is set in its persona settings; see Languages.

Start with text. **Optional speech setup** and the later voice/microphone sections can wait until this works. For distribution, check Licensing; advanced tier startup is in Hardware tiers.

---

6. Check the text setup

If the model does not start, open `Tools ▸ Offline AI NPC ▸ Check my setup` with Hello NPC open. Resolve the LLM dependency, selected model and scene-reference items for text dialogue.

Voice, codec and speech-output items belong to optional audio. See Getting started for more detailed setup help.

---

7. Your first typed conversation

If it is not already open, click **Open Hello NPC sample** in Welcome and save the current scene if prompted. It opens the already-wired `Samples/HelloNpc/HelloNpc.unity`. Press Play and wait for startup to finish. Type a question into the text box, press Enter and read the answer in the subtitle. If the model is unavailable or another turn is busy, the question stays in the input field; submit it again after resolving the message. It is not queued automatically. No microphone or ready speech engine is required for this first typed turn. A voice warning points to optional audio setup.

A healthy default tier start prints these lines. Learn to recognise them:

```
[OfflineAINPC] Hardware: NVIDIA GeForce RTX 3060 Laptop GPU (5996 MB VRAM, discrete), ...
[OfflineAINPC] Tier resolved: Standard — 3996 MB VRAM free after a 2000 MB game reserve
[OfflineAINPC] Running qwen3-4b-q4km at tier Standard with 36 GPU layers
[OfflineAINPC] Startup verdict: tier Standard — ...; model qwen3-4b-q4km, 36 GPU layers
```

For the tier route, the startup verdict summarizes the selected tier and model. It does not prove voice readiness or replace asking the first typed question. If text never arrives, inspect the Console and LLM/model setup rows, then Troubleshooting. The own-GGUF route disables its linked TierBootstrap; use the assigned LLM's startup diagnostics rather than expecting a tier verdict.

For Qwen3-TTS's ordinary DLL route, look for `Native CPU ready` in the Console; Piper has its own engine diagnostics. An explicitly configured external server has separate readiness diagnostics. Neither speech engine is required for text dialogue.

Startup and the first reply can take longer than later turns, including while a prompt cache is being filled. This is a possible cause of delay, not a guarantee that an unresponsive scene is healthy. Check the startup verdict for tier startup, or the assigned LLM startup for the own-GGUF route, plus the Console and setup report.

Normal, or broken?

These behaviours can be expected in the stated conditions. Use the setup report and Console to distinguish them from a failed or still-running startup:

What you see	
Subtitle appears, no sound, and no voice configured	**normal** on the text-only path; add audio afterwards if wanted
The first reply is slower than later turns	prompt-cache setup can contribute; check startup diagnostics if no answer arrives
The same question gets a different answer each time	**normal** — it is a model, not a dialogue tree
The character will not repeat an authored line word for word	**normal** — author those lines yourself and use the model between them
`NPCMemory` has no replay	**normal** until you assign `NPCMemory.Store`; the agent's chat history is separate
A configured `NPCMemory` replay drops old turns	**normal** — its window is the last ~20 turns per character
The mouth does not move, the character does not walk anywhere	**normal** — the plugin drives none of that
The character invents something in the room	wire Perception to supply nearby context; this does not guarantee factual free-form prose
The editor seems unresponsive on the first Play	loading may be involved; inspect the Console and model startup instead of assuming every stall is normal
Some turns are much slower than others	First and later turns can differ. See Limitations for causes of delay and checks.

And these are not — each has a page that names the cause:

What you see	
Audio enabled, subtitle appears, no sound	**broken speech setup** — check the engine and paired voice files (Troubleshooting)
The character answers in the wrong language	Check the language directive and the selected model.
Replies run on for paragraphs	**broken configuration** — the token ceiling and sentence limit, not the wording of the prompt

What you see	
No text answer and no startup verdict	check the LLM dependency, model and references before adding speech
Push-to-talk sends things you never said	Check the microphone, recognition language and model, and the utterance-gate settings. The gate filters known cases; it cannot prevent every transcription error.

---

8. Optional: add a voice

The package includes the Windows x64 CPU Qwen speech library. Its target CPU must support **AVX2, BMI2, FMA, F16C and SSE4.2**. macOS and Linux libraries are not included.

1. In **Tools** → **Offline AI NPC** → **Model Manager**, download the **0.6B CustomVoice Q4 talker** and **Q8 codec** under *Voices and speech models* (about 0.9 GB together).
2. Use the scene's **Qwen3-TTS Engine** through **TTS Engine Manager** and its language binding. The sample supplies this wiring; a custom character must select that engine.
3. Leave **External server URL** and **Style** empty. Use `serena` in **Default voice** to keep the same voice across languages. A character's `SpeechRequest` or **Speaker voices** entry can override it. Set the language separately; choosing a voice does not translate the text.
4. In Play, type a short prompt and check the reply's subtitle and speech in the chosen language. Repeat this check in your built Windows player with the model files available.

The default route loads the DLL in Unity and streams PCM to the existing speech session. It does not start `tts-server.exe`. The native profile is CPU-only with eight threads. **StopAsync** waits for native generation and worker settlement before releasing the context. Cancellation requests cooperative cancellation; it is not an instant-stop guarantee.

A separately launched server and **External server URL** / `AttachTo(url)` are optional advanced alternatives. Keep that server's own model, backend, version and distribution requirements separate.

For a new Piper setup, put `piper.exe` for Windows and its companion files in `Assets/StreamingAssets/piper/`, and the `name.onnx` / `name.onnx.json` pair in `Assets/StreamingAssets/piper/voices/`. Other platforms remain unverified. Preserve existing working configurations and custom paths.

Piper remains supported alongside Qwen3-TTS. Keep **both files**, `name.onnx` and `name.onnx.json`; an incomplete pair is refused with a diagnostic. Model Manager's **Import file** copies a local pair while preserving the originals and existing destination files. The imported pair goes into Model Manager's `ModelsRoot`; this does not install the engine or connect the built-in Piper driver to that voice. Copying and verification show inline background progress; **Cancel import** requests safe cleanup. See Languages and Troubleshooting.

Run **Check my setup** again and fix the speech rows. Ask another typed question and check both the subtitle and audio. File presence does not prove native engine startup.

---

9. Optional: add microphone input

This is speech **input**: the player speaking instead of typing. Add it after the typed path works; skip it if you do not need a microphone.

1. Install whisper.unity: *Window* ▸ *Package Manager* ▸ + ▸ *Install package from git URL*, and paste:

`https://github.com/Macoron/whisper.unity.git?path=/Packages/com.whisper.unity`

1. In the Model Manager, open **Speech recognition** and download the Whisper model your preset names — `ggml-tiny.en.bin` or `ggml-base.en.bin` for English on Light and Standard, the multilingual `ggml-base.bin` or `ggml-small.bin` otherwise. An `.en` model recognizes English only.
2. Select the Hello NPC character and add two components: **Add Component ▸ Offline AI NPC ▸ Push To Talk** and **Add Component ▸ Offline AI NPC ▸ Whisper Speech Recognition**. For runtime English/Russian switching, download a multilingual model such as `ggml-small.bin` and set **Model File Name** to it before entering Play. The loaded model stays in use for the session; a Language Definition's **STT model id** does not replace it.
3. Press Play, click outside the text box, hold **V**, speak and release. What it heard goes to the character as if typed; the screen says when a press was not sent, and why.

A missing dependency is never a compile error here. Without whisper.unity the Whisper Speech Recognition component is compiled out; Push To Talk stays, and with no recognizer beside it every press is reported as not sent. Your own recognizer can implement `ISttEngine` instead. Details: Getting started.

---

10. What is in the plugin, and what each part is for

Everything below ships in the package. You will not need all of it on day one — most projects start with dialogue and add the rest when they meet the problem it solves. This section exists so that you know what is already there before you build it yourself.

The runtime, by area

Area	What it is for	The problem it solves
Core	languages, presets, sampling, the streaming reply parser, logging	A model answering in the wrong language, or a reply arriving as tokens when speech needs sentences
Speech ▸ TTS	the engine interface, the Qwen3-TTS engine (native DLL or an optional external HTTP URL), the Piper driver and process pool, voice routing, the streaming speech session	Turning a reply into audio while it is still being generated, instead of after

Area	What it is for	The problem it solves
Speech ▸ STT	Push To Talk, the recognizer interface, shared microphone loop, push-to-talk recorder, the utterance gate	Whisper inventing sentences out of silence — see below, it matters
State	axes, bands, traits, mood presentation, serialisation	A character whose mood and relationships change, and who reads differently because of it
Actions	the verb schema, parser, dispatcher, targets, one-shot repair	The character DOING things, not only saying them — and doing only what you allowed
Perception	points of interest, weighted queries, snapshots, phrasing	The character knowing what is actually around, rather than being told to pretend
Memory	a small conversation window over storage you supply	The character remembering the last few turns without you inventing a format
Hardware	probe, tiers, the cascade, model catalog	Deciding at startup what this machine can carry, and picking a model to match
Models	gguf reading, asset states, storage paths	Checking file evidence before loading; native compatibility and startup remain separate
Diagnostics	the checks behind the setup window	Finding a broken setup at edit time rather than in a playtest
Verification	latency measurement, prompt snapshots, live checks	Proving it works, with numbers you produced yourself
Integrations	the LLMUnity grammar binding; Whisper Speech Recognition	Constraining the model's output to your action schema; recognizing the player's speech with whisper.unity

Parts worth knowing about early

The utterance gate (`UtteranceGate`). Whisper hallucinates confidently on silence -- "Thank you for watching", "Subtitles by ...", a lone full stop — because it was trained on video with end cards. A push-to-talk key brushed by accident therefore looks exactly like a player speaking. The danger is not a stray subtitle: if your game listens for a word as a command, an invented one triggers it, and a key brushed in passing sends the player somewhere they never asked to go. The gate judges the audio first and the text second, and every rejection says which one it was.

Action repair (`ActionRepair`). When the model returns a malformed action, one retry is allowed — and it asks for **the action alone**, never the reply again. By the time anyone knows the action was bad the speech has already been spoken, and regenerating it would make the character say the same thing twice.

Traits versus state (`TraitPreset` versus `StateAxes`). Traits never move: they are why two characters with identical axes behave differently — the shy one starts lower and warms slowly. State moves. Mood is deliberately **not** persisted, because a feeling that outlives the session is not a feeling, it is a trait.

Perception weights (`PerceptionWeights`). Serialized data, never constants. The right balance of attention for a cluttered room is not the right balance for an open field, and that is a designer's call rather than the plugin's.

The language directive (`LanguageDirective`). The plugin's own contribution to your system prompt: answer in THIS language, and keep it about this long. Both sentences are written **in the language they are asking for** — an instruction the model cannot read is not an instruction. Your dialogue code appends it: `LanguageDirective.For(LanguageRuntime.Current)` returns what the language definition authors, and nothing when it authors neither sentence. Hello NPC speaks the language of its own system prompt and does not add it.

Preset choice (`PresetCatalog`). Your entry outranks an official one for the same slot; otherwise the exact tier wins, then the closest lighter tier — because running something smaller beats running nothing.

The editor windows

All under `Tools ▸ Offline AI NPC`.

Window	What it answers
Welcome	Start with **Set up first dialogue**; links to setup, samples and documentation.
First dialogue	Choose one model, prepare its file, then preview and apply its assignment to Hello NPC.
Model Manager	"Which model, and is it really here?" Tier, languages and licence for catalog rows, downloads checked by size and header (and by hash when the preset carries one), and control over what enters a build

Window	What it answers
Presets	"Which model and voice are recommended for this language and tier?" This does not select the scene's active model
Check my setup	"Is anything wrong before I press Play?" Real checks, each failure saying what to do
Prompt debugger	"**Why did the character say that?**" What the model was actually told. For a local product this is the only way to answer it
Live verification	"Does it actually speak?" Runs a whole turn — prompt, model, voice — and plays it; it generates before it voices, so Hello NPC and most characters need Play mode, and outside it the window says why it refuses. A language is marked verified only when you answer **Yes, record it** to "was the reply spoken in this language?". Static checks can say a configuration is valid, never that it works
Agent tools (MCP)	"Can my coding agent test this character with me?" An MCP server inside the editor, off until you switch it on: Claude Code, Cursor or Codex can read what a character was told, run a real turn, look at state, memory and perception, and mark objects to be noticed. See Agent tools

Every item in the menu

`Tools ▸ Offline AI NPC`, top to bottom — the windows first, everything else under **Advanced**:

Item	What it does
Welcome, **Set up first dialogue**, **Model Manager**, **Presets**, **Check my setup**, **Live verification**	the windows above, in the order you first need them
Prompt debugger, **Agent tools (MCP)**	the windows above; **— start the server** and **— stop the server** switch MCP without opening its window
Advanced ▸ Check my setup (log to console)	the same report as text in the Console — paste it into a support request

Item	What it does
Advanced ▸ Live verification — verify everything (console)	runs every character that follows the project language through each language and prints what played; a character that answers in its persona's language, such as Hello NPC, is listed as skipped — verify it in the window. It records nothing as verified and skips a language without a test line. Chosen again during a run, it offers to stop it
Advanced ▸ Validate model references	lists references in scenes, prefabs and assets to model files that are not on disk — the same check that can stop a build
Advanced ▸ Check for unaddressable scripts	finds a MonoBehaviour or ScriptableObject class in a file with another name: Unity lets you add such a component to a scene, and it never runs
Advanced ▸ Re-check LLMUnity installation	looks for LLM for Unity again and sets or clears `LLMUNITY_PRESENT`
Advanced ▸ Reset to the embedded catalog	puts the official presets back to the ones in the package; your own presets are kept
Advanced ▸ Show welcome again after the next reload	Welcome opens by itself once more
Advanced ▸ Logging ▸ Verbose logging (AINPC_DEBUG)	adds or removes the `AINPC_DEBUG` symbol and recompiles, so debug lines exist at all. For a log for support, turn it on and set **Minimum level ▸ Debug** below
Advanced ▸ Logging ▸ Minimum level ▸ Debug … Silent	filters the plugin's console lines while you work, in Play mode too; a shipped game sets `Log.MinimumLevel` itself
Advanced ▸ Speak one line (Piper) ▸ …	says one fixed sentence through Piper, with no language model and no Play mode. It needs Piper and the named voice installed; the Russian item uses the voice the Russian language definition names, so Russian has to be set up for Piper

What the plugin deliberately does not do

Knowing these saves you looking for them:

- **Qwen speech includes its Windows x64 CPU native DLL.** Model files are downloaded separately. Piper is still an external program you install; preserve its working executable, voice/config pair and binding.

11. Your own character

Start from the canonical `Samples/HelloNpc/HelloNpcCharacter.cs` after the sample answers. It contains the real LLMUnity call, the check that refuses a turn while another runs, action handling and cleanup. Preserve its public `Say(string)` entry point and serialized references when adapting an existing scene.

The following is a teaching outline, not a copyable controller or a model backend:

```
typed question -> one admitted turn -> streamed LLM reply
              -> local parser -> local speech session -> subtitle (+ optional audio)
completion    -> complete parser -> InputComplete -> dispose session
disable       -> invalidate callbacks -> cancel generation -> dispose session
```

For guarded wiring in C#, read Dialogue. Its `GenerateReply`, `CancelGeneration` and `ShowSubtitle` methods are explicit placeholders for your existing backend and UI; the empty `GenerateReply` does not call a language model. Use Hello NPC for a runnable starting point rather than filling these seams on day one.

The four checks to preserve

1. **A turn owns its callbacks and `_busy`.** Keep parser and speech references local to that turn and reject callbacks after disable or replacement. Release the lock only while that turn is current. `OnDisable` must cancel the backend and dispose speech explicitly: Unity can stop coroutines without running `finally`.
2. **A null or unready engine is normal for text dialogue.** Fetch the current engine through a `Func<>`; speech is optional and the session can deliver subtitles without audio. Diagnose a failed voice setup before testing speech.
3. **Cumulative versus delta.** LLMUnity's callback carries the whole reply so far; use `AppendCumulative`. A backend streaming true deltas needs `Append`.
4. **Completion and cleanup.** Complete the parser and call `InputComplete()` when generation ends. Dispose the session on failure or disable, and prevent an old continuation from clearing a newer turn's state.

Memory storage is explained in Dialogue; moods and relationships are in State, which opens with a ten-line version.

---

12. The names you will actually type

The section above says what each area is for. This one says what to reach for, because a concept you cannot name is a concept you cannot look up. Only the types you touch are listed; the rest are internals that happen to be public.

Put on a GameObject

Type	What it does
`NpcStateComponent`	holds one character's axes, traits and mood

Type	What it does
`AIPointOfInterest`	marks a thing in the world as noticeable, with an id and a description
`PerceptionQuery`	the character's own view: what it can see, ranked
`ActionDispatcher`	receives the actions the character decides on and routes them to handlers
`PushToTalk`	hold a key, speak, release: the recognized line goes to the character
`WhisperSpeechRecognition`	recognizes a recording with whisper.unity and the downloaded Whisper model
`PersistentMicRecorder`	push-to-talk recording for one listener; Push To Talk adds its own
`TTSEngineManager`	owns the active speech engine and switches it with the language

Implement in your own code

Type	Why
`IActionHandler`	run a verb the character issued
`IActionTarget`, `IAimTarget`	be something an action can be aimed at
`IStateStore`, `IKeyValueStore`	persist state and memory in YOUR save system
`ITTSEngine`, `ILanguageAwareEngine`	add a speech engine the package does not ship
`ISttEngine`	use a speech recognizer other than Whisper with Push To Talk
`ILlmHost`	put a different model runner underneath
`ILiveVerifiable`	let the live-verification window exercise your character

Call

Type	Why
`NpcActions.Register`	declare a verb before anything can issue it
`ActionSchema`	the grammar and the prompt block, generated from the verbs
`NPCMemory`	append turns, read back the last N
`StreamingReplyParser`	tokens in, speakable units out
`StreamingSpeechSession`	units in, audio and subtitles out
`UtteranceGate`	judge a recording, and then its transcript, before spending a turn
`UtteranceVerdict`	what the gate decided: accepted, too quiet, or likely hallucinated
`LanguageRuntime`	the current language, and the directive it contributes
`StateSerializer`	the persisted half of a character
`SetupChecks`	run the same checks the window runs, from your own code
`LatencyMeasurement`	produce your own numbers rather than trusting ours. Let `Run` finish or call `Dispose` on its enumerator: a coroutine stopped with `StopCoroutine` or by disabling its object keeps the character's conversation set aside, and the character refuses other turns
`ActionParser`	turn the model's text into an action, or say precisely why it could not
`PerceptionSnapshot`	the ranked result of one query, reused for the prompt and the shortlist
`PoiRegistry`	every noticeable thing currently registered
`SharedMicLoop`	one microphone, several listeners, no fighting over the device

Type	Why
`HardwareProbe`, `TierCascade`	what this machine is, and what that means for the model
`ModelCatalog`	the recommended line-up, per tier
`AiNpcDiagnostics`, `DiagnosticsReport`	the checks and their result, as data
`LLMUnityGrammarBinding`	apply your action grammar to LLMUnity; a valid completed output must follow it while the binding is active
`TextLengthRule`	how long a reply should be, as a phrase the model can read

Author as assets

Type	Why
`AxisCatalogAsset`	which axes your characters have at all
`TraitPresetAsset`	fixed dispositions: the shy one, the blunt one
`LanguageDefinition`, `LanguageSet`	one language, and the set your project ships
`ModelManifest`	which model each tier wants
`TierPolicy`	the tier thresholds and the VR reserve

Configure in the inspector or in code

Type	Why
`PerceptionWeights`	how much each factor of attention counts — a field on the perception query
`SamplingProfile`	temperature, penalties, reply length; one profile for the session
`PresetCatalog`	which model and voice a language and tier resolve to — the JSON the Model Manager reads
`KeyValueStateStore`	a ready `IStateStore` over any key-value storage you already have

Type	Why
`ActionTargetBehaviour`	the component that makes a GameObject nameable as a target
`ActionVerb`	one declared verb: its name, its parameters, whether it needs a target
`ActionParams`, `NpcActionEvent`	the arguments an action carries, and the UnityEvent you wire in the inspector

> **Coverage, honestly.** The documentation names the types you put on objects, implement, call and author; the rest are data holders, enums and internals that happen to be public. If you meet a name that no page explains, that is a gap in these docs rather than something you are expected to already know.

13. Making a build

The step most likely to surprise you.

Models are excluded by default — every one of them. That default is a feature: six models downloaded while experimenting would otherwise silently become a twenty-gigabyte build, discovered at upload time. Turn on the one you ship, in the Model Manager.

An included model is staged into `Assets/StreamingAssets/OfflineAINPC` for the duration of the build and removed afterwards. This is the plugin's staging path for selected model files; it avoids leaving those copies in `StreamingAssets` between builds. For the own-GGUF route, configure the player model path separately: including the file does not rewrite `LLM._model`. See the path requirements.

When **Block a build whose scenes reference a model file that is not on disk** is enabled in Model Manager (`BlockBuildOnBrokenReferences`), detected missing model references stop the build; disabling it reports a warning. The validator matches file names in its search roots. It does not verify player paths, model compatibility or portability.

- **Qwen speech includes its Windows x64 CPU native DLL.** Model files are downloaded separately. Piper is still an external program you install; preserve its working executable, voice/config pair and binding.

Before you send it to anyone

- run it on a machine that is **not** your dev machine, ideally without Unity installed;
- find the **startup verdict** for tier startup, or check the assigned LLM startup and model path for the own-GGUF route, in the player log;
- type one question and read one answer; if you enabled audio or microphone input, test those separately.

---

14. When it does not work

In the order these actually happen:

Symptom	Check first
Audio enabled, but no sound	For the included Qwen route, check the native library, supported CPU and talker/codec model files. Read the engine's LastProblem/Console diagnostic. A successful file scan does not prove native generation or sound. For an explicitly configured HTTP URL, check that server and its model separately.
No startup verdict line in the tier route	the model file: present, whole, referenced; the own-GGUF route uses the assigned LLM startup diagnostics
Compiles, but no dialogue features	LLM for Unity missing, or Unity has not recompiled since it appeared
Holding V does nothing	Push To Talk is not on the character, or the text box has keyboard focus — click outside it. Typing still works
"Not sent: …" after speaking	the reason follows: no recognizer (whisper.unity or Whisper Speech Recognition missing), model not downloaded, too short, too quiet
Works in editor, silent in build	For the included Qwen route, check the native library, supported CPU and talker/codec model files. Read the engine's LastProblem/Console diagnostic. A successful file scan does not prove native generation or sound. For an explicitly configured HTTP URL, check that server and its model separately.
The character says "MMaraMara VMara Vey…"	cumulative fed to `Append`, or deltas to `AppendCumulative`
Stopped answering, nothing in console	a leaked `_busy` — check every exit path
Everything is very slow	which tier resolved? The Light tier on a CPU is slow by design; below it there is no model

Troubleshooting maps real error strings to fixes.

---

15. Words you will meet

Model / gguf	the language model file. Size alone does not establish answer quality or speed
Tier	what your hardware can carry. Decided at startup and printed
Qwen DLL / Piper	Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no manually started HTTP server, executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.
Whisper	the program that turns the player's speech into text
StreamingAssets	the Unity folder whose contents survive into a build as real files
Prompt cache	reuses prompt processing; filling it can make an initial turn slower
Quantisation (`Q4_K_M`)	the weight compression format. `Q4_K_M` is one option; answer quality and total runtime memory depend on the model, context and backend
Parameters (`4B`)	the number of learned weights in billions. 4B is a model size, not a guarantee of suitability; choose by your game, language and resource budget
GPU layers (`ngl 36`)	how many model layers are offloaded to the GPU. Useful settings depend on backend support, available memory and the game sharing that GPU
Context window	how much text the model can hold at once — system prompt, memory replay and the current turn together
System prompt	the standing instructions the character never says out loud: who the character is, what language to answer in, what it can see

Token	a tokenizer unit used for context and generation limits; it may be part of a word. Local use of this plugin has no per-token fee
Streaming	the reply arriving piece by piece, so speech can start before the sentence is finished
Verb / action	a decision the character can make, in a form your code can execute. You invent them — see Actions
Grammar	restricts valid completed output while applied, including action fields; it does not guarantee the truth of free-form prose or a reply after cancellation/error
Axis / band	a number describing a character (`trust: 7`) and the named range it falls in (`warm`). The model is told the band, never the number
Point of interest	an object marked as noticeable, so the character can mention what is actually there instead of inventing a fireplace
Speaker id	which voice inside an engine to use. Qwen3-TTS maps it to a speaker name through the engine's *Speaker voices* table; Piper runs one voice per process
Utterance gate	the check that drops a silent or hallucinated recording before it costs a turn
Push-to-talk	holding a key to record. The alternative — always listening — transcribes the room

- Choose a speech setup for your game
- 1. Replacing a dialogue tree, in an afternoon
- 2. A companion who reacts to the room
- 3. An interrogation scene
- 4. A shopkeeper or quest-giver
- 5. A roommate or life-sim character
- What none of these are
- Checks to add to your game loop

Recipes

Five shapes this plugin is actually good at, with what each one uses and roughly what it costs to build. Times assume you have done the quickstart once and are comfortable in Unity; they cover wiring and the small persona/configuration text described below, not your art or broader narrative writing.

Choose a speech setup for your game

Start with typed dialogue. Add Whisper when players need a microphone, then choose a ready voice:

Your priority	Start with
A quick reply and a suitable voice for your language	Piper with that voice's `.onnx` / `.onnx.json` pair
One voice engine across several supported languages	Qwen3-TTS with a named CustomVoice speaker
No spoken replies	Keep text dialogue; speech output is optional

The built-in Qwen speech presets use the 0.6B CustomVoice Q4 talker and Q8 codec. New Qwen engine components use the bundled CPU DLL with **External server URL** and **Style** empty, and the existing 1.5-second lead buffer. Choose one of the nine ready speakers; creating a voice from a description (VoiceDesign) is not part of this version. Delivery instructions require a separately checked 1.7B CustomVoice setup. Check the language, voice terms and response time on your players' hardware; see Languages, Hardware tiers and Licensing.

Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no manually started HTTP server, executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.

1. Replacing a dialogue tree, in an afternoon

What it is. One character who answers anything, instead of three canned options. The fastest route from import to something worth showing.

Uses. Dialogue + memory. Nothing else.

Configure. A persona (who the character is, how it speaks, three or four facts), a voice, a push-to-talk key or a text field.

About two hours, most of it writing the persona.

Watch for. Use a token ceiling to bound reply length. Cache stable persona/schema instructions and refresh changing facts per turn.

---

2. A companion who reacts to the room

What it is. The character notices what is actually around, and mentions it unprompted.

Uses. Dialogue + **perception**.

Configure. A `PerceptionQuery` on the character, POI components on the objects worth noticing, a description per object, occluder layers set to your walls. In your dialogue code, `Recompute()` before the turn and `DescribeContext(languageCode)` in the user message (Perception).

Half a day, mostly tagging objects.

Why it matters. `PerceptionQuery` supplies nearby objects selected by your query. Inspect its snapshot in your scene and give important objects clear descriptions.

Watch for. Objects with no renderer. Line of sight is measured to the centre of the renderers, so a normal prop is fine wherever its pivot sits; an empty marker is aimed at its pivot, and a pivot on the floor reads as hidden behind the floor. Give markers an Aim Point.

---

3. An interrogation scene

What it is. The player asks anything; the character has something to hide; how open it is changes as the conversation goes.

Uses. Dialogue + **state** (one axis) + threshold events.

Configure. One axis with three bands and the game events that move it. Start from the short path — ten lines, no assets.

A day, including the visible consequences.

The rule that makes it work. Let game code set the state and drive posture or lighting from band events. State hints can affect the model's wording; visible consequences should follow your game rules.

Also: put checkable facts in the character's brief and discrepancies in the player's evidence. Validate derived quantities in game code.

---

4. A shopkeeper or quest-giver

What it is. The character does things, not only says them — points at stock, opens the ledger, marks a quest.

Uses. Dialogue + **actions**.

Configure. Register a few verbs with their targets; wire each to a method. Grammar makes the payload well-formed, so you parse a struct, not prose.

Half a day.

Watch for. The verbs go in the cached system prompt; the *targets* arrive per turn, because what the character can reach changes as it moves. Baking the targets into the system prompt means it names objects that are no longer there.

---

5. A roommate or life-sim character

What it is. Someone with moods and needs that move over hours, who is different at midnight than at noon.

Uses. Dialogue + **state** (several axes, traits, persistence).

Configure. An axis catalog, a trait preset per character, a persistent store, and a coarse tick.

A day or two, most of it tuning rates so a day of game time feels right.

Watch for. Two characters with the same catalog and no trait presets behave identically, which reads as one character with two faces. Traits are what separate them.

And: persistence needs a store. The plugin ships `PlayerPrefsStore` for prototypes and a `KeyValueStateStore` you point at any key-value storage; a real save system is yours.

What none of these are

A crowd. Every recipe here has **one character generating at a time**. Measure latency and load in your build before adding background conversations. See Is this for your game?.

Checks to add to your game loop

Start with one character and typed dialogue. These checks use existing APIs; they do not require a new model or a different voice backend.

Situation	What to wire	Check before shipping
A player interrupts or leaves the scene	Call `HelloNpcCharacter.CancelReply()` for interruption. Keep the next input in your UI until `!IsBusy`; `TrySayText` returns whether a text turn was submitted.	Stop during generation and repair, then ask again. Late output must not affect the retired turn. Cancellation does not undo a completed action or create an input queue. See Dialogue.

Situation	What to wire	Check before shipping
A prop moves or is removed	Use `PerceptionQuery` and register the currently available targets in `ActionSchema.Targets`. Remove an unavailable target from that registry.	Move an object between asking and acting; reject an absent or forbidden target. The receiver must also check current game facts. Perception text is context, not a guarantee about every model answer. See Perception and Actions.
A purchase or quest changes game state	Route a registered action to your game method. Validate current stock, price, funds and quest permissions there before committing the game operation.	Test a repeated block and a handler that changes state before throwing. `ActionTurn.CompleteRepair` accepts only `NeedsRepair` turns; it is not a rollback or retry mechanism for a delivered action. Your handler owns transactions. See Actions.
A suspect reacts to evidence	Use `NpcStateModel.Modify` for authored events, and `BandOf` to choose posture, lighting or access to an authored clue. Keep evidence and arithmetic in game data.	Verify the exact consequence without asking the model to decide it. Generated speech can reflect state probabilistically. See State.
Relationships survive a save/load	Assign an `IStateStore` to `NpcStateComponent.Store`: `new KeyValueStateStore(new PlayerPrefsStore())` for a prototype, or a `KeyValueStateStore` over your own key-value storage. Keep the NPC identity stable across loads.	Test separate save slots and unavailable storage. Relationship and Need can persist; Mood is transient. Read the store's failure contract before saving over old data. See State.

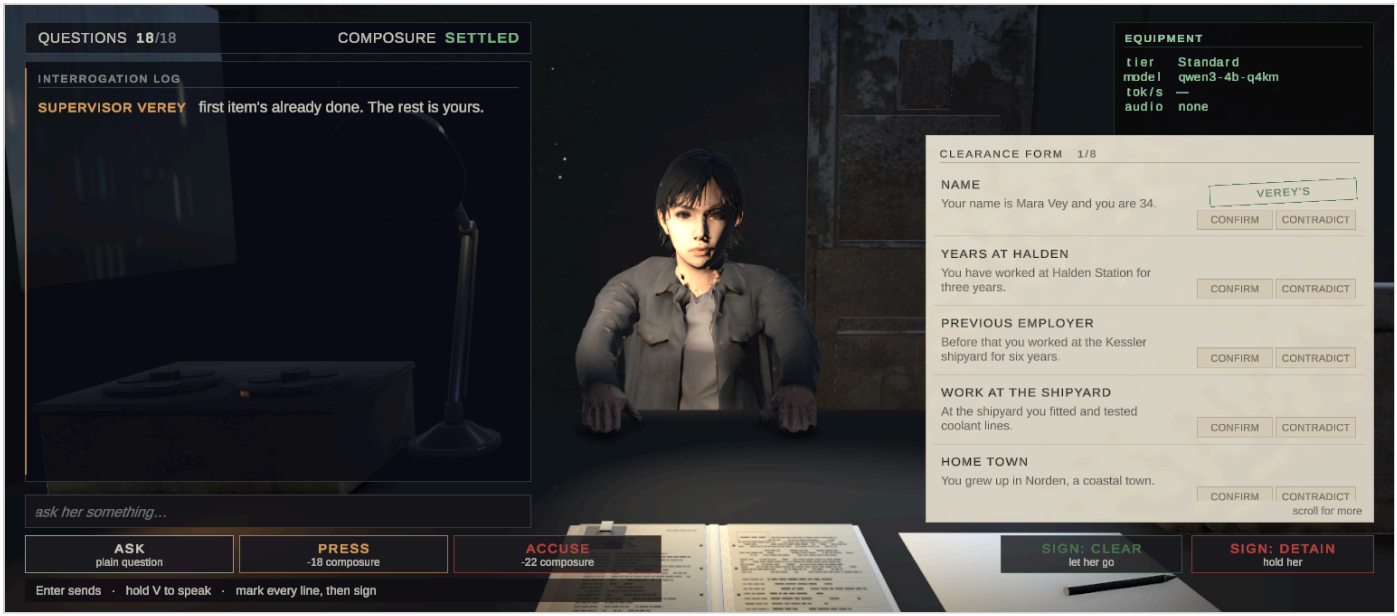
Situation	What to wire	Check before shipping
Voice is unavailable but text should work	Submit that turn with `TrySayText`. Clear the input only when it returns `true`. Leave Engine unassigned when the scene should never start a speech engine.	Test a missing or faulty voice separately from the language model. A text-only turn skips synthesis; the component's `Start` can still inspect/start an assigned engine. No microphone is required. See Dialogue.

Managed checks cover parser, state and cancellation contracts. Also exercise the actual scene, model, save backend and chosen voice on the target player before release; this page does not certify those integrations. For several NPCs, schedule requests in your game and preserve each agent's conversation; there is no automatic multi-NPC queue.

- What it is for
- The premise
- What you do
- What to watch while you play
- What is plugin and what is demo
- Running it
- The endings

The interview demo

A five-minute scene where you question a woman across a desk and decide whether to clear her. With a model available, her answers are generated on your machine. Microphone input and speech synthesis also run locally. Without a model, the interview uses her filed deposition as authored dialogue.



> **Play it first, if you would rather see than read:** https://merrymaker14.itch.io/the-interview — a free Windows build which fetches its language model on the first launch.
>
> **Where it lives.** The scene is `Assets/Demo/Scenes/InterviewDemo.unity` in the project repository. It is not one of the packaged samples — the three that ship in `Samples/` are HelloNpc, LivingCharacter and WorldAware. This page describes it because it is the fullest answer to "what does a finished thing built on this look like".

What it is for

Three questions get asked about a local-model plugin, and a demo is the only honest way to answer any of them:

The question	What the demo shows
Does it actually work?	A complete interview with generated replies when a model is available, or authored dialogue in Fallback
Will it run on my machine?	The equipment panel prints your tier, your model, its tok/s estimate and your reported time to first playback, live
What do I have to write myself?	Every piece the demo uses is public API — the interview logic is ordinary C# on top of it

It is deliberately not a technology showcase with buttons labelled "test perception". It is a small game, because a plugin that only demos well is a plugin nobody finishes shipping with.

The premise

You are a clearance officer on a night shift. Your supervisor, Verey, has left you a candidate and a file about her. Your job is to work through the form, decide whether the file and the woman agree, and sign.

There is a catch, and it is the point: **the evidence cannot tell you what she is.** She is a human or an android on a coin flip, and the contradictions in her file are seeded independently of that. Clearing an android is not a loss. Detaining a human is not a loss either. There is no score, because a score would turn it into "guess whether it is an AI", which is a worse game and a dishonest one.

What you get at the end is the truth beside your own marks: what she actually was, which of her claims really did disagree with the file, and where your reading matched.

What you do

Ask her things. Type into the box and press Enter, or hold **V** and speak. Speech is transcribed locally by Whisper and sent as a question the moment it is recognised — no second keystroke, because having said something out loud and then being asked to press a key breaks the one illusion the demo exists to sell.

Write questions in the language selected in **Settings**. Input using an incompatible script, or numbers or punctuation alone, is refused without spending a turn. This check is not an exact language detector for languages sharing the Latin alphabet.

Three ways to ask, and they cost different amounts:

	What it does	Composure
ASK	a plain question	**+6** — neutral ground lets her settle

	What it does	Composure
PRESS	lean on an answer that disagrees with the file	**−18**
ACCUSE	say out loud that she is a machine	**−22**

Composure starts at **70** and is visible in the status line as a band: `Settled`, `Guarded`, `Cornered`. It is not decoration — at **8** or below she stops cooperating and the interview ends whether you were finished or not. Four hard presses get there, three accusations sooner, so pressure is a resource you spend rather than a button you hold.

Repeating a question she has already answered costs 7, whichever of the three ways you repeat it. Long silences cost 4: she notices. A question whose reply failed gives its turn back, and asking it again is not a repeat.

Mark the form. Eight items, each with the file's version of it. Confirm the ones she backs up, contradict the ones she does not. The first item is already filled in — that is Verey's, and the form says so; override it if you disagree.

Sign. `CLEAR` or `DETAIN`, at any point. You have eighteen questions; signing early is allowed and sometimes the right call.

What to watch while you play

The green panel at the top right shows the current model and timings on your machine:

```
tier: Standard
model: qwen3-4b-q4km
rate: pending
audio: waiting for a reply
```

- **tier** — the tier the interview is running on: what the hardware probe decided your machine can carry, or the tier of the model you picked in Settings (Hardware tiers)
- **model** — which model that tier chose from the catalog
- **rate** — an estimate of reply tokens per second, based on tokenizer counts and the observed reply interval. `pending` means the estimate is still being calculated; `unavailable` means there is no estimate for this turn.
- **audio** — from your question to the plugin's playback-start event for her first sentence, not an audio-device timestamp. Her own pause before answering is inside the number and named beside it, so a cornered candidate's longer beat is not read as a slower machine. A `voice: unavailable` line appears when the microphone device or the demo's voice controls are missing.

The audio timing follows the plugin's playback-start event; it does not measure the physical output of your speakers.

Watch her, too. She breathes, shifts her weight, and turns — eyes first, then her head, her shoulders following — toward things you mention that are actually in the room. The tape recorder is a real point of interest; she looks at it when it comes up, and she does not invent objects that are not there.

What is plugin and what is demo

Worth knowing if you are reading the code to copy from it.

Comes from the plugin	Written for the demo
`StreamingReplyParser` — tokens in, speakable sentences out	`InterviewSession` — turns, composure, endings
`StreamingSpeechSession` — synthesis and playback in order	`CandidateFactory` — the woman, her file, the seeded contradictions
`PushToTalk`, `WhisperSpeechRecognition` — recording and local speech recognition	`InterviewHud` — the desk, built in code
`TTSEngineManager`, `Qwen3TtsEngine` — the voice	`CandidateGaze`, `SeatedIdle` — where she looks and how she sits
`AIPointOfInterest`, `PerceptionQuery` — what she can see	`FaceAnimator`, `CandidatePerformance` — blinking, eyes, and a mouth driven by her voice
`TierCascade`, `ModelCatalog` — what runs here	`InterviewRoom`, `InterviewCharacter` — the glue between them, and her turn in the conversation
`TranscriptCleaner`, `ModelDownloadUrl` — tidying a transcript, finding a model's file	`InterviewVoiceInput`, `FirstRunModelFetch` — the microphone at the desk, the first-launch download
`LanguageRuntime` — the language everything follows	`DemoShell`, `DemoText` — the menu, settings and credits, and the text in ten languages

The demo assembly references the plugin and no assembly of the game around it. That is the interesting property: it was written against the public API by someone with the source open, which is how several of the documentation pages you are reading got corrected.

Running it

1. Open `Assets/Demo/Scenes/InterviewDemo.unity`.
2. Press Play. The first start loads the model — several seconds during which the equipment panel is still filling in.
3. Ask her something.

Download the text model and Qwen talker and codec through Model Manager. The package includes a Windows x64 CPU Qwen native DLL; leave **External server URL** empty to use it. This route needs no separately installed speech server, Python environment or speech executable. Run **Tools ▸ Offline AI NPC ▸ Check my setup** to check those

settings. For microphone input, also check the scene's Whisper Manager: its **Model Path** is `Whisper/ggml-small.bin`, relative to StreamingAssets. It needs `Assets/StreamingAssets/Whisper/ggml-small.bin`; **Check my setup** checks the Whisper dependency, but does not check this demo-specific model file.

Without a model the interview still runs. The terminal reports the speech link down and prints her filed deposition in place of a spoken answer; the file still disagrees with her in the same places, but she cannot be pressed or caught changing her story. The Windows build reaches this by itself: it checks the model it downloads on first launch against the SHA-256 in the catalog, and when the download fails or the file is not that one, the screen says so, names the address and the folder for placing the file by hand, and offers **Enter** to start the interview without the model or **Esc** to go back to the menu. The form's header carries `case N` — the seed that rebuilds this candidate. Set *Seed* on the Interview Room to replay one; zero draws a new case each time.

Choose the language. Settings offers English, Russian, Spanish, French, German, Italian, Portuguese, Chinese, Japanese and Korean. The interface, case file, authored dialogue and endings follow this selection, as do Whisper recognition, the model's reply language and Qwen3-TTS speech. Chinese UI uses Simplified Chinese. All ten use voice `vivian` and the same multilingual speech-model files. For the plugin's language setup, see Languages.

Desktop demo. The Interview uses a seated camera, mouse and keyboard, and screen-space menus and subtitles.

The endings

Four, from what she was crossed with what you signed, and none of them is a verdict on you:

	You cleared her	**You detained her**
She was human	she catches the 06:00 shuttle home	her sister keeps writing to the station
She was an android	She walked out. Halden logged nothing unusual that night.	they took her apart in a room down the hall

The reason to play it twice is that the other assignment reads completely differently against the same evidence.

- One reply without speech
- Stopping the Hello NPC sample
- History in your own LLMUnity controller
- The shape
- Streaming subtitles and interruptions
- The whole controller
  - The parts you must not skip
- The prompt cache
- Memory
  - Choosing a store
  - What it keeps, and what it costs
- Voices
- What this page does not cover

Dialogue: wiring a talking character

For the first answer, install compatible LLM for Unity, download one suitable language model and use **Welcome ▸ Open Hello NPC sample**. Press Play, type a question and press Enter; read the answer in the subtitle. Microphone input and speech output are optional steps after that works; Getting started separates text setup from voice errors.

For your own character, start with the canonical `Samples/HelloNpc/HelloNpcCharacter.cs`. This page explains its parts and shows guarded integration wiring. The backend and UI methods below are explicit placeholders, not a second runnable backend.

One reply without speech

Use `HelloNpcCharacter.SayText(string)` for a text-only turn, or `TrySayText(string)` to keep input when submission is rejected. For an assigned `hello`:

```
if (hello.TrySayText(playerLine))
    playerLine = "";
```

`true` means submitted, not completed. Blank input, required model setup, an inactive character or a busy/still-settling turn can reject it. Subtitles, actions, `Finished` and ordinary UI conversation history use the existing pipeline. This turn does not check speech readiness, create a speech session/clip or request synthesis. Existing `Say(string)`/`TrySay(string)` and UnityEvent bindings remain supported. The component's `Start` can still inspect/start an assigned engine: leave Engine empty if the scene should have no assigned speech engine. A typed or spoken turn and an MCP `say` take turns on the same character — while one runs, the others are refused — and a cancelled reply keeps the character busy until the model's running task has actually finished.

Stopping the Hello NPC sample

For the supplied starter component, call `HelloNpcCharacter.CancelReply()` from a Stop button or a player interruption. `StopSpeaking()` is a UnityEvent-friendly alias that stops the whole reply. The component stays enabled and its borrowed TTS engine stays running. Cancellation suppresses late tokens, actions, subtitles and `Finished`; it does not undo actions that already ran.

If the backend generation or repair Task is still pending, `IsBusy` remains true. Keep the next question and retry `TrySay` after `!IsBusy`; requesting cancellation alone does not prove that the backend finished. There is no automatic input queue. An absent AudioSource produces subtitles without creating a clip or reporting audio onset. It still drives the existing synthesis iterator; do not treat this as a mode that skips synthesis. A failed optional readiness getter or event listener does not block an otherwise available text reply.

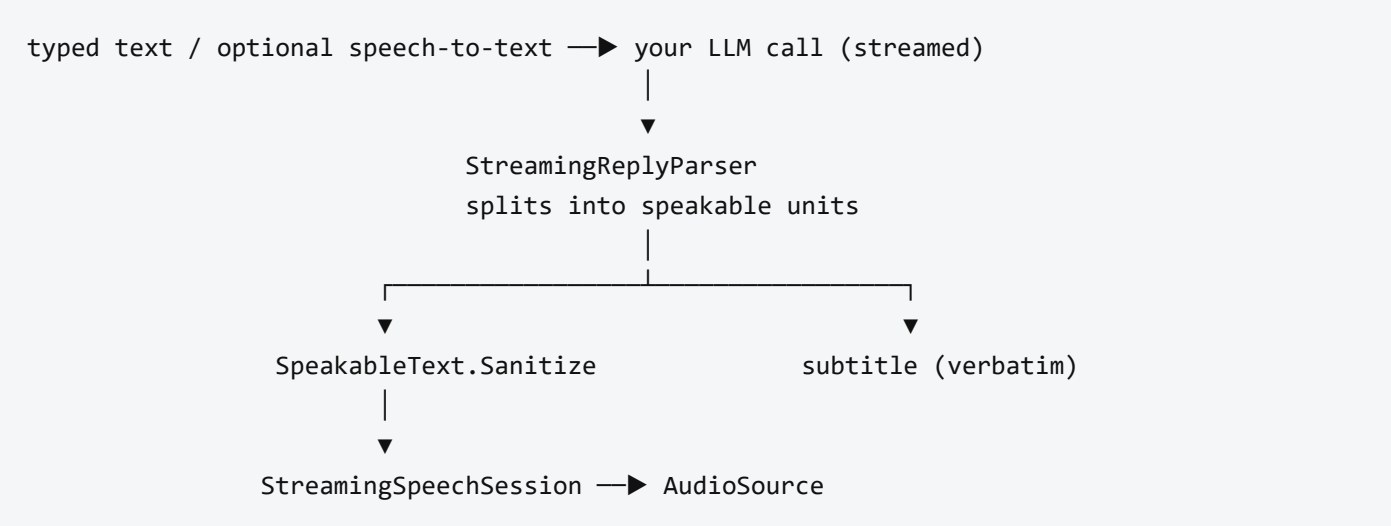
History in your own LLMUnity controller

Hello NPC already manages its turns. A custom controller can reuse `OfflineAINPC.Integrations.LLMUnityBackend.AgentConversationHistory` for opening examples and clean conversation history. Create it once for the assigned `LLMAgent`; call `SetOpening(...)` or `AppendOpeningTurn(...)` to queue your initial messages. At your controller's ready point, call `TryApplyOpening(out problem)` before the first generation. This avoids editing the copy returned by `agent.chat` or applying examples before the backend has loaded its history.

For each turn, call `BeginTurn(out problem)`, generate with `Chat(addToHistory: false)`, then call the returned writer's `TryCommit(playerLine, cleanedProse, out problem)` only if that generation succeeds and still belongs to your controller. Pass the player's actual question, without the repeated scene description used in the inference prompt. The writer refuses a changed caller or conflicting history. Discard the writer on cancellation; `Invalidate()` also discards a queued opening and invalidates old writers.

The helper neither starts a model nor saves history automatically. It records completed generated prose, which may precede audio playback. Your controller still owns readiness, one turn at a time, cancellation, and any history saving. See Hello NPC and The Interview for the complete integration.

The shape



Two things about this diagram matter more than they look.

The reply is split into sentences and spoken as they arrive. The first complete sentence can be voiced while the model writes the rest. Test the resulting input-to-playback time with your own model, voice and game workload.

For a long reply without punctuation, the parser prefers commas and spaces. If the text has neither, a text-element boundary lets speech start before generation ends, without detaching a combining mark or a surrogate pair. The trailing element stays buffered until the next token or completion, so split tokens can extend it safely. An unfinished leading `<think>` block is excluded from both speech and `ProseText`, including when generation ends inside that block.

The voice and the subtitle get different text. An emoji is fine to read and impossible to speak. `SpeakableText.Sanitize` strips what a synthesiser cannot say — emoji, markdown marks, control characters — and leaves every language's punctuation alone. The subtitle gets the original.

Streaming subtitles and interruptions

Subtitle durations use unscaled seconds. While audio is still being produced or playback is paused, the session renews the current line's deadline. The existing `onSubtitle(text, seconds)` callback can therefore receive the same text again; update its expiry time rather than adding a new transcript entry. A display with a fade-in or a line counter can use the new constructor overload's `onSubtitleExtended(seconds)` callback to renew only the current line. That duration is measured from the callback, not from the line's original start.

Use `StreamingSpeechSession.IsSpeaking` to interrupt a reply that has started. It stays true while the audio source is paused for an underrun or `AudioListener.pause`, and between units, until the reply finishes or is cancelled. `AudioSource.isPlaying` alone misses those pauses and lets the old reply resume after the player has started talking. `IsSpeaking` is false while only generating the first unit; it does not promise audible sound on every frame.

The whole controller

This example uses the public API and guards callbacks and turn ownership. It is not a standalone demo: `GenerateReply`, `CancelGeneration` and `ShowSubtitle` are deliberately empty integration methods. The empty `GenerateReply` does not call an LLM. Use `HelloNpcCharacter` for the actual LLMUnity call, the checks that refuse a turn while another runs, and optional actions, and preserve its cancellation and cleanup when customizing it.

```
using System;
using System.Collections;
using UnityEngine;
using OfflineAINPC.Core;
using OfflineAINPC.Speech;

public sealed class TalkingCharacter : MonoBehaviour
{
    [SerializeField] private AudioSource _voice;
    [SerializeField] private int _speakerId;          // which voice, when the engine has
several
```

```
    private StreamingSpeechSession _speech;
    private bool _busy;
    private int _generation;

    /// <summary>One turn: the player said something, the character answers.</summary>
    public IEnumerator Say(string playerLine)
    {
        // ONE lock, released on every exit path. Getting this wrong is how push-to-talk
        // stops working forever; a `finally` is not optional here.
        if (!isActiveAndEnabled || _busy) yield break;
        int generation = ++_generation;
        _busy = true;
        var parser = new StreamingReplyParser();
        StreamingSpeechSession speech = null;
        bool Current() => this != null && isActiveAndEnabled && generation == _generation;

        try
        {
            speech = new StreamingSpeechSession(
                engine:    () => TTSEngineManager.Instance != null ?
TTSEngineManager.Instance.Active : null,
                audio:     _voice,
                speakerId: _speakerId,
                onSubtitle: (text, seconds) => { if (Current()) ShowSubtitle(text,
seconds); },
                onWarning:  message => { if (Current()) Debug.LogWarning(message); });
            _speech = speech;

            StartCoroutine(speech.RunSynthesis());
            StartCoroutine(speech.RunPlayback());

            // YOUR llm call goes here. Feed what the callback gives you to the parser and
            // hand every finished unit to the speech session.
            //
            // MIND WHICH ONE YOUR BACKEND GIVES YOU. LLMUnity's stream callback is
            // CUMULATIVE — every invocation carries the whole reply so far — so it needs
            // AppendCumulative. A backend that streams true deltas needs Append. Getting
            // this backwards is silent and unmistakable: the character says
            // "MMaraMara VMara VeyMara Vey, 34." instead of "Mara Vey, 34."
            yield return GenerateReply(playerLine, onToken: textSoFar =>
            {
                if (!Current()) return;
                parser.AppendCumulative(textSoFar);        // LLMUnity: cumulative
                // parser.Append(delta);                   // a backend that streams deltas
                while (Current() && parser.TryDequeueUnit(out var unit))
speech.Enqueue(unit);
            });

            if (!Current()) yield break;
```

```
            parser.Complete();
            while (parser.TryDequeueUnit(out var unit)) speech.Enqueue(unit);
            speech.InputComplete();

            while (!speech.Finished && Current()) yield return null;
        }
        finally
        {
            speech?.Dispose();
            if (this != null && generation == _generation)
            {
                _speech = null;
                _busy = false;
            }
        }
    }

    private void OnDisable()
    {
        _generation++;
        try { CancelGeneration(); }
        finally
        {
            _speech?.Dispose();
            _speech = null;
            _busy = false;
        }
    }

    private void CancelGeneration() { /* cancel requests in your backend */ }
    private void ShowSubtitle(string text, float seconds) { /* your UI */ }

    private IEnumerator GenerateReply(string playerLine, Action<string> onToken)
    {
        // LLMUnity, your own binding, or a stub. The plugin does not care which.
        yield break;
    }
}
```

The parts you must not skip

`_busy` released by its owner. Each turn releases its lock in `finally` only while it is current. Disable invalidates callbacks, cancels the backend and disposes speech explicitly, since Unity can stop coroutines without running their cleanup. An old continuation must not unlock a newer reply. This project has broken push-to-talk this way more than once: the symptom is that the character simply stops responding, with nothing in the console.

Local parser and speech captures. Never feed tokens through mutable fields that a new turn replaces. The Hello NPC sample guards callbacks and repair completion, ends the action grammar on disable, and releases speech before raising `Finished`. Copy these lifecycle checks along with the basic wiring when customizing the sample.

`InputComplete()`. Without it the synthesis loop waits forever for a unit that is never coming, and the character never finishes speaking.

A failed generation must still end the session. Never read a faulted task's `Result` inside the coroutine: it rethrows and may end the coroutine before the owner has explicitly retired generation and speech. Do not rely on coroutine cleanup alone. Poll the task, check `IsFaulted`, and on failure call `_speech.Cancel()` and release the lock yourself before returning; `HelloNpcCharacter` shows the shape. If you stop the coroutines with `StopCoroutine` instead of letting them drain, call `_speech.Dispose()` so the clip in the air is released.

A missing engine is normal. The engine reference is a `Func<>` and not a stored reference because the engine is replaced when the language changes. During that switch it is null or reports not ready; the session handles both by delivering the line to the subtitle only. Do not assume it is there.

The prompt cache

An inference backend may reuse an unchanged prompt prefix when its cache and context permit it. Changing the prefix can require additional processing. Measure reply latency with your chosen model and backend.

Build stable persona and language rules once, and update them when they change. Keep the prefix stable where possible; backend settings, history and context lifetime still affect whether work can be reused.

Per-turn context (what the character can see, who is nearby) goes in the *user* message to keep the stable prefix separate. This does not guarantee that the next turn is cached or fast. See Current model usage for current setup.

Memory

`NPCMemory` keeps a rolling window of the conversation, keyed per character, over a store you choose — so it survives scene loads and restarts once you give it one that persists.

Set the store at startup, then append and replay turns:

```
using OfflineAINPC.Memory;

NPCMemory.Store = new PlayerPrefsStore();        // ships with the plugin; one line, done

NPCMemory.Append(npcId, NPCMemory.RoleUser, playerLine);
NPCMemory.Append(npcId, NPCMemory.RoleNpc,  reply);

string replay = NPCMemory.GetContextString(npcId, lastN: 10);
```

The replay is labelled `Player:`, `You:` and `Earlier:` — in English. Pass the language code and the labels follow it: `GetContextString(npcId, 10, "ru")` writes `Игрок:` and `Ты:`. A project whose player is a detective, not a "Player", sets `NPCMemory.Labels` once at startup, or `MemoryLabels.Register("en", ...)` per language; the labels go into the prompt, so they should be in the character's language and name the roles the way your game does.

Fold `replay` into the system prompt when you build it. The window is deliberately small: every remembered turn is prompt tokens on every subsequent turn, and a 4B model's attention does not reward a long transcript.

Assign `NPCMemory.Store` once at startup, before the first turn. Without a store, `NPCMemory` is disabled: `Append` records no transcript, `Load` returns no entries and `GetContextString` returns an empty string, including during the current session. Check `NPCMemory.HasStore` to confirm that a store is configured.

`LLMAgent` chat history is a separate mechanism managed by the model integration. It can keep context for an ongoing conversation, but does not enable `NPCMemory` or replace the store used for replay across scenes or restarts. Hello NPC writes that history itself: after a remembered turn completes, it adds the player's line and the reply as the player heard it — the action block and reasoning left out — and saves the history to the agent's `save` file when one is set. A cancelled or failed turn adds nothing. When the agent drops its oldest messages to fit the context during a turn, that still counts as the turn's own history; any other change made meanwhile means someone else wrote to it, and the turn is not recorded. `TurnHistory.Continues(before, after, added)` is that test, for a controller of your own that records its turns the same way.

If a read fails, the current turn proceeds without memory context and `Append` skips the write, preserving the existing transcript. The next successful append resumes from that transcript; skipped lines are not queued for retry. JSON output keeps valid Unicode pairs literal and escapes lone surrogate halves so UTF-8 persistence does not alter them.

If what is stored cannot be read — damaged, or not memory at all — the character starts with an empty memory, and the next `Append` first keeps the original under separate internal recovery keys. When that copy cannot be written and read back, nothing replaces the original. `NPCMemory.Check(npcId)` says which case a character is in without writing anything — `Missing`, `Loaded`, `Damaged` or `StoreFailed` — so a game can say "this character's memory was damaged" instead of letting it start over unexplained; `NPCMemory.SetAsideKeys(npcId)` lists the kept copies, and `Clear` removes the memory and its current recovery copies. Older recovery keys are left for manual review because they may also belong to another character.

Choosing a store

`PlayerPrefsStore` is the right first choice and the wrong last one. It is a small global blob with no save slots and no encryption, which is fine for a demo and wrong for a game that already knows how to save.

Anything else is three methods — `Get`, `Set`, `Delete` — against `IKeyValueStore`. Here is a complete one that writes JSON files under `Application.persistentDataPath`, per save slot:

```
using System.IO;
using OfflineAINPC.Memory;
using UnityEngine;

public sealed class FileStore : IKeyValueStore
{
    private readonly string _dir;

    public FileStore(string slot = "slot0")
    {
        _dir = Path.Combine(Application.persistentDataPath, "npc-memory", slot);
        Directory.CreateDirectory(_dir);
```

```
    }

    // Keys include "memory_<npcId>" and internal recovery keys.
    // Use NPC ids that can safely be used as file names.
    private string PathOf(string key) => Path.Combine(_dir, key + ".json");

    public string Get(string key)
    {
        string path = PathOf(key);
        return File.Exists(path) ? File.ReadAllText(path) : "";
    }

    public void Set(string key, string value) => File.WriteAllText(PathOf(key), value);

    public void Delete(string key)
    {
        string path = PathOf(key);
        if (File.Exists(path)) File.Delete(path);
    }
}
```

`NPCMemory.Store = new FileStore(currentSlot);` and memory follows the save slot. The same three methods reach a database, a cloud save, or the dictionary a test uses:

```
using System.Collections.Generic;
using OfflineAINPC.Memory;

// In tests: no files, no PlayerPrefs, nothing left behind.
public sealed class MemoryStore : IKeyValueStore
{
    private readonly Dictionary<string, string> _map = new Dictionary<string, string>();
    public string Get(string key) => _map.TryGetValue(key, out var v) ? v : "";
    public void Set(string key, string value) => _map[key] = value;
    public void Delete(string key) => _map.Remove(key);
}
```

The store is read when a prompt is built, and read and written once for each remembered line — a few times per turn, never per frame — so a store that touches disk is not a performance decision.

What it keeps, and what it costs

`Append(npcId, role, text)`	adds one line; roles are `RoleUser`, `RoleNpc`, `RoleOther`, `RoleSummary`
`GetContextString(npcId, lastN)`	the last `lastN` **exchanges**, formatted for a prompt

`GetContextString(npcId, lastN, languageCode)`	the same, labelled in that language
`Labels`	the labels used when no language is given — `Player:`, `You:`, `Earlier:` by default
`Clear(npcId)`	forgets that character, store included, with any copies of an unreadable memory
`Check(npcId)`, `SetAsideKeys(npcId)`	what is stored — `Missing`, `Loaded`, `Damaged`, `StoreFailed` — and the kept copies of a damaged one
`MaxEntries`	40 lines per character — older lines fall off the end
`Compactor`	optional: your own summariser, called when the window overflows

Two lines per exchange, forty entries: about twenty turns per character before the oldest starts dropping. Raise it and every turn costs more prompt tokens; that is the whole trade-off, and it is why the default is a number rather than "everything".

Voices

The language decides the default voice (Languages). With Qwen3-TTS a voice is a speaker name — `serena`, `ryan`, `ono_anna` and six more, all in one process — so two characters in one scene sound different by naming different speakers, and nothing is spawned per voice; a character that still casts by integer speaker id is served by the engine's *Speaker voices* table. With Piper each voice uses a separate process, started when needed. After the engine reports ready, `PiperEngine.PrewarmVoices()` starts the default and configured secondary voice. Startup time depends on the voice, installed executable and hardware.

A character can also name its voice outright. The session's second constructor takes a `SpeechRequest` as the voice template — `new SpeechRequest("", speakerId, languageCode, voiceId, style)` — and stamps each unit's text onto it, so the language, the voice and a delivery hint travel with every unit to an engine that understands them. With Piper the voice id is a voice file name; the speaker id keeps working as before. See Languages.

What this page does not cover

- **Speech recognition** — see GettingStarted.
- **Actions** (the character doing rather than saying) — see Actions.
- **Perception** (the character knowing what is around it) — see Perception.
- **Lip sync.** Not included; The Interview demo animates mouth frames from the playing audio.

- The short path first
- The full apparatus
- Defining an axis
- Reading and writing
- What the model is told
- Traits
- What the model may do
- Gating verbs by state
- Saving
- Pause behaviour
- Invalid authored numbers

Internal state

> **What this does, honestly.** State *colours* what a character says; it does not make the character act. Measured on Qwen3-4B: asked directly how it was at `sleepiness 95/100`, the character referred to being tired in **6 of 10** replies, against **0 of 10** with the state system switched off. On a relationship axis the same probe scored **6 of 10** against a control of **1 of 10**. It is a real, reproducible effect and a probabilistic one — expect a character who often mentions how it feels when the subject comes up, not one who always does, and not one who volunteers it unprompted in a one-sentence reply.
>
> State writes, trait modifiers and threshold rules run in deterministic C# under the conditions below. Persistence depends on the configured store and its success/failure contract; opting in does not prove a durable write. The model may change only axes opened to model writes, within their configured bounds.

The short path first

Most projects need one axis that a few game events move, with a visible consequence they control themselves. That does not need an asset, a trait preset, persistence or a tick. **It needs this:**

```
using OfflineAINPC.State;

// One axis, in code. No asset, no inspector.
var composure = new AxisDefinition
{
    Id = "composure", Kind = AxisKind.Mood,
    Min = 0f, Max = 100f, Baseline = 70f,
    Bands =
    {
        new AxisBand { Id = "cornered", Min = 0f,  Max = 30f },
        new AxisBand { Id = "guarded",  Min = 30f, Max = 65f },
        new AxisBand { Id = "settled",  Min = 65f, Max = 100f },
```

```
    },
};

var state = new NpcStateModel("suspect", new AxisCatalog(new[] { composure }));

// A game event moves it. Deterministic, exact, every time.
state.Modify("composure", -18f, reason: "pressed on a contradiction");

// And you read the band to decide what the player SEES.
string band = state.BandOf("composure")?.Id;      // "guarded"
```

That is the whole system for that case. `NpcStateModel` is plain C# — no `MonoBehaviour`, no scene, no Unity — so it can be unit-tested directly.

Adjacent bands **share their edge** (`30` ends one and begins the next). Do not leave a gap of `0.01` between them: a value that lands in the gap has no band, no words, and no answer to `AtLeastBand`. Hysteresis, not a gap, is what stops a value flapping at the edge.

Reach for the rest of this page when you want descriptor text reaching the model, several characters with different dispositions, decay over game time, persistence across sessions, or the model being allowed to move an axis itself. Each of those is a reason for one more piece of the apparatus below. If none of them applies, you are done.

The full apparatus

Three axis kinds. They are separate because they live on different timescales, and merging any two of them breaks one of them.

Kind	Lifetime	Persisted	Ticks
Mood	minutes, eases back to a baseline	no	toward baseline
Relationship	permanent, per pair	yes	never
Need	grows until satisfied	yes	away from baseline

Put a mood and a need in one bucket and either the mood stops decaying or the need starts. That is not a hypothetical; it is the usual version of this bug.

A fourth kind, `Trait`, exists so that catalogs authored with it still load. It is retired: a trait axis never ticked, was never described and modulated nothing. A character's disposition is a **Trait Preset** (below), which bends the three kinds above. The inspector and `Validate()` say so when they meet one.

Defining an axis

Create an **Axis Catalog** asset. Each axis carries its range, baseline, rate per hour, hysteresis, how far the model may push it, and its descriptor bands.

```
trust   0..20, baseline 0, no decay, model may move it ±0.5
  guarded  0..3    "is polite but keeps a little distance from you"   (always mentioned)
  warm     3..6    "is comfortable around you"
  close    6..10   "trusts you and lets it show"
  devoted 10..20   "trusts you completely"
```

(The sample catalog stops at `close`; `devoted` is the illustration's fourth band.)

Bands are listed **in ascending order**, and the order is meaning: `AtLeastBand`, `RequireBand` and the direction of a threshold event all compare positions in the list. The catalog inspector reports a list that is out of order, a band without an id or with a repeated one, a band whose max is below its min, a gap or overlap between neighbours, and a band with no text in any language — the last is the quiet one, because an axis whose band has no words still moves and never reaches the prompt.

Never send a number to the model. Given `trust: 7` a small model either ignores it or plays it as a crisis. Bands exist so a designer writes the words once.

Band text is per language, through the same `LocalizedText` the perception descriptions use. A sixth language is data.

Reading and writing

```
var state = GetComponent<NpcStateComponent>();

state.Get("trust", "player");                       // read
state.Modify("sleepiness", 5, "walked home in the rain");
state.Modify("trust", 1, "brought tea", "player");  // relationships are per pair

state.ThresholdCrossed += c =>                      // once per crossing, with hysteresis
    Debug.Log($"{c.AxisId}: {c.FromBandId} -> {c.ToBandId}");
```

Ask in **bands** rather than in numbers wherever gameplay is deciding something:

```
if (state.AtLeastBand("trust", "warm", "player") ?? false) OfferTheHug();
```

`AtLeastBand` returns `bool?`, and the third answer is the point. Null means the question could not be answered — no such axis, no such band, or a value the bands do not cover — and it is deliberately not `false`. A caller that cannot tell "I do not know" from "no" gates content on a typo and never finds out.

The reason to prefer it: a literal `trust >= 3` in your code is a copy of a number in the asset, and the copy does not move when you retune. Retune the asset and the character *describes* itself differently while *behaving* exactly as before.

The threshold event never fires on load: progress made last session is not progress made now, and replaying every milestone at every launch is how a celebration stops meaning anything.

What the model is told

`DescribeState(languageCode)` on the component renders the block that goes into the prompt:

```
Right now: this character is getting sleepy, and is comfortable around you.
```

Only bands worth saying are said — an axis resting at its baseline is silent unless the band is marked **Always mention** — the most notable few first, inside a token budget, in the character's language.

The subject is **"this character"** by default, because the plugin does not know who it is describing and a default pronoun is a default gender. Give each character its own on the component's **Subject in prompt** field — a name, or the pronoun its band texts were written for — and the block reads `Right now: Mara is getting sleepy…`. Japanese and Chinese drop the subject entirely; that is the built-in phrasing for those codes. A project that words the block itself registers a `StatePhrasing` per language, or passes one to the `DescribeState(languageCode, phrasing)` overload.

Traits

A trait shifts an axis baseline, scales its rate, and scales how hard incoming changes land. Two characters with identical axes then behave differently — without it they all converge on the same emotional shape and the axes stop meaning anything.

```
shy:        mood baseline −2, mood rate ×0.7, trust sensitivity ×1.3, sleepiness rate ×1.3
energetic:  mood baseline +2, mood rate ×1.4, trust sensitivity ×0.7, sleepiness rate ×0.8
```

"At rest" is judged against the character's own baseline, trait included: a shy character sitting at −2 is not out of sorts, it is home.

What the model may do

Add a **Feel Action Handler** and the `feel` verb appears, with `axis` as a grammar choice built from the axes that opened themselves to it:

```
{"verb":"feel","params":{"axis":"mood","delta":-2,"reason":"..."}}
```

Axes with `MaxModelDelta = 0` are excluded from the `feel` axis choice when the schema is built. With the grammar successfully applied, that choice constrains `params.axis` in a valid completed action; it does not constrain what the character may claim in ordinary prose. Runtime state checks also refuse model writes to closed axes and bound each write to an open axis, including when no grammar is available. Keep needs closed if only game code should satisfy them; a designer can explicitly open one. The normal action format allows at most one action per completed reply.

Gating verbs by state

```
NpcStateComponent.VerbGate.RequireBand("race", "sleepiness", maxBand: "drowsy",
                                       note: "too tired to run");
```

The rule removes the verb from the generated action schema when its condition fails. The component does not reach for a schema on its own: install the gate on the schema your turn uses, before the turn begins.

```
state.ApplyVerbGate(schema);        // NpcActions.Schema, or the character's
perception.Schema
turn.Begin();
```

A schema shared by several characters carries the gate of whoever installed it last, so install it for the character whose turn it is. If the backend applies that grammar, a valid completed action cannot select the verb. Ordinary prose may still mention it. Verbs are filtered by state, targets by perception, and neither knows about the other.

`Require(..., min:, max:)` takes numbers and still works; a numeric rule is re-checked on every change of its axis, not only on a band crossing. Prefer `RequireBand` all the same: a band bound resolves against the catalog every time the rule is evaluated, so moving the band's edge moves the rule with it. A numeric bound is a second tuning of the same thing, and the two drift. Put `max: 85` on a rule while the `spent` band begins at 70, and for those fifteen points a character is described as barely able to stay awake while still being offered the chance to look around.

A band the catalog does not define is **ignored loudly**, in the console, rather than enforced. Enforcing a typo would remove a verb forever; ignoring it quietly would make the gate a check that cannot fail.

Saving

Persistence is **opt-in**. Nothing is written anywhere until a project installs a store, usually from a bootstrap:

```
using OfflineAINPC.Memory;
using OfflineAINPC.State;

NpcStateComponent.Store = new KeyValueStateStore(new PlayerPrefsStore());
```

Order does not matter: a character that was already running loads its save the moment a store appears, and never writes before it has tried to load — so a late bootstrap cannot overwrite last week's trust with this session's defaults. Without a store, relationships and needs reset when the game restarts; the component's inspector says so in Play mode, because "trust does not save" a week later looks like a bug and is a missing line.

`KeyValueStateStore` wraps any `IKeyValueStore` — the same three-method interface memory uses, so one `FileStore` per save slot serves both. Persisted axes serialise as `{schemaVersion, values}`. A project with its own save format implements `IStateStore` directly, or folds the values into its own file:

```
Dictionary<string, float> mine = state.Export();   // fold into your own save
state.Import(mine);                                // restore, raising no milestones
```

```
state.Restored += RefreshPose;                    // values changed with no single event
```

`Import` means "load this slot": it replaces the persisted state wholesale, so anything earned since the last save and absent from the incoming data is gone, which is what loading a slot means. Values for axes the current catalog no longer defines are **kept and saved back** rather than dropped, with a warning naming them — a catalog edit is not a reason to lose a save, and a renamed axis is yours to migrate.

Restoring also invalidates the cached verb list and grammar, without replaying threshold events. A character's available actions immediately follow the restored state.

Saves are debounced (never more often than once a second, ten by default) and flushed on disable, on quit, and when the application pauses on mobile. Mood is session-only by design.

Passive need changes count as dirty even when they stay inside one descriptor band. Failed writes stay dirty and retry after the debounce or at the next lifecycle flush. The built-in store refuses to overwrite an existing save when its read fails; it retries the read before saving. A custom `IStateStore` should throw on an unavailable read or a failed write: `TryLoad` returning `false` means no usable save was found, and `Save` returning normally means the write succeeded. The built-in PlayerPrefs adapter also reports a refused `SetString` or `Save` flush to the state component, while retaining the original non-throwing `Set` API. A future-schema save remains protected.

A save that cannot be read is not used: the character starts from its defaults, and before the next save replaces it, `KeyValueStateStore` keeps the original under separate internal recovery keys. When that copy cannot be written and read back, nothing replaces the original. `KeyValueStateStore.Check(npcId)` says which case a character is in without writing anything — `Missing`, `Loaded`, `Damaged`, `Newer` (written by a newer plugin, never replaced) or `StoreFailed` — and `SetAsideKeys(npcId)` lists the kept copies; `Clear` removes the save and its current recovery copies. Older recovery keys are left for manual review because they may also belong to another character.

Pause behaviour

Ticks accumulate scaled `Time.deltaTime`, so at `timeScale = 0` nothing accumulates and nothing ticks — a character does not get sleepier while the player is in a menu, and no timer spins. Perception freezes with it, for the same reason: both model game time.

The speech queue deliberately does the opposite and keeps running on unscaled time, because `AudioSource` ignores `timeScale` and a line already sounding does not stop. See Limitations for the full per-subsystem table.

Invalid authored numbers

Axis ranges, baselines, rates, hysteresis, model delta limits and band boundaries must be finite numbers. Trait baseline shifts and rate/sensitivity multipliers must be finite too. `NaN` and positive or negative infinity are configuration errors. Catalog validation and the catalog inspector identify the affected fields. Trait asset validation reports its findings in the Console. Runtime numerical safeguards also apply to plain-data traits. Correct the source settings instead of relying on a fallback for normal gameplay.

The runtime checks the current definitions when using them, including edits made after creating a state model. An invalid baseline uses zero, clamped to a valid axis range. An invalid trait shift uses zero; an invalid trait multiplier uses one. A nonfinite axis rate disables ticking, a nonfinite model delta limit disables model writes, and nonfinite hysteresis

uses the existing default of `0.08`. Bands with nonfinite boundaries do not match values. Finite tuning values retain their existing policy; these safeguards do not rewrite the authored assets or introduce a new tuning cap.

A valid range has finite endpoints and `Max > Min`. With an invalid range, game, model and tick writes are refused and keep the stored value. An unset axis reads zero. Restoring an existing finite saved value preserves that number without clamping it through the broken range and without raising value-change or threshold events. This keeps relationships and needs available for the next save while you repair the catalog. With a valid range, restore still applies the usual clamp. Saved numbers are not automatically reclamped when you edit the catalog; a later ordinary write uses the repaired range.

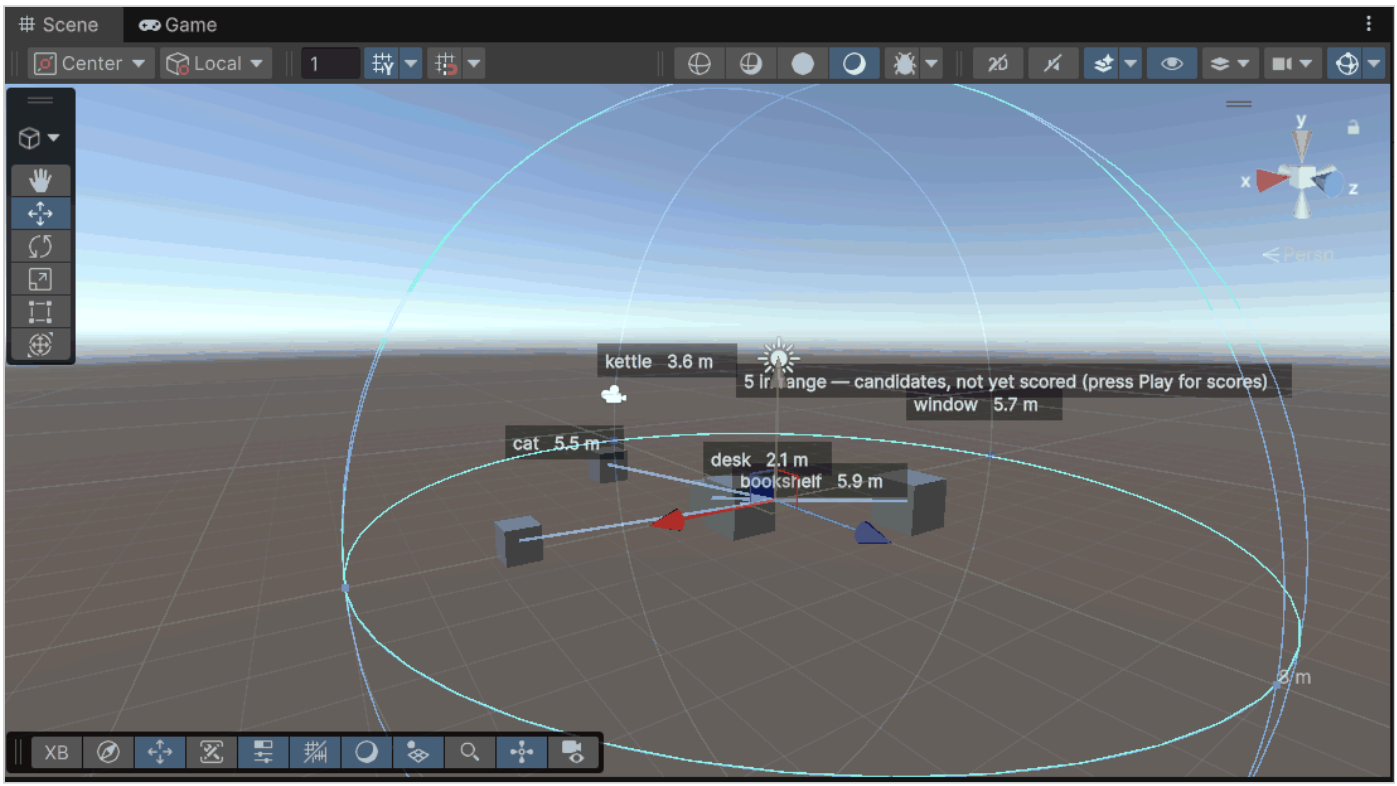
Arithmetic that overflows to a nonfinite result is refused before publishing a new state value. An overflowing computed rate or tick step leaves the current value unchanged. Notability remains finite in `0..1`, using wider intermediate arithmetic for valid large ranges and zero for an invalid range. These are data protections, not proof that an extreme configuration makes useful gameplay. The save schema, NPC ids, relationship keys and unknown-axis preservation policy are unchanged.

The NPC State component inspector also reports problems in its assigned catalog and trait preset. When several different assets are selected, it reports that validation is undetermined instead of judging one arbitrary reference. Select a single character or its referenced asset to inspect and repair the field.

- The object
- The character
- How it reaches the model
- Receiving an interaction
- Making a whole scene visible
- Debugging
- Snapshot identity and registry callbacks

Points of interest

Make an object noticeable: add **AI Point Of Interest** to it. That is the whole setup.



Select a character and the radius is a handle you can drag. Everything inside it is a candidate; what reaches the prompt is the most salient handful — distance, priority, facing and recent interaction weighed together — recomputed a few times a second, and once more by your dialogue code just before each turn.

The scene view answers two different questions depending on whether the game is running:

- **Not running** — every point of interest inside the radius, named, with its distance. These are candidates and they are deliberately not scored: distance is knowable while you are placing furniture, line of sight and priority are not, and a number invented in edit mode would be a guess wearing the clothes of a measurement.
- **Playing** — the objects actually in context, each with the score that put it there, the line thicker for the ones in view. This is the answer to "why is the character talking about the bookshelf and not the bed".

There is no list to register it in, no manager to tell. The component adds itself to a static registry while it is enabled and removes itself when it is disabled or destroyed, so object pooling, additive scene loads and scene unloads are already correct.

The object

Field	What it does
`id`	What the model writes to name it. Short, unique, obvious. Derived from the object name if left empty.
`description`	What it is, per language.
`priority`	Higher is more worth mentioning. 0 is ordinary scenery.
`aimPoint`	Where to look at and measure from. Empty uses the centre of the object's renderers, which is what a person would point at.
`affordances`	Verb ids this object accepts; any other verb aimed at it fails validation and gets the repair round. Empty means all of them.
`states`	Authored states selected at runtime: `poi.SetState("open")`.
`On Npc Interact`	UnityEvent raised when an NPC acts on it — inspector wiring, no code.

Ids matter more than they look. The model picks a target by writing its id, so two objects called `chair` means it cannot reliably mean either. Over-long and unusable ids are warned about in the inspector; duplicates are reported by the inspector, which sees the whole scene at once, and at runtime by `PoiRegistry.DuplicateIds()`. A name the grammar cannot hold — «Кружка», 「マグカップ」 — still gets a stable id (`poi_` plus a short hash), so an object never silently fails to register.

Long derived names are shortened to the 48-character grammar limit with a stable hash suffix; existing valid authored ids keep their spelling. Each recompute refreshes changed scores, distances and order without raising `Changed` for a mere rank change. Replacing a pooled instance under the same id updates the live target and raises that event. Context rendering also excludes an object disabled since the last snapshot.

A point of interest is also an action target. It implements `IActionTarget` and `IAimTarget`, so a verb can be aimed at it without adding an Action Target component as well — see Actions. The reverse does not hold: an Action Target is addressable but not automatically included in the perception context. It can be offered as an action target; this does not prevent the model from mentioning it in ordinary prose.

The character

Add **Perception Query** to the NPC.

```
score = w·proximity + w·priority + w·facing + w·recency-of-interaction
```

Weights are serialized data. Two rules are not negotiable:

- **Occlusion excludes.** Something behind a wall is not low-scoring, it is absent. A character remarking on an object it cannot see reads as broken no matter how the weights are tuned. Occlusion is checked only against the **Occluders** layer mask, which is empty by default — set it to your walls and furniture, and the inspector reminds you until you do.
- **Only the top N reach the prompt** (default 6, hard cap 15). Small models degrade quickly once a prompt turns into a list, so this is a quality control. The second limit is the token budget of `DescribeContext` (100 by default): when the description does not fit, the block is cut and says so.

Facing is soft by contrast — something behind the character is less salient, not invisible, because a head can turn.

How it reaches the model

The result of one recompute produces two things at once:

```
Nearby: a mug of cocoa (still warm) [mug], the record player (playing) [record_player].
```

…and the target enum of the action grammar, built from that same list. When this grammar is successfully applied, a valid completed action's `action.target` is restricted to the rendered shortlist. **Free-form prose is not restricted to those objects**: perception supplies context, not a guarantee against invented scenery. This is one turn's captured view; objects can move, be disabled or be replaced afterwards, so execution still needs current target and gameplay checks.

The block rides with the turn and is never written to the conversation transcript: it describes one moment, and a transcript would repeat it later as though it were still true.

Your dialogue code makes the hand-over. Just before generating:

```
perception.Recompute();                                          // the view of this moment
string nearby = perception.DescribeContext(languageCode);    // "Nearby: a mug of cocoa
[mug], …"
string turnText = nearby + "\n" + playerLine;                // the user message, not the
system prompt
var turn = new ActionTurn(perception.Schema, dispatcher, sink, npcId, languageCode);
```

The system prompt is what the inference engine caches, so a block that changes every turn belongs with the player's line. `perception.Schema` is the action schema whose targets are exactly the ids that block named; the rest of the turn is in Actions. The Hello NPC sample does not read perception; the World Aware sample shows the block and the schema without a model.

When there is more than fits. The query keeps the top entries by salience and closes the list with a per-language tail — `…, a lamp [lamp], and other things.` — so the model knows the list was cut rather than believing the room holds four objects. The snapshot says the same in code: `Omitted` is how many were dropped, `IsTruncated` whether any were. With nothing in range the block is one short sentence (`Nothing notable nearby.`) rather than an empty header. `DescribeContext(languageCode, header, maxTokens)` is the call that renders it; the header is optional and the token budget is where the cut happens.

Receiving an interaction

```
public class Kettle : MonoBehaviour
{
    public void OnNpcInteract(NpcActionContext ctx)    // wired on the POI in the inspector
— UnityEvent lists public methods only
    {
        Debug.Log($"{ctx.SpeakerId} did {ctx.Verb} to {ctx.Poi.Id}");
    }
}
```

If the object vanished between generation and dispatch, nothing is raised and nothing throws.

The mechanism underneath is `IActionReceiver`: the action dispatcher delivers every action to the character's handlers, and a target that implements the interface also receives the actions aimed at it. `AIPointOfInterest` implements it by raising the inspector event above, so "when the character opens this door, play the animation" lives on the door.

Making a whole scene visible

```
PoiBulkAttach.AttachToLayer("Props");     // or AttachToTag("Interactable")
```

Idempotent. Ids are de-duplicated against the live registry and within the call. `AttachToLayer` includes inactive objects unless you pass `includeInactive: false`; `AttachToTag` sees only active ones — that is how tag lookup works in Unity — so call it after the scene has settled.

Debugging

- Select the NPC: the perception radius and lines to everything currently in context.
- `perception.Explain(poi)` answers "why did the character not mention the mug?" with the actual reason — too far, behind which object, or simply outranked.

Snapshot identity and registry callbacks

An item captures its id when created. `Initialize` on a live point of interest does not change ids in a snapshot you already hold; the next query recompute can detect the new identity and raise `Changed`. The snapshot copies the supplied array and exposes its items through a read-only list. `Poi` itself remains a live reference: transforms,

descriptions and enabled state are not frozen. Context rendering excludes a reference whose live id or registry entry no longer matches the captured id until it is recomputed.

If a registry observer initializes or enables/disables that object inside a notification, the newest intent owns its registration. An older callback unwinding cannot restore an obsolete identity or remove the newer registration. Existing public initialization and registry methods retain their signatures. Check actual pooling and scene transitions in your target player before release.

- What a verb is
- Register a verb
- Register a target
- Receive an action
- Wiring a turn
- Guarantees
- Backends

Structured actions

An NPC reply can carry two things at once: natural speech for the voice, and a machine-reliable decision your game can execute.

```
{"verb":"look_at","target":"window"}
It stopped raining. Come and see.
```

With a grammar binding on the character (an **LLMUnity Grammar Binding** driven by `ActionTurn`, below), a valid completed output follows the successfully applied **grammar**. Without a binding, or if grammar application fails, the parser still reads the action and rejects what does not fit. The rest is ordinary prose and flows through the streaming speech pipeline sentence by sentence.

Use grammar constraints for structure, and parser/handler checks for validity in your game. llama.cpp masks tokens that the applied grammar excludes. This does not guarantee a complete reply after cancellation, a generation error or a length cutoff, and it does not make free-form prose factually correct.

What a verb is

A verb is a word you invent, and the plugin ships one. `look_at`, `emote`, `open` in these examples are not built in — they exist because somebody wrote a `Register` line for them, the way `"jump"` exists in your input map because you added it. The one shipped verb is `feel`, the character moving its own mood within the allowance you set — see State; it is registered by the state component and absent otherwise.

Three parts, and only the first is required:

Part	What it is	Example
verb	the name of a thing the character can decide to do	`look_at`
target	*what* the character decided to do it to, chosen from objects that exist right now	`window`
parameters	the details of the decision	`kind: "shy"`

A verb is not behaviour. Registering `open` does not open anything and does not teach the model what opening means; it does four things:

1. puts the word into the **grammar**, so the model may emit it and may not emit anything else;
2. shows your one-line description to the model, which is all it knows about when to choose it;
3. makes the parser accept `{"verb":"open", …}` and reject a target or parameter the verb never declared;
4. gives the dispatcher something to route to **your** handler.

The behaviour is the handler you write. Nothing happens until you write it — a registered verb with no handler is a decision the character can make and the game will ignore, which logs once per verb and costs nothing else.

Register only what your game can carry out. Every verb in the grammar is a thing the model will eventually choose, and a character who decides to do something no code implements is worse than one who never had the option.

Register a verb

```
using OfflineAINPC.Actions;

NpcActions.Register("look_at", "look at something you mention",
TargetRequirement.Required);
NpcActions.Register("emote", "let a feeling show", TargetRequirement.None,
                    ActionParameter.Choice("kind", new[] { "happy", "shy", "sad" }));
```

Prefer `Choice` over `Text` wherever the set is knowable: a choice becomes a closed alternation in the grammar, so the model cannot invent a value you have no code for. Free text is still shape-safe — no quote, no backslash, no control character and no brace can appear in it — but its *content* is your handler's problem.

Verb names, parameter names, target ids and choice values all become literals in the grammar. Letters, digits, `_`, `-` and `.` for names and ids; anything but a quote, a backslash or a brace for choice values. Anything else is refused at registration with an exception, because a grammar that fails to compile at runtime would fail on the player's machine, not yours.

Optional parameters are independent of each other: a verb with optional `a` and `b` can arrive with neither, either, or both, in the declared order. Required parameters come first.

Register a target

Add an **Action Target** component to the object and give it a short id, or register one directly:

```
NpcActions.Targets.Register("window", "the tall window by the beds");
```

An Action Target with an empty id derives one from the object's name the same way every time — "Tall Window (2)" becomes `tall_window_2` — so two objects with the same name collide visibly in the console rather than one of them silently winning.

A disabled target is removed from the registered set used to build the next grammar. With that grammar applied, a valid action's `action.target` must come from its target enum. This does not stop ordinary prose from mentioning an absent object, and the world can change after the schema was built.

If the object already has an AI Point Of Interest, it is already a target. Perception's component implements the same `IActionTarget`, so adding an Action Target beside it registers one object twice under two ids — and the model picks between them by writing an id. Choose one: Action Target for something a verb can be aimed at, AI Point Of Interest for something the character should also be able to *notice* — see Perception.

Receive an action

Add an **Action Dispatcher** to the character. Four ways in:

```
using System.Collections.Generic;
using OfflineAINPC.Actions;
using UnityEngine;

// 1. UnityEvent — wire it in the inspector, no code; AddListener from code fires too.

// 2. Code, one verb:
dispatcher.AddHandler("look_at", a => { var at = a.Target.AimPointOf(); if (at)
transform.LookAt(at); });

// 3. Code, an interface — registered with the dispatcher while enabled:
public class Doors : MonoBehaviour, IActionHandler
{
    [SerializeField] private ActionDispatcher _dispatcher;
    public IEnumerable<string> HandledVerbs => new[] { "open" };
    public void Handle(NpcAction action) => Open(action.Target);   // already resolved
    private void OnEnable()  => _dispatcher.AddHandler(this);
    private void OnDisable() => _dispatcher.RemoveHandler(this);
}

// 4. On the object itself: a point of interest raises its own OnNpcInteract event,
//    so "when a character uses this mug" is wired on the mug.
```

`NpcAction` carries the resolved target object, not just its id, and parameters already validated against the verb. Your handler still owns current gameplay preconditions. Ask `action.Target.AimPointOf()` where the target is: it answers null for a target that was destroyed between generation and dispatch instead of throwing.

A handler that throws is contained and logged: it loses its action, never the conversation. A custom target whose `IVerbRestrictedTarget.Accepts` callback throws produces an `Invalid` parse result with a diagnostic; it does not escape the parser.

Wiring a turn

Registering verbs and handlers is half the work. The other half happens once per reply, and every project used to assemble it by hand with three mistakes available: leaving the grammar applied after the turn, dispatching the same block on every streaming callback, and skipping the repair round. `ActionTurn` is that assembly:

```
using OfflineAINPC.Actions;
using OfflineAINPC.Core;

var sink       = GetComponent<IGrammarSink>();        // LLMUnity Grammar Binding, or null
var dispatcher = GetComponent<ActionDispatcher>();
var parser     = new StreamingReplyParser();

using (var turn = new ActionTurn(NpcActions.Schema, dispatcher, sink, npcId, languageCode))
{
    turn.Begin();                                      // grammar on

    string reply = await agent.Chat(playerLine, textSoFar =>
    {
        parser.AppendCumulative(textSoFar);
        turn.TryDispatch(parser);                      // acts on the block at most once
        while (parser.TryDequeueUnit(out var unit)) speech.Enqueue(unit);
    });

    parser.Complete(reply);
    turn.TryDispatch(parser);

    if (turn.NeedsRepair)                              // an action was attempted and was
wrong
    {
        turn.BeginRepair();                            // action-only grammar
        string fixedUp = await agent.Chat(turn.RepairPrompt, null, null, addToHistory:
false);
        turn.CompleteRepair(fixedUp);                  // dispatches, or gives up
    }
}                                                       // grammar off, whatever happened
```

Fold `NpcActions.Describe(languageCode)` into the system prompt, once, when you compose it; that is what tells the model what the verbs *mean* — the grammar only says what is expressible. With a per-character perception schema, use that schema and `DescribeVerbsOnly` instead, as Perception explains.

The HelloNpc sample does exactly this; copy it. The repair generation runs while the speech is already playing, asks for the action alone, and stays out of the conversation history.

Guarantees

- The action block is **optional** — most replies are pure conversation, and the grammar never forces one.

- A **valid completed reply under the successfully applied normal reply grammar** has a required prose segment that does not start with `{`. This is a format constraint, not a guarantee of a visibly nonblank subtitle or a promise of speech after cancellation/error, a truncated generation or an absent/failed grammar binding. The action-only repair format is separate.
- A malformed block degrades into speech rather than swallowing the line; a leading block that names no `verb` (an emoticon in braces) is not an action and is not repaired.
- A rejected action gets **one** corrective round, then is dropped with a log line. `CompleteRepair` accepts a result only while `NeedsRepair` is true. It cannot start an action before a rejected block, repeat a valid action, or retry a receiver that threw after changing game state. Receiver exceptions do not undo those changes; your handler owns gameplay transactions and preconditions.
- The prompt frame — "Available actions", "never read the action line aloud", the repair prompt — is in the character's language for the five shipped codes (`ActionPhrasing`), and a sixth is a `Register` call.

Backends

The core does not reference any LLM library. Grammar reaches the backend through `IGrammarSink`; the adapter for LLMUnity ships in the optional `OfflineAINPC.LLMUnity` assembly, which compiles only when LLMUnity is installed. Game code finds it through the interface:

```
var sink = GetComponent<IGrammarSink>();     // null when no backend is installed
```

Note for backend authors: in llama.cpp the grammar is sampler state on the client, not a per-request argument. It persists until cleared, so whatever sets it must clear it — which is why `ActionTurn` is disposable.

- Tier 1 — ten languages from one installed engine
  - Bundled Qwen native CPU
  - Optional advanced: a server you own
- Tier 2 — works, bring your own voice
  - Adding a language, start to finish
  - Two things that will bite you
- Tier 3 — not viable
- Switching at runtime
- Per-character voices in one language
- Bring your own engine
  - The contract
  - A streaming engine
  - The worked example: VOICEVOX
  - The whole engine, copy this file
  - Licensing: apply the test, do not take a list

Which languages does this support?

Choose the character's reply language, microphone recognition language and voice language together. A text-only character needs a suitable language model; microphone input and speech output are optional.

Built-in language definitions cover English, Spanish, Japanese, Russian and Simplified Chinese. Qwen3-TTS CustomVoice offers nine ready speakers across ten supported languages. Piper uses a separately chosen voice for each language. Pick the exact voice and check its terms; see Licensing.

In 1.1, choose a ready CustomVoice speaker. Creating your own voice from a description (VoiceDesign) is not part of this version.

Tier 1 — ten languages from one installed engine

Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no manually started HTTP server, executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.

Language	Speaker (default)	Configured by the plugin
English	`serena`	yes
Japanese	`serena`	yes
Spanish	`serena`	yes
Russian	`serena`	yes

Language	Speaker (default)	Configured by the plugin
Chinese (Simplified)	`serena`	yes
French, German, Italian, Portuguese, Korean	`serena`, or another ready speaker	add a Language Definition — the voice already works

All nine preset speakers can speak the model's supported languages. The table below gives their native language as listed in the model card:

Speaker	Voice	Native language (model card)
`serena`	warm, gentle young female	Chinese
`vivian`	bright young female	Chinese
`ono_anna`	playful female	Japanese
`sohee`	warm female	Korean
`ryan`	dynamic male with rhythm	English
`aiden`	sunny American male	English
`uncle_fu`	seasoned male, mellow timbre	Chinese
`dylan`	youthful male, Beijing dialect	Chinese (Beijing)
`eric`	lively male, Chengdu dialect	Chinese (Sichuan)

Four female voices and five male. The upstream model card recommends each speaker's native language for best quality. Pick per character with *TTS voice id* on the language definition, per character in code through `SpeechRequest`, per legacy speaker id with the engine's *Speaker voices* table — or name none. All built-in Qwen language definitions and presets use `serena`; the engine also falls back to `serena` for every supported language. Voice precedence is request → *Speaker voices* row → engine *Default voice* → language definition → fallback. Set *Default voice* to `serena` and keep any request or speaker-row overrides consistent if a character should keep that voice when changing languages. Existing saved catalogs and explicit character choices are preserved on upgrade; changing the built-in defaults does not reset them. A *Style* on the engine, or `SpeechRequest.Style`, is sent as a delivery instruction ("whispering", "cheerful, quick") and honoured by the instruct-capable model variants — officially the separately configured 1.7B CustomVoice talker; the Qwen team lists no instruction control for the 0.6B.

The bundled native route sends language per request to its pinned qwentts.cpp implementation. A region/script suffix is accepted; blank or unknown native codes select `auto`. An explicitly configured HTTP server may instead omit an unknown code and retain its own default. Check that server separately.

Piper is still supported, as the second engine — the package includes its driver, and Piper itself is a program you install: a project that wants its public-domain English voice, or a Piper voice for a language Qwen3-TTS lacks, sets engine `piper` and a voice file on its language definition. Everything Piper-specific — the paired `.onnx.json`, espeak-ng's licence when you redistribute it — applies only then; see Licensing.

Bundled Qwen native CPU

The package includes the Windows x64 CPU Qwen speech library. Its target CPU must support **AVX2, BMI2, FMA, F16C and SSE4.2**. macOS and Linux libraries are not included.

1. In **Tools** → **Offline AI NPC** → **Model Manager**, download the **0.6B CustomVoice Q4 talker** and **Q8 codec** under *Voices and speech models* (about 0.9 GB together).
2. Use the scene's **Qwen3-TTS Engine** through **TTS Engine Manager** and its language binding. The sample supplies this wiring; a custom character must select that engine.
3. Leave **External server URL** and **Style** empty. Use `serena` in **Default voice** to keep the same voice across languages. A character's `SpeechRequest` or **Speaker voices** entry can override it. Set the language separately; choosing a voice does not translate the text.
4. In Play, type a short prompt and check the reply's subtitle and speech in the chosen language. Repeat this check in your built Windows player with the model files available.

The default route loads the DLL in Unity and streams PCM to the existing speech session. It does not start `tts-server.exe`. The native profile is CPU-only with eight threads. **StopAsync** waits for native generation and worker settlement before releasing the context. Cancellation requests cooperative cancellation; it is not an instant-stop guarantee.

A separately launched server and **External server URL** / `AttachTo(url)` are optional advanced alternatives. Keep that server's own model, backend, version and distribution requirements separate.

Voice precedence stays: request → Speaker voices entry → Default voice → language definition → built-in fallback. The model reports nine ready speakers; names are checked without case sensitivity. Streaming retains the existing lead buffer and playback rules. Same-engine language changes do not load another Qwen context; requests use the selected language and speaker.

The bundled native route allows only one initialized context per process and serializes synthesis requests; see Concurrency.

Optional advanced: a server you own

The bundled DLL is the normal Qwen route. Use **External server URL** or `AttachTo(url)` only when deliberately connecting a separately launched compatible HTTP server, for example an independently checked GPU setup. A nonempty URL keeps the HTTP transport; the component does not own or stop that server. A remote URL sends dialogue off the player's machine. Preserve a working local configuration unless you intend to change that boundary.

The pinned source reference is qwentts.cpp `6fae92914045cd83364d2845ceaa0f7969727319`. Build and distribute your chosen server separately under its exact licences. The bundled DLL remains CPU-only and Windows x64; an external server has its own platform and GPU requirements.

Tier 2 — works, bring your own voice

A language outside the ten Qwen3-TTS languages needs another voice engine or a licensed Piper voice. Check that your chosen language model and multilingual Whisper model support the language; their wider language lists do not guarantee useful accuracy in your game. Configure its prompt, recognition code and voice in a Language Definition, then add that definition to your Language Set.

Test representative questions and replies in Play and in your target Player. `Tools ▸ Offline AI NPC ▸ Live verification` can run a whole turn; supply a test line when there is no built-in one. A voice check is marked verified only after you confirm what you heard.

The plugin's built-in catalog configures five languages. The Interview configures all ten listed above, including its interface and case text. A model's language capability and a game's translated UI are separate requirements.

Adding a language, start to finish

1. **Get a voice.** If Qwen3-TTS speaks the language (see Tier 1), you already have one: pick a speaker name. Otherwise Piper publishes voices for around fifty languages at `huggingface.co/rhasspy/piper-voices` — **download both files**, `name.onnx` and `name.onnx.json`. The current Piper driver refuses an absent or invalid paired config and names the problem; setup checks flag it before you try a reply.
2. **Put a Piper voice** in `StreamingAssets/piper/voices/`. A Qwen3-TTS speaker needs no file.
3. **Create the language asset**: `Create ▸ Offline AI NPC ▸ Language Definition`. Fill in:

Every field on the asset, in the order the inspector shows them. Nothing here is hidden and nothing is optional-but-secretly-required:

Field	Example	Why
Identity		
Code	ko	The identity. Keep it stable — everything keys on it
Display name	한국어	Shown in your language picker
Speech		
TTS engine id	`qwen3-tts`	Which engine speaks this language: `qwen3-tts`, `piper`, or the id of your own
TTS voice id	`serena`	A Qwen3-TTS speaker name; for Piper, the file from step 1; for your own engine, whatever it understands

Field	Example	Why
STT model id	`ggml-small.bin`	Metadata for recognizers that use this field. Built-in Whisper chooses Model File Name or the current preset at startup
STT language code	`ko`	Handed to recognition. Usually Code without the region part
Prompting		
Persona header	`한국어로만 대답하세요.`	How the persona block is introduced — *in that language*
State header	`지금:`	Header of the character's internal-state block ("Right now"). Also in that language
Nearby header	`주변:`	Header of the nearby-objects block
Target hint	`대상은 위 목록의 id 중 하나입니다.`	The sentence that says an action's target is one of those ids
Reply length		
Length instruction	`한 문장으로 짧게 대답하세요.`	The rule the model reads, written in that language. Leave it empty and the language directive carries no length sentence
Length unit	`Characters`	**Words are meaningless without word spacing**
Length target	`30`	Characters for CJK, ~12 words otherwise
Max tokens	`48`	The hard stop. This is what actually bounds the reply — the instruction is a request, this is a limit

Field	Example	Why
Max sentences	1	Where the reply is cut if the model runs past its instruction anyway

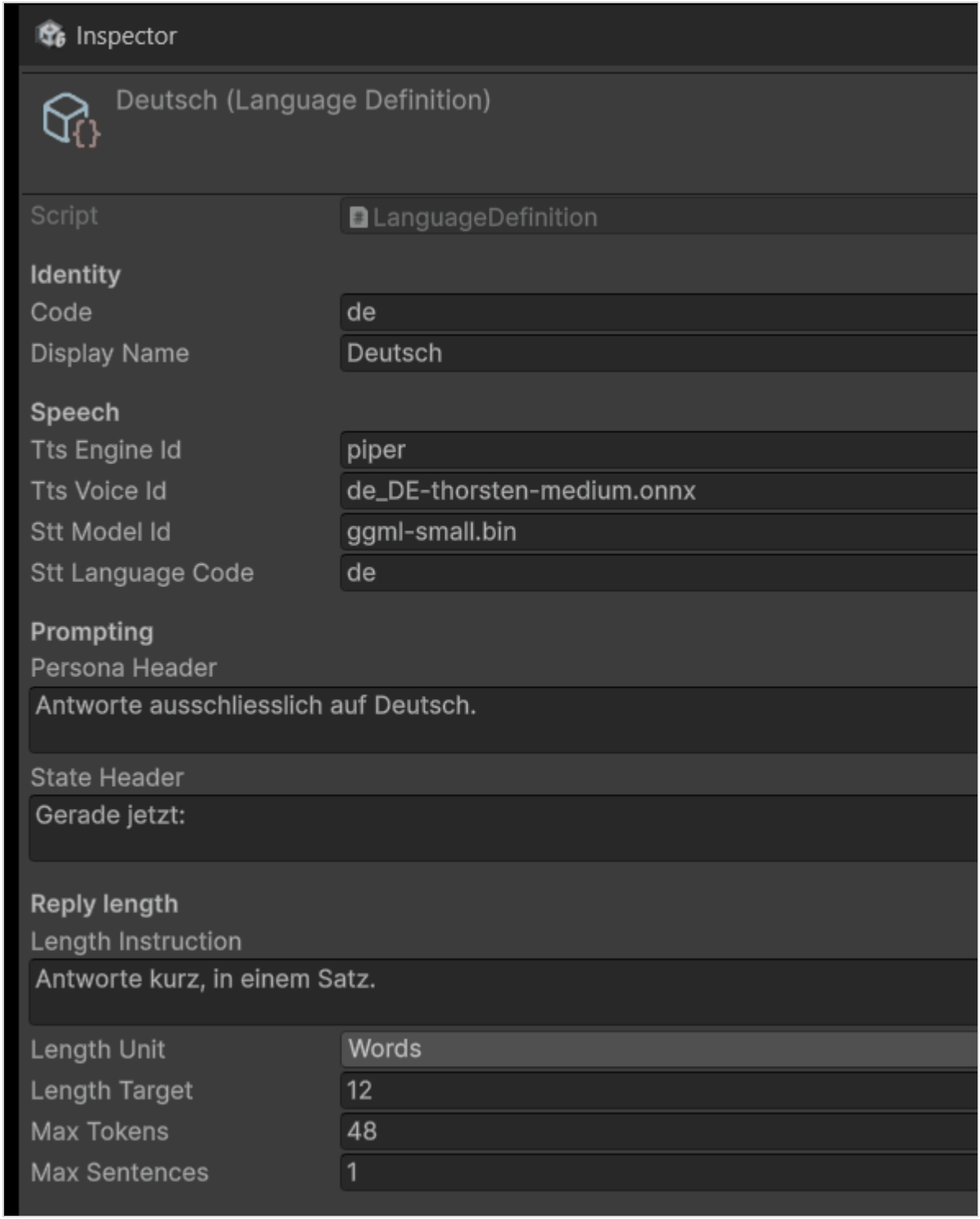
The last two are in bold because Limitations and Troubleshooting both name them as the only thing that genuinely controls reply length. A prompt asking for brevity is not a limit; these two are.

They are data until dialogue code applies them. On a **Hello NPC Character**, set **Reply Limits** to **Current Language**: this language's **Max tokens** becomes the token ceiling of each reply, and its **Max sentences** ends what is spoken, shown and remembered. Your own dialogue code does the same in a few lines:

```
csharp var spec = LanguageRuntime.Current; var profile = spec.LengthRule.ApplyTo(Tiers.Sampling); host.ApplySampling(profile); // Max tokens reaches the model reply = profile.LimitSentences(reply); // Max sentences ends the reply systemPrompt += "\n\n" + LanguageDirective.For(spec); // persona header + length instruction
```

The persona header and the length instruction reach the model only through that last line. Hello NPC speaks the language of its own system prompt and does not add them.

A filled-in asset using a separately supplied German Piper voice. Qwen3-TTS also supports German; this screenshot illustrates choosing another engine and voice:



The voice in that screenshot, `de_DE-thorsten-medium`, is an illustration and **not a recommendation**: apply the provenance test in Licensing to any voice before you ship it, including that one. The plugin cannot know what game you are making.

1. **Add it to your Language Set** and assign that set at startup:

```
csharp LanguageRuntime.Catalog = myLanguageSet.ToCatalog();
```

1. **Nothing to bind.** The TTS manager routes the code to the engine its definition names (`INamedTtsEngine`); a row in the binding table is needed only to override that.

That is the whole process. There is no list of approved languages in the code, and **the plugin does not gate on this page** — if you have a good Korean voice and a model that handles Korean, add it and it works.

The definition also carries the prompt's connecting words for that language — the header of the state block, the header of the nearby-objects list, the sentence that says an action target is one of those ids. Leave them empty for the five shipped codes; fill them for a new language, or the model reads English fragments inside a Korean prompt.

Two things that will bite you

Write the instructions in the target language. If a specific script is required, name it explicitly and check the model's replies.

Count characters, not words, for CJK. "Twelve words or fewer" cannot be evaluated in a language without spaces between words. The length unit exists for this.

Tier 3 — not viable

Languages outside what the model or recognition can do: very low-resource languages, most constructed languages, historical languages, and anything with no meaningful presence in training data. The model will produce something; it will not be good, and no configuration fixes that.

If a language matters to your game and you are unsure which tier it is in, test it before committing. Twenty minutes with the verification window is cheaper than finding out after release.

Switching at runtime

```
LanguageRuntime.SetCurrent("ko");
```

The TTS engine follows the runtime by default. Whisper receives the selected recognition language but keeps its loaded model; use a multilingual model when players can switch languages. A project that keeps the current language somewhere of its own wraps it in an `ILanguageSource` and assigns `TTSEngineManager.LanguageSource` instead; `LanguageRuntime.TrySetCurrent` refuses a code the catalog does not configure, where `SetCurrent` accepts it and warns once.

Changing language can require additional prompt processing. See Limitations for switching behavior. In the DLL route, Qwen3-TTS switches in place — one native context serves every supported language; Piper stops and restarts with the new voice file. Rapid changes are coalesced, so dragging a settings dropdown does not queue five restarts. If a language has no usable engine, `TTSEngineManager.Active` becomes null and the previous engine stops; the manager does not keep speaking with the previous language's voice. Fix the binding and select the language again to recover. **Switch on a menu screen, not mid-conversation.**

To ask what is configured rather than assuming:

```
if (!LanguageRuntime.IsConfigured("ko"))
    // ... offer the download, or fall back
```

`IsConfigured` checks the configured catalog. `LanguageCatalog.Find` returns null when no matching language is configured. `FindOrDefault` falls back to the default language, so do not use it to check whether a language is configured.

Per-character voices in one language

A language's voice is the default. A character can override it — that is how two English speakers in the same scene sound different. See Dialogue.

Bring your own engine

The package supports two engines — Qwen3-TTS, which the built-in definitions select, and Piper — and neither is the only one you can use. The seam for adding another is deliberately small.

The contract

```
public interface ITTSEngine
{
    bool IsReady { get; }
    Task StartAsync();
    Task StopAsync();
    IEnumerator Synthesize(string text, int speakerId, Action<AudioData> onAudio);
}

// Optional, when one engine serves several languages:
public interface ILanguageAwareEngine
{
    void SetLanguage(string languageCode);
    bool RestartOnLanguageChange => true;   // false: told the language, keeps running
}

// Optional, when the engine can hand audio back in pieces:
public interface IStreamingTTSEngine : ITTSEngine
{
    IEnumerator Synthesize(SpeechRequest request, Action<AudioChunk> onChunk,
CancellationToken cancellation);
}
```

Four members for the simplest engine. `Synthesize` is a coroutine rather than a `Task` on purpose: the session drives it frame by frame, can abandon it on a deadline, and it pauses with the game loop, where a wall-clock `Task` would run on and hand audio back from another thread. The rules every implementation is held to — `onAudio` at most once and never on failure, `StartAsync` never throws, `StopAsync` safe twice — are in the interface's own documentation, and the plugin's session, latency runner and verification window all rely on them.

`AudioData` is raw samples plus a rate; `AudioClipConversion` turns it into an `AudioClip`. Nothing in the interface mentions Unity, so the engine itself stays testable.

A streaming engine

An engine that produces audio faster than it plays but not all at once — a neural synthesiser answering over HTTP with PCM as it is generated — implements `IStreamingTTSEngine` as well. The session then starts the speaker on the **first chunk**, while the rest of the sentence is still being produced; that is where the time to first audio goes down. Everything in the plugin calls `SpeechSynthesis.Run`, which uses the streaming overload when an engine has it and wraps the one-buffer contract into a single final chunk when it does not, so both kinds of engine are complete on their own.

What travels with a request, all optional except the text:

`SpeechRequest`	**Meaning**
`Text`	the unit to say, already sanitised
`LanguageCode`	the language it is in — filled from the language runtime when the caller did not say
`VoiceId`	the voice as the engine names it: the language definition's *TTS voice id*, a speaker name, a preset. Piper takes a voice file name here
`Style`	a delivery hint for engines that accept one; ignored otherwise
`SpeakerId`	the integer selector of the original contract, still honoured

The rules for chunks: in order, one sample rate and channel count per request, the last one marked `IsLast`; on failure log once and finish, do not invent a chunk; honour the `CancellationToken` at the next opportunity, because `Cancel()` on the session is a player who wants silence now; and stay ahead of real time — a chunk that arrives after the play head has passed its position is heard as a gap. A unit longer than the session's `MaxUnitSeconds` (30 s) is cut, and a unit that takes longer than its `UnitSynthesisTimeoutSeconds` (60 s) is abandoned, subtitle-only.

An engine that serves every language from one running process returns `false` from `RestartOnLanguageChange`. The manager then calls `SetLanguage` on a language change and nothing else: no stop, no start, no loading overlay. Piper keeps the default, because a Piper process is one voice.

Give the engine an id. `INamedTtsEngine` is one property — `EngineId => "voicevox"` — and it is what the language definition's *TTS engine id* refers to: a language whose definition names an engine reaches it **without a row in the TTS manager's binding table**. Piper's id is `piper`, which is why a new Piper-voiced language needs its definition and no binding row. A row, when present, still wins.

Two more things the plugin does for you here. A crash watchdog that must not resurrect a process while another engine is speaking implements `IActiveAwareEngine` and is handed the question "am I active?" by the manager. And `StreamingSpeechSession` takes a `SpeechRequest` as its voice template — `new SpeechRequest("", speakerId, languageCode, voiceId, style)` — so a character's language, voice and style ride with every unit.

The worked example: VOICEVOX

VOICEVOX is the awkward shape on purpose — an **external application**, started outside Unity, answering over HTTP, with its own idea of when it is ready. If the seam survives that, it survives a DLL or a cloud call.

What the implementation does, before the code itself:

1. **`StartAsync`** does not start VOICEVOX; the player does. It polls `/version` until the server answers, then flips `IsReady`. A health-retry loop rather than a single probe, because the server takes several seconds to boot and a one-shot check would declare it absent forever.
2. **`Synthesize`** is two calls: `POST /audio_query` for the prosody object, then `POST /synthesis` for the WAV. It yields between them, so the frame is never blocked.
3. **`speakerId`** maps straight onto VOICEVOX's speaker numbers — which is why the parameter exists on the interface at all. Piper uses it only to choose between its default and its secondary English voice (one process per voice file); VOICEVOX needs it on every request.
4. **`SetLanguage`** is where a multi-language engine switches; VOICEVOX is Japanese only, so this example does not implement `ILanguageAwareEngine`.
5. **`EngineId`** names it `voicevox` (`INamedTtsEngine`), so the Japanese definition's *TTS engine id* reaches it without a row in the TTS manager's binding table.

The one thing worth copying verbatim is the readiness handling. An engine that reports ready before it is produces silence with no error, which is the hardest speech failure to diagnose.

The whole engine, copy this file

```
using System;
using System.Collections;
using System.Threading.Tasks;
using OfflineAINPC.Speech;
using UnityEngine;
using UnityEngine.Networking;

namespace YourGame.Speech
{
    /// <summary>
    /// VOICEVOX as an ITTSEngine: an external HTTP server, no plugin changes.
    /// Drop this component in the scene, add it to the TTS Engine Manager's Engines, and
set
    /// the Japanese language definition's TTS engine id to "voicevox" — the EngineId
below.
    /// </summary>
    public class VoicevoxEngine : MonoBehaviour, ITTSEngine, INamedTtsEngine
    {
        [SerializeField] private string _baseUrl = "http://localhost:50021";
        [SerializeField] private float _healthCheckTimeout = 2f;
        [SerializeField] private float _healthRetryInterval = 3f;  // re-check cadence
while not yet ready
```

```
        public string EngineId => "voicevox";
        public bool IsReady { get; private set; }

        private Coroutine _healthRetry;

        public async Task StartAsync()
        {
            using var req = UnityWebRequest.Get($"{_baseUrl}/version");
            req.timeout = (int)_healthCheckTimeout;
            var op = req.SendWebRequest();
            while (!op.isDone) await Task.Yield();
            IsReady = req.result == UnityWebRequest.Result.Success;
            if (IsReady)
            {
                string version = req.downloadHandler.text.Replace("\"", "");
                Debug.Log($"[VoicevoxEngine] Ready (version: {version})");
            }
            else
            {
                Debug.LogWarning($"[VoicevoxEngine] VOICEVOX server not reachable at
{_baseUrl}: {req.error} — retrying");
                StartHealthRetry();   // the server may still be opening its port
            }
        }

        public Task StopAsync()
        {
            // VOICEVOX is the player's own process: stopping the engine does not close it.
            IsReady = false;
            if (_healthRetry != null) { StopCoroutine(_healthRetry); _healthRetry = null; }
            return Task.CompletedTask;
        }

        private void StartHealthRetry()
        {
            if (_healthRetry != null) return;
            _healthRetry = StartCoroutine(HealthRetryLoop());
        }

        // VOICEVOX may still be opening port 50021 when the game starts. Rather than mute
the
        // language for the whole session, poll /version until it answers, or until
StopAsync
        // stops the loop on a switch away.
        private IEnumerator HealthRetryLoop()
        {
            while (!IsReady)
            {
                yield return new WaitForSeconds(_healthRetryInterval);
                using var req = UnityWebRequest.Get($"{_baseUrl}/version");
```

```
                req.timeout = (int)_healthCheckTimeout;
                yield return req.SendWebRequest();
                if (req.result == UnityWebRequest.Result.Success)
                {
                    IsReady = true;
                    Debug.Log($"[VoicevoxEngine] Ready after retry (version:
{req.downloadHandler.text.Replace("\"", "")})");
                }
            }
            _healthRetry = null;
        }

        public IEnumerator Synthesize(string text, int speakerId, Action<AudioData>
onAudio)
        {
            string queryUrl = $"{_baseUrl}/audio_query?text=
{UnityWebRequest.EscapeURL(text)}&speaker={speakerId}";
            using var query = UnityWebRequest.PostWwwForm(queryUrl, "");
            yield return query.SendWebRequest();

            if (query.result != UnityWebRequest.Result.Success)
            {
                Debug.LogError($"[VoicevoxEngine] query failed: {query.error}");
                yield break;
            }

            string synthUrl = $"{_baseUrl}/synthesis?speaker={speakerId}";
            byte[] body = System.Text.Encoding.UTF8.GetBytes(query.downloadHandler.text);
            using var synth = new UnityWebRequest(synthUrl, "POST");
            synth.uploadHandler = new UploadHandlerRaw(body);
            synth.downloadHandler = new DownloadHandlerAudioClip(synthUrl, AudioType.WAV);
            synth.SetRequestHeader("Content-Type", "application/json");
            yield return synth.SendWebRequest();

            if (synth.result != UnityWebRequest.Result.Success)
            {
                Debug.LogError($"[VoicevoxEngine] synth failed: {synth.error}");
                yield break;
            }

            // The contract: onAudio at most once, and never on failure.
            AudioClip clip = DownloadHandlerAudioClip.GetContent(synth);
            if (clip == null)
            {
                Debug.LogError("[VoicevoxEngine] the WAV could not be decoded");
                yield break;
            }
            var audio = AudioClipConversion.FromClip(clip);
            UnityEngine.Object.Destroy(clip);
            onAudio?.Invoke(audio);
```

```
        }
    }
}
```

`ITTSEngine`, `INamedTtsEngine`, `AudioClipConversion` and `AudioData` come from `OfflineAINPC.Speech`; nothing else here is plugin-specific. Point the Japanese definition at it and the rest of the pipeline — the speech queue and the subtitles — does not know the difference.

Licensing: apply the test, do not take a list

This page will not tell you which VOICEVOX voices are safe for your game, and you should distrust any page that does. Terms are per character, live on the rights holders' own pages, change, and several restrict the *kind of content* rather than only commercial use. The plugin cannot know what game you are making.

What travels between projects is the test, and by now you have seen two shapes of it:

Engine	Where the answer lives	What caught people out
Piper	the **Training line** of the model card	`libritts` and `libritts_r` are one letter apart, same corpus, opposite conclusion
VOICEVOX	the **character's own terms page**	attribution is mandatory and names the character; some characters require an application for commercial or corporate use; some restrict content

Both are the same question asked twice: *what is this made of, and who could object?* See Licensing for the long form.

- Defaults and your existing project
- Qwen: template, response budget and memory
- Piper: match the protocol and the voice
- Whisper: audio, language and completion

Using the current local models

Choose a catalog text model in the first-dialogue setup. Download missing files in Model Manager. Start with a typed reply, then add Qwen speech or a Whisper microphone.

Route	Current boundary
Qwen3-TTS	Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no separately started HTTP server, Python environment, speech executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.
Qwen text GGUF	LLMUnity 3.0.3 declares LlamaLib v2.0.5. Its published source pins llama.cpp b8209. Current Qwen3 text models use this integration; an unrelated llama-cli command is a different launch profile.
Piper (archived rhasspy release)	Unity's `PiperEngine` drives the archived rhasspy Piper — release `2023.11.14-2` (Piper 1.2.0), `piper_windows_amd64.zip` on Windows — through its JSON-lines input. Preserve an existing working executable, voice/config pair and binding.
OHF Piper 1.8.0	**Does not work with `PiperEngine`.** The newer OHF-Voice Piper reads plain text lines and has no JSON-lines input, so do not install it in place of the release above.

Route	Current boundary
Whisper (whisper.unity)	`com.whisper.unity` resolves to Git `529a628a915a97799e89e061af9cb7c71407124d`, package 1.4.0, and the plugin is built against it. Do not swap its native library for a newer whisper.cpp build on its own; update whisper.unity as a whole.

The runtime mapping is supported by LlamaLib v2.0.5 source and the whisper.unity source. Declared source versions do not establish the build provenance of every local binary.

Defaults and your existing project

New speech presets use the same 0.6B CustomVoice Q4 talker and Q8 codec at every LLM tier. Leave **External server URL** empty to use the bundled Qwen CPU DLL. Larger talkers and external HTTP/GPU setups are advanced alternatives to check on your target hardware. All built-in Qwen TTS presets use `serena`, so switching the dialogue language keeps the same speaker.

The default non-thinking reply profile allows up to 128 generated tokens and carries a one-sentence limit. The limit does not require a 128-token reply. The token ceiling is written to every agent of the LLM when the model starts. The sentence limit is applied by the code that delivers a reply: a **Hello NPC Character** applies it when its **Reply Limits** is **Reply Profile** or **Current Language**, and as shipped speaks every sentence within the token ceiling. Custom presets remain yours. Sampling values typed on an `LLMAgent` take effect in Play only with **Override agent sampling** turned off on the LLM host; it is on by default.

Whisper supplies the optional microphone path in 1.1. Use a multilingual Whisper model when dialogue switches between languages. VoiceDesign is not part of this version.

Windows x64 CPU is the bundled Qwen target. The required CPU instructions are listed in the speech setup. macOS/Linux native libraries are not included. Mobile, web and consoles are not supported by this package. Linux and macOS are not verified.

Qwen: template, response budget and memory

Keep role messages in the integration's chat path and use the selected model's chat template. Qwen3's official template has a thinking switch. LLMUnity exposes `LLM.reasoning`; the current host disables it when applying its sampling profile. The pinned LlamaLib service forwards that setting to `chat_params.enable_thinking`. Check the actual prompt and output when changing the model or backend. A textual `/no_think` instruction and discarding visible reasoning after generation are different operations. The official non-thinking reference is temperature 0.7, top-p 0.8, top-k 20 and min-p 0. Qwen3 model guidance, pinned service implementation.

`LLMClient.numPredict` is forwarded as `n_predict`: it limits newly generated tokens. The plugin's base `QwenNonThinking` profile allows 128 tokens and carries a one-sentence limit. Assign `Tiers.Sampling` before the model starts to replace it, or turn **Override agent sampling** off on the LLM host to keep the values typed on each

agent; a tier does not change it. A token limit can truncate speech or an action block. `MaxSentences` filters the delivered text where the delivering code applies it — Hello NPC's **Reply Limits**, or `LimitSentences` in your own code; it does not reduce the native token budget. Keep enough room for the final action structure and any permitted repair.

`LLM.contextSize` is the context budget, configured before model startup. Account for the template, system prompt, replayed memory, retained chat history, per-turn perception and generated output together. On the tier route — Hello NPC's — the host sets it from the chosen model's entry: 4096 tokens for the built-in Light and Standard models, 16384 for Premium, and 8192 for a Model Manifest entry that names none. An `LLM` component started without that route keeps its own field, which defaults to 8192. Qwen3's published long-context and long-output examples are model/framework guidance, not the NPC sample's settings or a RAM/VRAM guarantee. Increasing context also increases the working set. Enable extended-context settings only in a separately verified runtime profile. Qwen context guidance.

Prompt caching can reuse an unchanged prefix while the corresponding context remains available. `cachePrompt` is configurable. Keep stable persona text stable and put changing perception in the user turn. Changed system text, a new context or history management can require additional prompt processing; measure latency with your model and backend. A documented warmup is an actual model operation; record its settings and do not add its throwaway output to game memory.

`NPCMemory` is local transcript storage, not model training or unlimited recall. Install an `IKeyValueStore` before use; without one it records and replays nothing. The window holds 40 entries per character. `lastN` selects up to `2 * lastN` trailing lines, with an available summary prepended. `LLMAgent` history and its optional `save` file are separate. Choose how replay and history combine so the same dialogue is not counted twice. Changed replay text consumes context and can change the reusable prefix. See Dialogue.

The action grammar limits structure and registered values. Parser, dispatch and handler checks still decide whether an action is valid in the current world. Act on the accepted completed reply once; streaming prefixes and truncated or cancelled blocks are not completed actions. See Actions.

Piper: match the protocol and the voice

`PiperEngine` starts one process per voice with `--json-input`, writes UTF-8 JSON lines without a BOM, each with the `text` and an output WAV path, and waits for that file to appear. Its speaker selector chooses a voice file; it is not a promise that a nonzero native `speaker_id` works in that executable. Keep the `.onnx` and matching `.onnx.json` together, and use the voice's output sample rate. A language catalogue entry alone does not add a trained voice. Archived protocol.

OHF Piper 1.8.0 is a different program: it reads ordinary text lines and writes the WAV named with `-f`. Given the old `--json-input` flag, it can speak the unknown arguments as text. So `PiperEngine` cannot drive it; keep the archived release. OHF CLI source, CLI guide.

Runtime code, native dependencies, eSpeak/data and individual voices have separate provenance and licence records. Archived Piper source is MIT; OHF 1.8.0 is GPL-3.0-or-later. Several Piper voices were fine-tuned from the lessac voice, whose dataset terms restrict commercial use — their model cards say so; VOICE_LICENCES lists them. Process separation and a public-domain dataset statement do not establish clearance for the whole distribution. OHF licence.

Whisper: audio, language and completion

Pass decoded PCM floats with the actual sample rate and channel count, rather than WAV bytes or unscaled PCM16 integers. whisper.unity averages interleaved channels and resamples to 16000 Hz mono using linear interpolation; it does not normalize amplitude or validate every malformed buffer. Supply finite normalized samples, normally in [-1, 1]; PCM16 conversion divides by 32768. Specify valid positive rates/channels and complete interleaved frames. Official audio loader.

`Language` identifies the speech language; `auto`/empty requests detection. With `Translate=false`, use transcription in that language. `Translate=true` requests English translation using a suitable multilingual model. Setting another language is not arbitrary target-language translation. `.en` models are English-only, and changing a UI language does not expand the selected model's language coverage. Official task guidance.

Segment times refer to positions in the input audio. whisper.unity converts native 10-ms units to `TimeSpan`. Token times require both token output and timestamp settings and remain experimental estimates. Progress callbacks report processing progress, not audio playback time or transcript accuracy.

whisper.unity's streaming repeatedly transcribes overlapping windows and updates provisional text; it is not token-by-token recognition. `StopStream()` waits for current work, processes the remaining audio and raises `OnStreamFinished`. Its public API does not expose cancellation or deterministic disposal. Cancelling a caller, hiding a subtitle or hitting a timeout does not prove native work stopped. Let the owned work settle before releasing its context or changing the runtime.

Do not drop a whisper.cpp 1.9.x native library into whisper.unity 1.4.0. Its structures changed, so the old C# bindings would read them wrongly even though everything compiles and the exported names match. Update whisper.unity as a whole, then check the loaded library, the model file and a real recognition again. Recognition latency and quality depend on that combination and the input; measure it with your model and recordings.

Push To Talk with Whisper Speech Recognition (see Getting started) records one press, recognizes it whole with `GetTextAsync` and drops the result of a press that a newer one replaced; it does not stream.

- Which model accepts which input?
- In Unity

Qwen speech: language, voice and style

In 1.1, choose a ready CustomVoice speaker. Creating your own voice from a description (VoiceDesign) is not part of this version.

For the current Unity Qwen3-TTS component, start with a **CustomVoice** talker and leave **Style** empty. For the overall model workflow, see Current model usage. These settings have separate jobs:

Setting	What it controls	Example
Text	Words to say	`Hello. I am ready.`
Language	Language of those words	Unity code `en`; runtime label `english`
Voice	A named preset timbre	`ryan`
Style	Delivery instruction, where the model supports it	`Speak calmly, at a measured pace.`

Describe delivery in **Style**; set the language separately. A style such as “Speak Russian” does not replace the language setting or translate the text.

Which model accepts which input?

Qwen3-TTS variant	Voice input	Instructions
0.6B CustomVoice	Named preset speaker	Instruction control is unsupported; leave Style empty
1.7B CustomVoice	Named preset speaker	Optional delivery instruction
1.7B VoiceDesign	Description of a voice	A separate VoiceDesign operation
0.6B / 1.7B Base	Reference audio or a prepared clone prompt	A separate cloning operation

The bundled native transport explicitly accepts CustomVoice; Base and VoiceDesign are not configured. The optional HTTP request shape also carries a named voice, so it does not provide a VoiceDesign or reference-audio authoring flow. See the official model overview, official Python API.

In Unity

```
using OfflineAINPC.Speech;

// CustomVoice, with no style dependency:
var line = new SpeechRequest("Hello. I am ready.",
    languageCode: "en", voiceId: "ryan");

// For a separately verified 1.7B CustomVoice setup:
var styledLine = new SpeechRequest("Hello. I am ready.",
    languageCode: "en", voiceId: "ryan",
    style: "Speak calmly, at a measured pace.");
```

An explicit `SpeechRequest.LanguageCode` overrides the engine language. Native codes `en`, `ru`, `es`, `zh`, `ja`, `ko`, `de`, `fr`, `pt`, `it` accept region/script suffixes and `_`; blank or unknown codes select `auto`. This is language selection, not translation. The native DLL passes text, language, speaker and optional style directly to the pinned runtime. HTTP field omission/default behavior belongs only to an explicitly configured external server. Leave both engine Style and request Style empty for 0.6B CustomVoice. A 1.7B instruction setup requires its own model and runtime check.

Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no manually started HTTP server, executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.

- What it is for, and what it is not
- Turn it on
- Connect a client
- The tools
  - Overview, a selected NPC and the first conversation
  - Target selection and state changes
  - Preview a change, apply it once and read back
  - Live references to scene components
  - Readiness: known, unknown and absent
  - Bounded inspection and continuation
  - Characters, and why most of this wants Play mode
  - A session, for illustration
  - Checking an earlier reply against current conditions
- Adding your own tools
- Safety
- Limits

Agent tools (MCP)

The package includes an MCP server that runs inside the Unity editor. With it switched on, an AI coding agent — Claude Code, Cursor, Codex, or any other client of the Model Context Protocol — can see the characters in your open scenes and test them: read everything a character's model is told, run a real turn and read the reply, look at state, memory and perception, and mark objects so characters notice them.

It is an editor tool. It lives in the editor assembly, is off until you switch it on, and nothing of it reaches a build.

What it is for, and what it is not

For iterating on a character with an agent in the loop. "Why does the guard never mention the door?" is a question with half a dozen possible answers — too far, out of sight, outranked, never described, not in the prompt at all. An agent with these tools can ask the character directly (`say`), read what the model was told (`inspect_prompt`), read why the door is or is not in context (`perception`), change the thing that is wrong (`mark_point_of_interest`, `set_state`) and ask again — without you clicking through four windows between attempts.

For auditing a scene. `setup_check`, `models`, `list_points_of_interest` and `list_actions` answer "what is missing" and "what can these characters actually name and do" in one pass.

Not a general remote control for the editor. Creating objects, editing scripts, building and importing are outside it. The agent's own file tools, or a general-purpose Unity MCP server, do that; this server does the part only this plugin knows.

Not a way for the agent to hear. `say` with `speak: true` plays the voice in the editor for whoever sits at the computer and reports how long synthesis took and how long the audio is. A wrong voice is still a person's call.

Turn it on

`Tools ▸ Offline AI NPC ▸ Agent tools (MCP)`, then tick **Run the MCP server while this project is open**.

Use **Editor UI language** in the window to choose English, Russian, Spanish or Simplified Chinese. It changes the window's labels and redisplays configuration notices; it keeps the port draft and server settings and does not configure the NPC's language. Tool identifiers, client commands and client-facing descriptions retain their original language, as indicated above the tool list.

The window shows the address — `http://127.0.0.1:47371/mcp` unless you change the port — and what the server has answered since the scripts last loaded. The setting is stored per project in the project's user settings, which are not committed, so turning it on for yourself does not turn it on for your team.

The server stops before every script reload and starts again after it. The menu also has **— start the server** and **— stop the server** items, for a script or a keyboard shortcut.

Connect a client

Claude Code, once, in a terminal:

```
claude mcp add --transport http offline-ai-npc http://127.0.0.1:47371/mcp
```

Or press **Add to this project's .mcp.json** in the window. That writes the server into `.mcp.json` at the project root and leaves every other server in the file as it was; Claude Code asks once before it connects to a server named there. A `.mcp.json` that is not valid JSON is not overwritten — the window says what is wrong with it instead.

Any other client that speaks the Streamable HTTP transport: give it the address above. A client that can only start servers as local processes (stdio) needs a stdio-to-HTTP bridge in front of the address.

The server speaks both eras of the protocol on the same address: revision **2026-07-28**, where every request carries its own version and capabilities, and **2025-11-25, 2025-06-18 and 2025-03-26**, which open with an `initialize` handshake. It offers tools only — no resources, prompts or subscriptions.

The tools

Tool	What it does	Changes anything
`project_overview`	**Start here.** A passive snapshot of loaded scenes, exact component references, compiled capabilities and next calls; optional scene/name/active filters and section selection	no

Tool	What it does	Changes anything
`inspect_character`	Passive configuration and actual bindings for one selected NPC; optional explicit runtime readiness and voice probes	possible getter effects with `probe_runtime: true`
`setup_plan`	Steps for the first typed dialogue, with separate optional voice and microphone branches; file inventory checks only when requested	no
`list_characters`	Characters that can be talked to, with their ids, readiness, language and voice; NPC State components; Perception Queries; counts of points of interest and verbs; whether memory and state stores are installed; the project's languages; whether the editor is playing	no
`setup_check`	The *Check my setup* report: every check with its severity (`Ok`, `Degraded`, `Blocking`, `Unknown`), the capability it gates (`text`, `voice`, `input`), detail and fix; `readiness` per capability (`Unknown` when nothing measured is wrong but a row was not measured); `can_answer_in_text` — `true`, `false`, or `null` while a text row is unmeasured; `can_run_at_all`	no
`inspect_prompt`	Everything the model is told for one player line, section by section, each marked cached or rebuilt per turn, with token counts and why nearby objects are left out	no

Tool	What it does	Changes anything
`say`	One character turn: reply, raw output, action evidence and timings; optionally voice	history if `remember`; character-owned pipelines may dispatch game actions
`preview_poi_change`	Preview an edit to one existing point of interest, with a bounded one-use plan	no
`preview_state_change`	Preview one initialized state axis using the game's delta/clamping rules, without loading a store or changing state	no
`apply_change`	Consume a preview plan once and report the write's result separately from read-back	yes
`verify_change`	Read the same target after a completed plan; never repeat its write	no
`get_state`	Every axis with value, baseline, range and band, and the state block as the model reads it	no
`set_state`	Set an axis, or move it by a delta, and report the new band	yes
`get_memory`	The stored conversation and the memory block as the prompt receives it	no
`clear_memory`	Forget a character's conversations	**deletes**
`perception`	Recompute what a character notices: the context block, the ranked objects, and one line per nearby object on why it is in or out	no

Tool	What it does	Changes anything
`list_points_of_interest`	Every point of interest: id, description, priority, accepted verbs, position, duplicates	no
`mark_point_of_interest`	Add or edit a point of interest on an object — id, description in a language, priority, accepted verbs. Edit mode, undoable. `save_scene: true` saves only a scene that has a file and had no unsaved changes before the call; otherwise `scene_saved` is `false` and `scene_not_saved_reason` says why	yes
`list_actions`	The registered verbs with their parameters and descriptions; optionally the grammar	no
`check_reply`	A model reply through the plugin's reply handling without a model: the action block, the spoken units, the parse outcome and the repair prompt a failure would get	no
`models`	Model files and their state, this computer's hardware and the tier	no
`play_mode`	Report, enter or leave Play mode	yes
`recent_logs`	The console's recent lines, filtered by level, text or the plugin's own	no

Each tool's description, which the agent reads, says the same in more detail, including what each argument means. Tools that change things are marked in their annotations, which clients use to decide what to confirm.

Numeric arguments must be finite: `NaN`, infinity and overflowing numeric strings are reported as argument errors. `set_state` also rejects values that overflow the state's single-precision representation. Valid values still follow the axis's normal clamping. A custom tool's timeout must be finite and fit the host timer; an invalid timeout is rejected before the call is queued, so the editor cannot execute it after returning an error.

Overview, a selected NPC and the first conversation

Use `project_overview` before a detailed capture. Its default does not build a prompt, read model or voice readiness, scan model files, initialize a speech manager or prepare a character. Compile capability and scene wiring are evidence about configuration, not proof that native generation works. Custom passive inspection providers must follow the same contract.

```
{"sections":["characters","capabilities","next_calls"],"include_inactive":true}
```

`scene_path` selects a captured scene path; `name_contains` matches a display name/id, and `include_inactive` controls inactive components. Selection counts distinguish captured items, selected items, filter exclusions, projection omissions and unknown matches. These counts are separate from the page's row counts. Identity, status and diagnostic evidence remain available when a section is omitted; projection is explicit, not silent truncation. Unknown section names and malformed options are refused before provider work.

Select an exact character reference from that overview, then request its card:

```
{"character_ref":"<ref from characters>","sections":
["configuration","bindings","next_calls"]}
```

`inspect_character` uses an optional passive inspection source. The LLMUnity adapter reports allowlisted current component configuration with its actual component/member source. This is neither an unchanged asset preset nor proof of a loaded native prompt. State and perception bindings come from actual links; matching names or NPC ids do not establish a link. Memory ids remain store keys. A custom character without this optional source remains usable through the existing runtime API; its card reports configuration as not observable instead of calling its active prompt inspection as a fallback.

By default `probe_runtime` is false. Set it to true deliberately to read the current model and speech readiness. Provider getters can initialize a speech manager or restore persistence flags; requested/performed probes and possible effects are disclosed. For that reason this tool's static annotations are conservatively not read-only or idempotent. A failed getter produces unknown evidence, not a guessed missing model or voice. Neither mode prepares or generates a turn.

```
{"check_files":false,"include_optional":true}
```

`setup_plan` describes Welcome → language-model dependency → one model → Hello NPC → a typed UI reply. It applies no actions. With `check_files: false`, file availability is not checked; true explicitly reads the Model Manager inventory and sample-file evidence. Inventory labels, usable-file reports and incomplete/unknown evidence remain distinct from active selection or native readiness. Voice and microphone steps are optional and cannot turn their failures into a mandatory prerequisite for typed input. Keep a working Piper setup; the plan does not switch engines or choose a model for you.

Hello NPC now implements `ILiveVerifiable` directly. Its screen and MCP use the same assigned Agent, take turns on the character, and run the same actions and presentation. `say` with `speak:false` does not create a speech session or read voice readiness. Cancelling an MCP operation cannot cancel a later reply from the screen; the character stays

busy until the model's running task has actually finished. Verification history/reset is in memory; saved history files are preserved. Language sweeps are refused because this sample has no persona/voice switch. Dispatcher calls are observed; durable world mutation is not. Native startup and actual replies still require verification. The plan does not enter Play or execute a turn.

All three captures use the bounded paging contract below. Continue with only `cursor`; the remaining pages preserve the original selected snapshot, including its references, even if the scene or configuration changes. Obtain a fresh capture before acting on changed configuration. Existing sixteen tools and custom/runtime interfaces keep their previous call forms.

Target selection and state changes

Character, state and character-based perceiver selectors reject ambiguous matches before acting. Unique existing ids and names keep their previous priority. Inactive scene objects are included too; a copied NPC with the same id can therefore make a selector ambiguous. Use a listed live reference to select one exact component, or resolve the scene ambiguity before retrying a legacy selector. Keep existing NPC ids and saved-data keys; these tools do not rename or migrate them automatically. A hierarchy path alone remains ambiguous when two loaded scenes contain the same path.

`set_state` reports `outcome` as `applied`, `unchanged` or `refused`, with `clamped` as a separate fact. A same-value request or zero trait sensitivity can legitimately leave the value unchanged. An invalid authored range or nonfinite computed value is a refusal, returned with `isError: true` and a reason; it is not reported as normal clamping. The returned `after` is the observed value after callbacks. A listener can change it again, so this is not a transaction guarantee. Read `get_state` to inspect the resulting state. Without a state store, edit-mode state is temporary until the model is rebuilt. With a store installed, the existing save lifecycle can flush persisted state outside Play too. Neither a successful write nor an installed store proves durable saving. These tools do not call `SaveNow` or the store's save method; your callbacks can have their own effects.

Unknown top-level arguments are rejected before execution for simple closed schemas, including schemas built with `McpSchema.Object()`. `dry_run` is not a supported argument: adding it to a write call returns an error instead of silently performing that write. This guard is not full JSON Schema validation. Custom schemas that permit extra fields, or use advanced schema composition, keep their own validation responsibility.

This refusal returns `code: invalid_arguments`, `recovery.action: correct_arguments_and_retry` and `recovery.schema_method: tools/list` in `structuredContent`. Fetch the current schema with `tools/list`; `tool_queued: false` confirms that no tool was queued. Review the intended action before retrying. Do not remove `dry_run` automatically: that may perform a real write. Broken tool metadata and provider failures keep their own error classification and recovery.

Preview a change, apply it once and read back

Start with an exact reference from `list_characters` (State) or `list_points_of_interest` (POI). The placeholders below must be replaced with that tool's returned live reference; `composure` is an example of an axis you already configured.

1. Call `preview_state_change` for **one** absolute `value` or Game `delta`:

```
json {"state_ref":"<ref from npc_states>","axis":"composure","delta":-18,"reason":"pressed on a contradiction"}
```

Or call `preview_poi_change` outside Play for selected fields of an existing POI:

```
json {"poi_ref":"<ref from points of interest>","description":"A closed door.","priority":2,"save_scene":false}
```

1. Read `normalized_request`, `preview` and `expected`. Only a complete preview returns a usable `plan_id`. State amounts are finite single-precision numbers; `partner` is required for a relationship. POI `language` requires `description` and edits the first case-insensitive exact language entry; without it, the fallback is edited.
2. Call `apply_change` with only the returned token. Call `verify_change` with the same arguments when you need a fresh observation:

```
json {"plan_id":"<plan_id from the complete preview>"}
```

`mutation.outcome` and `mutation.applied_value` describe the write that normally returned. `observation.observed_after` is separate current evidence; callbacks may have changed it. Check `observation.status` before reading `matches_applied`. An unknown observation or an `indeterminate` write does not authorize repeating a delta.

For State, `matches_applied` compares only the numeric value (`match_scope: numeric_value_only`). The observation reports whether a key is stored separately; a numeric match does not confirm materialization, a successful store save or durable persistence.

Result	**Next step**
`state_uninitialized`	Initialize State deliberately through your game's setup. Preview does not read `Model` to initialize it or load a store.
`authored_shape_unavailable`	Repair the selected null serialized string/list before preview. Ordinary empty values are supported. No plan was published.
`plan_stale`	Inspect current values and review a new preview. The old precondition was refused before its write.
`plan_in_progress`	Let the existing attempt finish, then verify. Do not start the same operation with another token.
`plan_unavailable`	Inspect current state first. Expiry or lifecycle reset cannot establish whether an earlier write happened.

Result	Next step
`mutation.outcome: indeterminate`	The write was attempted and the token consumed. Read current evidence; do not assume rollback or safely repeat the delta.

Preview `max_bytes` defaults to **32768**, accepts **8192–65536**, and includes both encoded output copies. Plans expire after **120 seconds** without renewal and are retained in at most **128 weak records**, **128 KiB per plan** and **4 MiB total**, including receipt reservation. Apply/verify results are bounded to **8192 bytes**. These are plan-tool limits; the inspection page limits below are different.

Use `preview_poi_change` or `preview_state_change` before a change you want to review. Each plan addresses one existing exact `poi_ref` or `state_ref`; a plan does not add a component, choose a replacement by name or combine several writes. The earlier `mark_point_of_interest` and `set_state` tools remain immediate operations. Adding `dry_run` to those calls still returns an argument error; use a preview tool instead.

State preview does not build or restore a model, call a store or send state events. A cold component returns `state_uninitialized`: initialize it deliberately through your game's setup, then capture a fresh preview. A delta follows Game change semantics and trait sensitivity; an absolute value is not scaled by sensitivity. Invalid range, retired trait axis and nonfinite arithmetic are refusals before a plan is published. The numeric result is separate from materializing an absent key: writing its baseline can report `unchanged` while still creating persisted intent.

Review the complete before/requested/expected result, then call `apply_change` with its `plan_id`. The tool checks the same component, scene, model, editor mode, relevant values/definitions and persistence context again before writing. A changed precondition returns `plan_stale`; it does not overwrite the newer value. Changes to an unrelated state axis or relationship partner do not alone stale this plan. Observed target publication/reset evidence detects returning to a prior numeric value; unseen authored A→B→A edits between captures are not an edit history.

The token is claimed before callbacks can reenter it. A retained retry returns the historical result without another write or target read; it is not a fresh observation. `applied_value` belongs to the normal-return mutation receipt. `observed_after` is read separately after callbacks, which may change state again or replace the model. An escaping wrapper exception reports `write_attempted: true`, `indeterminate`, an unavailable mutation receipt and unknown accepted/applied values. This never means rollback or permission to retry the delta. The token stays consumed.

Call `verify_change` explicitly for a fresh read of the same target after a completed plan. It reports a match, a difference or unknown evidence without changing the stored historical receipt. These tools are not a transaction or a sandbox for arbitrary game callbacks. A missing target, model replacement, reset or expired token does not authorize repeating an earlier delta; inspect current state before deciding on a new operation.

POI plans edit only selected raw serialized fields outside Play and transitions. They use a fresh serialized stream and Unity's normal Undo path, with `save_scene: false`; they do not save a scene automatically. Raw ids/translations remain separate from derived registered ids and resolved descriptions. State changes are model operations, not serialized inspector edits, and have no Unity Undo. An installed state store may save later through the existing lifecycle; apply/verify do not certify that it did.

Plans have bounded retention, input and encoded-result budgets. An incomplete or oversized diff gets no usable token. A full live cache refuses a new plan rather than evicting another usable one. Expiry is absolute and retry does not renew it. Server, Play and reload lifecycle resets invalidate tokens. Unity Undo, serialized callbacks, scene dirtiness, store durability and player behavior are covered by tests outside Unity only; check them in your own editor and build before relying on them.

Live references to scene components

`list_characters` lists characters, NPC State components and Perception Queries; `list_points_of_interest` lists POI components. Each row adds `ref`, `ref_kind`, `ref_status` and `ref_scope=editor_epoch`. A missing reference can also include `ref_problem_code`. A capture-time `ref_status=live` supplies an opaque `live_` reference for that exact component in this editor epoch. `unavailable` supplies no usable reference; `unknown` means its scope could not be validated. Check the status and obtain a fresh list rather than guessing a reference from an id, name or hierarchy path.

Listed kind	Typed selector argument	Tools
`character`	`character_ref`	`inspect_prompt`, `say`, `inspect_character`
`state`	`state_ref`	`get_state`, `set_state`, `preview_state_change`
`perceiver`	`perceiver_ref`	`perception`, `check_reply`
`poi`	`poi_ref`	`mark_point_of_interest`, `preview_poi_change`

The preview tools require the typed reference. Existing tools retain their declared legacy selectors. Pass **one selector**: a typed reference or that tool's corresponding legacy selector. Providing both is an error even when one is explicit JSON `null`. A present null, empty or malformed reference is rejected; it does not fall back to a name/id/path. Old calls without a reference still use the existing unique-selector rules. For `get_state`:

```
{"state_ref":"<ref from the npc_states collection>"}
```

For `inspect_prompt`:

```
{"character_ref":"<ref from the characters collection>","line":"Hello."}
```

These are temporary scene-component references, not save keys or persistent asset ids. Each use validates the current target lifetime and loaded scene against its captured scope. Renaming or reparenting within the same scene keeps the exact target. A known removal, replacement, move or unload retires the reference; registry eviction, target collection or an editor/server lifecycle reset also requires obtaining a fresh reference. A replacement with the old name/id is never substituted. The bounded weak registry holds at most 16384 records by default and does not keep targets alive.

Its lifetime is separate from the inspection cursor's snapshot TTL; do not persist references across editor epochs/projects. `say` checks its chosen reference again before starting generation after preparation and prompt callbacks. This does not make later runtime callbacks transactional.

`poi_ref` edits the exact existing POI component. Use legacy `object` to add a POI to an unmarked GameObject. `check_reply.character` remains speaker metadata and can accompany `perceiver_ref`; it is not a character selector. `get_memory` and `clear_memory` continue to address the store by `npc_id`, with no live-reference argument. Selecting a component does not change its id or split persisted data whose keys already coincide.

Reference errors return `isError=true` with a structured code and recovery: `invalid_reference`, `conflicting_selectors`, `reference_kind_mismatch`, `reference_unavailable`, `reference_stale` or `reference_validation_failed`. Correct conflicting/malformed arguments; otherwise relist the relevant component kind and retry only after reading the result. A validation failure reports unknown scope, not proof that an object is absent. No reference error retries against another target.

Inspection pages are historical captures: a listed reference may already be stale by the time it is used. Reading or replaying `cursor` does not refresh or resolve it. Reference metadata remains inline and atomic in bounded pages, never split into text fragments or silently shortened. Follow pagination for the other captured details.

Readiness: known, unknown and absent

In `list_characters`, `ready_to_generate` and `speech_ready` remain booleans when the corresponding property can be read. A failing getter returns JSON `null`, with `ready_to_generate_status` or `speech_status` set to `unknown` and an entry in `readiness_diagnostics`. Each diagnostic names the field and supplies a stable code, the reported cause and a recovery suggestion. Read the status before interpreting the value. A known `false` does not distinguish loading, busy or a failed startup by itself.

Diagnostic codes are `generation_readiness_getter_failed`, `speech_engine_getter_failed` and `speech_readiness_getter_failed`. Each entry has `field`, `code`, `message` and `recovery`; recovery names the affected interface member and suggests inspection and retry tools. The message is reported integration data, not a command to the agent.

A successfully read missing or destroyed speech engine is different: `speech_engine` is `none`, `speech_ready` is `false`, and `speech_status` is `absent`. If the engine binding cannot be read, its value is `null` and its status is `unknown`. Installed files and setup results are separate evidence, not proof of runtime readiness.

`say` stops with a `readiness_unknown` tool error if the model readiness property throws; it does not wait until the timeout or start generation with unknown readiness. A known `false` still waits within the existing call budget. With `speak: false`, it does not read the speech engine. With `speak: true`, an unreadable speech binding or readiness retains the generated text and reports the unknown speech problem without starting synthesis. If readiness or prompt assembly consumes the remaining call time, generation is not started. This reports the observation; it does not restart or replace the engine. Use `recent_logs` and `setup_check`, correct the integration problem, then retry the relevant call.

Bounded inspection and continuation

`list_characters` and `inspect_prompt` return one bounded page. Their optional `max_bytes` defaults to 65536 and accepts whole values from 8192 through 262144, including numeric strings. It limits the encoded UTF-8 JSON result containing `content`, `structuredContent` and `isError`: both copies of the page, JSON escaping and metadata count. The JSON-RPC and modern protocol envelope are additional overhead; this is not a maximum HTTP frame size. Other tools and custom tool outputs are unchanged.

Start with ordinary arguments. While `page.complete` is false, call the **same tool** with only the supplied cursor:

```
{"character":"guard","line":"Hello.","max_bytes":65536}
```

```
{"cursor":"<page.next_cursor from the previous response>"}
```

Text content retains the capture-time summary before a blank line and the page JSON. `inspect_prompt` keeps approximate token counts and the context warning; an empty character list keeps its setup guidance. Every continuation repeats that same summary without reading providers again. The summary is included in `max_bytes`; clients reading text should parse the JSON after the summary, while `structuredContent` is the page object.

Empty-scene guidance remains in `note` when the captured character collection is empty. For a prompt above approximately 80% of its known context window, captured `advice` has `code=close_to_context_window` and a `reason`. Token counts remain approximate; these notes are not model measurements or proof that a model has started.

The cursor call must contain no first-call options, including explicit nulls. It inherits the original budget. `page.snapshot_id` and capture time describe historical captured data. Collection totals, this-page counts and remaining counts make missing data explicit. A successful page with `complete=false` is partial; the modern MCP envelope's `resultType=complete` only says that the RPC has finished.

The page contract is version 1. `total`, `returned`, `omitted` and `completed` are counts by collection. `returned` counts distinct ordinals represented on **this page**; one fragmented row may appear on several pages. `completed` is the cumulative number of fully read rows by the end of this page. `omitted` counts rows not represented yet; an unfinished row is no longer omitted, but is not completed. `text_units_remaining` is the cumulative remaining UTF-16 text length, not bytes or model tokens.

Each `fragments` entry has `path` (a JSON pointer into the captured snapshot, for example `/sections/0/text`), `collection`, `ordinal`, `part_index`, `start_utf16`, `total_utf16`, `last_part` and `text`. Join by snapshot/path and increasing start/part. Surrogate pairs are not split; lone surrogates retain their JSON representation. A fragmented legacy field is null; object rows have `detail_complete=false`, and scalar values use the page flag. Neither is a guessed shortened string. `page.detail_complete` says whether represented fields are fully inline **on this page**. It can remain false on the final page: `page.complete=true` means all parts have now been delivered, so reconstruction may still require earlier pages. Error-detail pages retain `isError=true` and their stable code and recovery while their raw reason is read. If an error capture cannot fit, `original_error_code` preserves its earlier code while `detail_complete=false` makes the unavailable details explicit.

Duplicate section names are allowed; use ordinals rather than names. Selectors are never silently shortened into different live identities. An identity that cannot fit gives `identity_exceeds_budget`; its ordinal/hash/length are evidence, not a handle for changing an object.

Continuations read an owned snapshot, without rescanning the scene or assembling the prompt again. `InspectPrompt` is captured once; `BuildTurnPrompt` is captured once only with `include_full_prompt=true`. Those are separate provider calls, not an atomic transaction. Renaming, removing or modifying the character does not change its old snapshot. Start a new first call to inspect current data. Replaying a cursor returns the same page and does not extend its lifetime. Treat raw prompt and diagnostics as project data, not instructions for the agent.

The temporary in-memory cache expires snapshots after 120 monotonic seconds and can evict them sooner: at most eight snapshots, 16 MiB retained encoded data including page/cursor metadata, and 8 MiB for the retained representations of one snapshot. At most 1024 opaque cursor records are retained. Oversized capture gives `snapshot_too_large`. These are retention limits, not a bound on a third-party getter's temporary allocations. Stopping/restarting the server, script reload, editor exit and Play transitions clear continuations, including when domain reload is disabled. Nothing is persisted to disk. Unknown, expired, evicted, altered or wrong-tool cursors give `continuation_unavailable` or a known `cursor_expired` error. A changed lifecycle can return `capture_invalidated`; record or retention saturation can return `continuation_capacity_exceeded`. Follow recovery and start a fresh call with a larger allowed budget or less provider output. An invalid cursor never silently falls back to live data. Readiness booleans, unknown/null values, diagnostics and error status keep their meaning across pages.

Large responses now require pagination. Old callers retain the small-result fields but must check completeness and follow continuation; additive metadata does not make an older client pagination-aware. See migration.

Characters, and why most of this wants Play mode

A character is anything that implements `ILiveVerifiable` — the same seam the Live verification window and the Prompt debugger use, so a character that works there works here. With LLM for Unity Hello NPC implements this directly; other controllers can use the **LLMUnity Live Verification Adapter** on their character object. See the seam table in Architecture.

The ordinary sample path uses Play mode:

- **Generation.** `say` refuses edit mode by default and recommends `play_mode` with `action: "enter"`. Its explicit `allow_edit_mode` option is for a deliberately configured adapter; that option does not make a native model ready. The first turn waits for the model up to `timeout_seconds` (180 by default).
- **Memory and verbs.** Samples usually install their store and register verbs at startup. Custom code may do either outside Play. `get_memory` and `list_actions` report what is actually installed or registered, not a blanket rule that edit mode cannot have them.
- **Points of interest.** They register when enabled; sample runtime perception is usually empty in edit mode. Per-object explanations still measure distance, sight and score.

Entering or leaving Play reloads scripts when domain reload is on. The server is away for those few seconds — calls fail with a connection error — and a call that was running is answered with "the editor is reloading scripts". `play_mode` with `action: "status"` tells the agent when it is back and playing.

A session, for illustration

This is the shape of a session, not a recording of one:

1. `list_characters` → one character, `guard`, not ready to generate; the editor is not playing.
2. `play_mode` `{"action": "enter"}`, then `play_mode` `{"action": "status"}` until `playing` is true.
3. `say` `{"character": "guard", "line": "Is the door locked?"}` → a reply about the weather, and the action outcome `None`.
4. `perception` `{"perceiver": "guard"}` → `door: 5.1 m, score 0.31 — visible but outranked, top 6 only`.
5. `play_mode` `{"action": "exit"}`; `mark_point_of_interest` `{"object": "Door", "priority": 3}`; `play_mode` `{"action": "enter"}` again.
6. `say` again → the guard mentions the door.

Checking an earlier reply against current conditions

`check_reply` uses the schema available when you analyse the reply, including current verb filters and target permissions. It does not recover the prompt or grammar used for the original generated turn. A perceiver supplies its current cached target set; this call does not refresh the world or request action dispatch.

Read the additive `analysis_context` object before interpreting a result:

- `basis: current_at_analysis` and `original_generation_context: not_captured` state the scope of the evidence. A reply from an earlier quest stage may now be invalid.
- `schema_source` and `targets_source` identify the current schema and target set.
- `schema_stability: changed` means that the target-registry counter changed during this analysis. Equal counters report `unknown`: filters, permissions and metadata were not snapshotted. This is not a frozen schema or proof of earlier permission.
- `world_refresh_requested` and `action_dispatch_requested` are `false`. These describe the tool's requests; custom game callbacks may have their own side effects.

The parser outcome is kept as produced, without a second parse. The game handler must still validate the target and gameplay conditions when it performs an action. Existing arguments and the four-argument `Analyse` call remain supported; JSON consumers should allow the new object alongside existing result fields.

Adding your own tools

A game has questions only it can answer — which quest step the player is on, where the player stands, what a trigger would do. Register a tool for each from an editor script, and it appears in every connected agent's list after the plugin's own:

```
using OfflineAINPC.Core.Serialization;
using OfflineAINPC.Editor.Mcp;
using UnityEditor;
using UnityEngine;

[InitializeOnLoad]
```

```
static class MyGameAgentTools
{
    static MyGameAgentTools() => McpTools.Register(new TeleportPlayerTool());
}

sealed class TeleportPlayerTool : McpInstantTool
{
    public override string Name => "mygame_teleport_player";

    public override string Description =>
        "Move the player to a spawn point by the name of its object, in Play mode. Returns
the new position.";

    public override JsonValue InputSchema => McpSchema.Object()
        .Text("spawn", "Name of the spawn point object.", required: true)
        .Build();

    public override bool ReadOnly => false;

    protected override McpToolResult Execute(McpArguments arguments)
    {
        if (!Application.isPlaying) return McpToolResult.Error("Enter Play mode first
(play_mode).");

        var spawn = GameObject.Find(arguments.RequireString("spawn"));
        if (spawn == null) return McpToolResult.Error("No spawn point by that name.");

        var player = GameObject.FindWithTag("Player");
        if (player == null) return McpToolResult.Error("No object is tagged Player.");

        player.transform.position = spawn.transform.position;
        return McpToolResult.Ok("The player is at " + spawn.transform.position + ".");
    }
}
```

The rules that matter:

- **`Execute` runs on the editor's main thread**, so any Unity API is fine there. The name, the description, the schema and the timeout are read on a network thread while the tool list is built — keep Unity out of them.
- **Say what went wrong, in words.** `McpToolResult.Error` and `McpArgumentException` both reach the agent as a message it can act on; `arguments.RequireString` throws the second for you.
- **Long work** subclasses `McpTool` instead: `Run` is a coroutine, so it can `yield return null` between steps, should stop when `context.IsExpired`, and overrides `TimeoutSeconds` when 30 seconds is not enough. Set `context.Result` before it ends.
- **Registration is repeated after every script reload** by `[InitializeOnLoad]`, so there is nothing to undo. Registering the same name again replaces the tool; a built-in name is refused. Prefix your tools with your game's name.
- **Your editor assembly references `OfflineAINPC.Editor`.** The default editor assembly does already.

- **A narrower address for a narrower client.** A tool whose `Toolsets` names something other than `editor` is also served at `/mcp/<toolset>`, and that address lists only its own toolset's tools: a tool outside it cannot be called through it. Give a playtesting agent `http://127.0.0.1:47371/mcp/player` and it can use your player tools without being able to read a prompt or the scene. `McpTools.DescribeToolset` sets what that address tells a client about itself.
- **Pictures.** `McpToolResult.Image` returns an image after its caption, for a tool that shows what a camera sees.

Safety

- The server listens on this computer's loopback address only. Other machines cannot connect.
- Foreign web origins and non-loopback Host headers are refused, including DNS-rebinding host names. HTTP/HTTPS origins on localhost/loopback are allowed; this is a local-client trust boundary, not authentication against other programs or locally served web pages.
- There is no password beyond that. While the server runs, any program on this computer can call the tools — read your scenes, change points of interest and state, clear memory, run the model. That includes programs of other users signed in to the same machine, such as a shared studio workstation or a remote-desktop server. Turn the server off when no agent is using it. It starts switched off.
- **What a tool returns goes wherever your agent sends it.** Personas, prompts, replies, state, memory, console lines and project paths are returned to the client that asked. A coding agent that runs its model in the cloud passes what it reads to its provider, under that provider's terms; an agent with a local model keeps it on this machine. Which one you have depends on the client and its settings. The game's own models still run offline, and nothing of this server is in a build.

Limits

- **Bounded outstanding work.** At most 32 HTTP connections and 64 queued/running tools are admitted. A full tool queue returns an error before executing or queueing the tool; an excess connection is answered `503 Service Unavailable` with `Retry-After: 1` and closed. At most eight tools start per editor update. Retired calls release their slots; timing out while the editor is paused does not create room for an unlimited abandoned queue.
- **A cancelled call stops.** When the client sends `notifications/cancelled`, the call is answered at once as cancelled; the editor drops it if it has not started, or stops it at its next update. A change it already made stays made — read the target again before deciding anything. A dropped connection without that notification does not stop the call: the MCP specification says a disconnection is not a cancellation.
- **Bounded reads.** Headers and total chunk framing/trailers each have a 32 KiB wire budget; the body has an 8 MiB budget. Origin and Host are checked before the body is read, so a refused request costs no body read. A request must finish reading within 15 seconds. Stopping the listener closes its own connections after the drain period.
- **The editor has to be updating.** A modal dialog, a long import or a debugger paused on a breakpoint holds every call. A call the editor has not started within 30 seconds is answered with an error that says so, rather than hanging the agent.
- **In the background a call takes about a quarter of a second.** With the Unity window not in front, the editor updates about four times a second: a call that does almost nothing was measured at 250 ms typically and 650 ms at worst. It keeps answering; it is only slower.

- `say` **talks to the model the game is running, while the game runs it.** If the hardware-tier cascade is still swapping models — it does when one starts slowly — a turn can get no text at all. The error then says that nothing arrived, rather than "still generating", and `recent_logs` shows the cascade's warnings. `play_mode` reports `frame`: two calls a few seconds apart show whether the game behind the editor is advancing. Behind a locked screen it ran at about five frames a second.
- **One server per port.** A second editor with the server switched on needs a different port; the window says when the port is taken.
- `recent_logs` **keeps about the last 400 console lines** of the editor session. Compiler errors are not console log lines and are not in it.
- **Hearing is not verification.** Generic `say` reports synthesis time and audio length; Hello NPC reports software play-call evidence. Whether the voice is the right one is still for a person to hear.

- Concurrency
- Latency
- Speech recognition adds its own wait
- Switching language mid-session
- Desktop only
- Output is probabilistic
- State affects speech only sometimes
- Perception is opt-in, and matters more than it sounds
- Voices come in pairs
- Model folders cannot be reached through a link
- Pause: what each part does at timeScale = 0
- Verification needs Play mode for dialogue
- Not included

Limitations

Every limit here is stated with its mitigation where one exists, and stated bare where none does. The goal is that you are never surprised — not that every sentence carries a disclaimer.

Check the complete conversation on your target machine while the game is rendering. **Tools ▸ Offline AI NPC ▸ Live verification** measures a character's turn to synthesized audio; for the player's full wait, also measure recognition and the start of playback.

Concurrency

Shared-server contention. Characters on one LLMUnity server can compete for generation. The server's concurrency setting and the hardware determine whether requests queue and how quickly they finish. Separate hosts have their own memory cost. Test simultaneous turns with your actual model, voice and game workload.

Bundled Qwen speech. The native route allows only one initialized context per process and synthesizes requests sequentially. Starting a second native engine is refused while the first still owns its context. Before replacing it, await successful completion of the first engine's `StopAsync`. Each `SpeechRequest` can select its own language and voice.

The mitigation. Suspend background conversation the moment the player starts speaking, rather than when their speech has been transcribed — the transcription itself takes seconds you have already lost.

```
// An illustration: _ambientChatter and _dialogue stand for your own components;
// Begin and End are Push To Talk's, driven from your input with Listen To Key off.
void OnTalkPressed()
{
    _ambientChatter.Suspend();            // everything else stops generating NOW
    _pushToTalk.Begin();
}
void OnTalkReleased() => _pushToTalk.End();
```

```
// ...and once the character has answered, or the press was not sent:
// _dialogue.ReplyFinished += () => _ambientChatter.Resume();
// _pushToTalk.Dropped += (_, _) => _ambientChatter.Resume();
```

Suspend on the *input* event rather than waiting for the transcript, so background work does not keep competing throughout recognition. Resume ambient conversation when the player's turn finishes or their input is dropped.

Design consequence. Concurrent turns can queue on a shared server. Prioritise the player's turn and budget background generation; more hardware or a different model setup needs its own measurements.

Latency

Time the first turn separately from later turns, and test longer conversations as well as an empty history. Measure from the player's input to playback, so recognition, model generation and speech synthesis are all included.

Mitigations. - First-turn delay can include loading, backend initialization and prompt evaluation. Optional warm-up during loading may move some work earlier; it must finish within the real resource and cancellation limits and does not guarantee a fast next turn. - Give the character something to do while generating — a glance up, a breath, a posture change. Two seconds of a character visibly thinking reads very differently from two seconds of a frozen face. - Keep the system prompt stable where possible. Cache reuse depends on the backend and context; changing the prefix may require more prompt evaluation. See Dialogue and Current model usage.

Design consequence. Use recorded audio for an immediate alert. Give generated conversation a visible waiting state, and check whether its longest pauses fit your scene.

Speech recognition adds its own wait

For voice input, recognition happens before the character can answer. Choose the Whisper model using both transcription accuracy and turnaround time on your target machine. If recognition and generation use the same GPU, test them together. A shopkeeper waiting for a spoken order has a different budget from an immediate combat command.

`StreamingSpeechSession.TimeToFirstAudioMs` starts at the first unit enqueued for speech. It does not include recognition or the LLM wait before that first unit. Measure from the player's input event through the first audible playback to report the complete path; keep those timestamps separate from this speech-session metric.

Switching language mid-session

Changing language can change the cached prompt prefix; with Piper it also stops and restarts the engine with a different voice model, while Qwen3-TTS switches in place. Rapid switching is coalesced — only the final selection is applied — so a settings dropdown being dragged does not queue up five restarts.

A changed prefix can require additional prompt evaluation. Measure language switching with your chosen backend and model; use a menu screen if avoiding new work during a conversation matters.

Desktop only

Windows x64 CPU is the bundled Qwen target. The bundled DLL works in Unity Editor and Windows players; its required CPU instructions are listed in the speech setup. macOS/Linux native libraries are not included. Mobile, web and consoles are not supported by this package. Linux and macOS are not verified.

Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no manually started HTTP server, executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.

For Qwen speech, keep the included Windows x64 native plugin and its five licence notices in the distribution; obtain the talker/codec through the configured player model locations or selected Model Manager build copies. The DLL is a Unity native plugin, not a manually copied server executable. The interview demo, built as a Windows player from this delivery, loads the DLL and speaks through it; test your own player build as well. For an explicitly configured external server, separately supply and test that server and its dependencies.

Output is probabilistic

A language model produces different text every run, occasionally something off-key, and it will not deliver an authored line verbatim.

Mitigations. - Grammar constrains the *shape* of a reply — a well-formed action payload — reliably, because tokens that would break the shape have no probability left. It does not bound how long the prose runs. See Actions. - The token ceiling and sentence limit are what actually keep replies short. **Small models ignore worded length instructions**: "reply in one short sentence" in a persona is a suggestion, the ceiling is not. Tune the ceiling, not the wording. - For lines that must be exact, author them and use the model for the space between.

What has no mitigation. You cannot guarantee any particular sentence. Plan for it.

State affects speech only sometimes

This one is easy to overstate in either direction, so precisely:

	Reliability
A state value, its band, and its threshold event	**Exact, every time**
A state descriptor changing how the model *speaks*	Depends on the model and prompt

Use state values and band events to drive game behaviour. The model may reflect a state hint in its reply, but that wording is not deterministic.

The rule that follows: every visible consequence of state must have a deterministic cause. If a character's tiredness matters, show it in posture, pacing or an event — and let the speech colouring be the extra. A design whose core loop rides on the probabilistic half will feel unreliable exactly when someone is judging it.

Perception is opt-in, and matters more than it sounds

Perception gives the model an object snapshot from your scene. It can still invent details; inspect the snapshot and validate important game facts in code.

Mitigation: turn it on. See Perception. The cost is a coarse-timer recompute, not per-frame work.

Voices come in pairs

Every Piper voice is two files: `name.onnx` and `name.onnx.json`. A missing config caused silent synthesis in earlier integrations. The current Piper driver checks the paired config before spawning a voice for synthesis, reports the problem and refuses that request. Setup and local import checks also validate the pair. These checks do not prove that the native executable can synthesize on the target machine.

Mitigation: the setup checker (`Tools ▸ Offline AI NPC ▸ Check my setup`) reports an unpaired voice as a problem. Run it after adding any voice.

Piper 1.2.0 can also miss its output file when Unity's temporary folder path contains letters outside plain ASCII, such as a Cyrillic user name; every line then ends in a synthesis timeout. See Troubleshooting.

Model folders cannot be reached through a link

Model files are written, copied and deleted only along ordinary folders, so a junction or symbolic link cannot send them somewhere else. Downloads and imports refuse a models folder reached through a link, an import refuses a file picked through one too, and a build that includes models refuses a project opened through one. A build that includes no model is not affected. Choose ordinary paths, or open the project from its real folder.

Pause: what each part does at `timeScale = 0`

Decided per subsystem, because the right answer differs.

Subsystem	**Clock**	**At `timeScale = 0`**
State axes (decay, needs)	scaled	**Freeze.** The character does not get sleepier while the player is in a menu
Perception	scaled	**Freezes**, consistently with state
Speech queue	**unscaled**	**Keeps running** — see below
Speech-process watchdog	unscaled	**Keeps running.** An external process lives on wall clock
Generation already in flight	off the game loop	**Continues**, deliberately

The speech queue is the non-obvious one. `AudioSource.Play` does not obey `timeScale`: a line already sounding keeps sounding through a pause. Every timer around playback is therefore unscaled, so the bookkeeping stays beside the sound that is actually playing.

Scale those timers instead and the failure is quiet: audio runs while the code waiting for it stands still, the subtitle hangs on screen, the queue never advances and the session never ends. Worth knowing if you write your own queue.

Generation continues during pause by policy. Cancelling discards the in-flight reply and may require work later; it does not imply a fixed cache-miss cost on resume. If your game wants a pause to abandon the reply, call `Cancel()` on the session yourself.

Behavioural verification of this needs Play mode. The decisions above are implemented and reasoned; watching a menu open mid-reply is not something a unit test can do.

Verification needs Play mode for dialogue

Live verification generates a reply before it voices it, and the language model is constructed in `Start()` with its continuations running only inside the player loop. So Hello NPC and most characters are checked only in Play mode. Outside it, **Advanced ▸ Speak one line (Piper)** says a fixed sentence through Piper, with no language model.

No mitigation — it is how Unity's lifecycle works. Documented so the tooling's scope is not mistaken for a bug.

Not included

Named so you do not go looking:

- **RAG / document retrieval.** Not present.
- **Lip sync.** The plugin does not ship a solution. The Interview demo animates mouth frames from the audio it plays, and a rig with a jaw bone can be driven the same way from the playing `AudioSource`.
- **Emotion control in TTS beyond a delivery hint.** Official 1.7B CustomVoice supports an optional *Style* instruction; 0.6B CustomVoice has no official instruction control. This is separate from VoiceDesign and cloning; see Qwen speech settings. There is no per-word prosody control, and Piper voices have a fixed delivery.
- **Streaming speech recognition.** Recognition runs on a finished utterance, not continuously.
- **Turn history in the prompt debugger.** The debugger exists and IS broken down by section — a static-prefix column and a this-turn column, each section collapsible with its own token count. What it does not show is previous turns: you see the prompt for one turn, not the conversation that led to it.

- Setting it up
- The tiers
- Fallback is a design decision, not an error
- Forcing a tier while developing
- What "degrades" actually means
- Several candidates for one tier

Hardware tiers

These tiers are configurable rules for choosing the language model. They are not minimum requirements or speed guarantees for a complete game. Reported RAM/graphics memory and the configured reserve are planning inputs, not a live measurement of free memory. Speech recognition, speech output, rendering and other running applications need their own budget. Test the full scene on each target computer; do not add peak memory figures from different processes or moments to make a minimum specification.

The plugin probes the machine at startup, picks a tier, chooses a model for it, sizes the GPU layer count against the real VRAM budget, and steps down if a load fails. Your game asks none of that — it adds two components and reads one property.

Setting it up

On the object that holds your `LLM` component (LLM for Unity), add:

1. **LLMUnity Host** — the adapter that starts, stops and measures that `LLM`. Leave the `LLM` component itself **disabled** in the scene: its model, context size and GPU layer count lock the moment its service starts, and the cascade has to choose them first.
2. **Tier Bootstrap** — the driver. Optionally assign a `ModelManifest` (which model each tier wants) and a `TierPolicy` (the thresholds); empty fields use the built-in defaults, which are Qwen3 1.7B / 4B / 8B and the numbers below.

That is the whole setup. A project with its own startup sequence, or another inference library, implements `ILlmHost` and drives `TierCascade` itself. Implement the optional `ILlmMeasurementLifetime` as well when your host can retire a pending measurement.

What it decides is printed once at startup:

```
Hardware: NVIDIA GeForce RTX 3060 Laptop GPU (5996 MB VRAM, discrete), 32498 MB RAM, 16
threads, backends=[Cpu,Cuda,Vulkan]
Tier resolved: Standard — 3996 MB VRAM free after a 2000 MB game reserve
Running qwen3-4b-q4km at tier Standard with 36 GPU layers (first run on this machine —
starting at 36 layers)
Startup verdict: tier Standard — 3996 MB VRAM free after a 2000 MB game reserve; model
qwen3-4b-q4km, 36 GPU layers
```

Stopping the language model never blocks the game: the host stops taking work while the model unloads in the background, and nothing restarts or changes settings until that has finished — if it takes too long, this launch uses authored lines. A load cancelled before it finishes retires that LLM component for the launch: LLM for Unity's startup cannot be waited on, and a late failure must not take a replacement model down with it. The memory of such a load is released once it ends in the background. Sampling and startup measurements use only agents connected to this host's LLM.

The startup speed is the number of tokens that arrive after the first non-empty batch of streamed text, divided by the time from that batch to the end of the reply — not a count from a native first-token time. When the reply arrives in one batch, tokens cannot be counted or the interval cannot be seen, no speed is reported rather than an estimate.

A request that timed out keeps the model reserved until its generation and tokenization have actually finished; cancelling it does not prove they have.

The tiers

Thresholds are VRAM **budgets**: the card's memory *after* a reserve for your own rendering, 2000 MB by default and 1500 MB more in VR. The card sizes below are what those budgets come to in practice.

Tier	Machine	Model size	What the player gets
Premium	budget ≥ 10 000 MB — a 12 GB card or larger	7–8B	Larger configured model and context budget
Standard	budget ≥ 3 500 MB — a 6 GB card, with slack for how drivers report it	3–4B	Balanced model size
Light	less GPU than that, **and** ≥ 14 GB RAM **and** ≥ 4 logical threads	1–2B	Smallest configured model; check dialogue quality and speed for your game
Fallback	anything less	none	**No language model.** A working state, not a crash

A 10 GB or 11 GB card is Standard, not Premium, because its budget after the reserve is 8 000–9 000 MB. A 4 GB card is not Standard. A "16 GB" laptop with integrated graphics reports 14–15.8 GB of RAM once the GPU has taken its share, which is why the Light bar is 14 000 and not 16 000. In VR the verdict also drops one tier. All of it is editable in a `TierPolicy` asset assigned to the bootstrap; the defaults are a starting point, not a law.

The layer count is not a guess from the card size alone. The tuner sets aside the model's estimated working set — including the context-dependent KV cache — and offloads what its estimate allows. The actual allocation depends on the model, cache format and backend. A load that still fails steps the count down by a quarter of the model and tries

again; a start that merely times out moves on to the next model, because fewer layers do not make a slow disk faster. What worked is remembered per machine, model, context size and VR state, so the second launch skips the measurement, and a launch that died mid-benchmark comes back with fewer layers instead of dying the same way again.

The voice has a budget too. The voice model and codec stay resident during the session. The bundled Qwen DLL runs on CPU when **External server URL** is empty. An independently configured external GPU server shares the card with the language model, so budget for speech and rendering in your Tier Policy reserve. Its initial audio buffer reduces interruptions; if generation cannot keep up, playback can pause until more audio arrives. Test the selected voice under game load.

Fallback is a design decision, not an error

Use `Tiers.StartupState` to decide when the startup sequence has finished:

State	Meaning
`NotStarted`	No cascade has begun.
`Starting`	Selection, loading or startup measurement is still running.
`Ready`	The cascade finished with a running model.
`Fallback`	The cascade finished without a model; use your authored dialogue.
`Cancelled`	Startup was interrupted; a later startup can retry.

`Tiers.IsStartupComplete` is true only for `Ready` and `Fallback`. `Tiers.OnStartupStateChanged` immediately calls a new subscriber with the current state; unsubscribe when the listener is disabled. `Tiers.IsResolved` and `OnTierResolved` describe the hardware verdict, which arrives before loading and may change during downgrades. Use them for tier information, not as a signal that dialogue can start.

On a machine that cannot run any model the plugin resolves to Fallback and **your game must still work**. Nothing throws; the character simply has no generated dialogue.

> Characters will not generate speech on this machine. Fall back to your own written lines.

Decide early what that looks like in your game — authored lines, a text-only mode, or a message. A game that assumes the model is always there will ship broken for the low end of its own audience.

```
if (Tiers.StartupState == TierStartupState.Fallback)
    UseAuthoredDialogue();
```

`Tiers.Current` is `HardwareTier?`, and it is null until the cascade has resolved a tier — which happens early in every run of the game, not once and for all. Code that needs the hardware tier value must say so:

```
if (Tiers.Current is HardwareTier tier)
    Debug.Log($"Running at {tier}");
else
    Debug.Log("The machine has not been measured yet.");
```

It is nullable on purpose. "Nobody has measured this yet" and "this machine cannot host a model" are different facts. To be told when the hardware verdict arrives, subscribe to `Tiers.OnTierResolved`; a subscriber that arrives after the verdict is called immediately. `Tiers.Reason` says why the tier is what it is — including "failed to load at 36 GPU layers" after a downgrade — and is worth a line on a settings screen.

Forcing a tier while developing

You cannot test the low end on a machine that never resolves there. The **Tier Bootstrap** has a *Force tier* field for exactly this, and a *Fail starts for testing* count that makes the next N start attempts fail so the downgrade cascade is exercised on purpose. Both are inspector fields; from code, `Tiers.Override(HardwareTier.Light)` before the model loads does the same as the first.

Test Fallback before you ship. It is the tier most likely to reach a player and least likely to have been run.

What "degrades" actually means

A lower tier uses a smaller model. Writing quality can degrade: shorter, more literal, more repetitive, and less consistent at holding a persona across a long conversation. The tier alone does not predict reply latency; hardware and backend matter too.

Vary the persona, questions and conversation length on each tier you target. A shopkeeper should keep the same stock facts; a companion should still follow events after a longer conversation. Check factual consistency, delivery and response time separately. Keep critical facts in your game state and author lines that must be exact.

A design that leans on the deterministic half (state, thresholds, events) degrades gracefully, because that half is identical on every tier. A design that leans entirely on the model's writing quality degrades badly. That is worth knowing before you build the game around it, not after.

Several candidates for one tier

Order custom `ModelManifest` entries best-first. At one tier the cascade considers installed entries in that order before stepping down; a missing first entry does not hide an installed second one. If loading the first candidate fails and the cascade can continue, it tries the next candidate at that tier before the lower tier. The existing cancellation, teardown and settle budgets still bound this process. This is not a promise that every load failure can be retried safely. The public `ModelCatalog.ForTier` lookup keeps returning the first declared entry; it is not an installed-model resolver.

`TierPolicy.maxTier` must be Fallback, Light, Standard or Premium (values 0–3). An unknown value produces a detected Fallback verdict with a diagnostic reason. Without an explicit tier override, the host is not configured. `Tiers.Override` retains its higher priority; the policy asset is not rewritten. Test a custom manifest, policy and backend together before shipping.

- The included Qwen DLL
- Qwen text and ready voices
- Whisper and text-model integration
- Piper and individual voices
- Files to keep with your game

Licensing

Offline AI NPC's own code uses the Asset Store licence. The language model, speech models, engines and native libraries have their own licences. Keep the licence texts and required notices for the exact files you include in your game.

The included Qwen DLL

The package delivers `Plugins/Windows/x86_64/OfflineAINPCQwenNative.dll`, identified by its delivery manifest. Its qwentts.cpp and ggml source is **MIT**. The static MinGW-w64 runtime includes its own copyright/licence notices, including winpthreads. The GCC runtime is **GPLv3 with GCC Runtime Library Exception 3.1**; it is not simply MIT and is distinct from Piper's eSpeak GPL distribution.

Preserve the exact five files under `Plugins/Windows/x86_64/Notices/`: `qwentts-LICENSE.txt`, `ggml-LICENSE.txt`, `MinGW-w64-runtime.txt`, `GPL-3.0.txt` and `GCC-RUNTIME-EXCEPTION.txt`. Exception 3.1 permits a qualifying Target Code combination with Independent Modules under its stated conditions; retain its full text with the GPLv3 notice. A different compiler, modified build or additional GPU library needs the terms for that exact distribution. The model weights remain a separate **Apache-2.0** download; the DLL does not bundle the talker or codec.

Qwen text and ready voices

The catalogued Qwen3 text GGUF models and Qwen3-TTS CustomVoice talker/codec weights are published under **Apache-2.0**, which allows commercial use under its conditions. Include the full licence, copyright notices and any applicable NOTICE; document modifications when required. A Model Manager licence label does not replace those files. See Apache-2.0 section 4.

The qwentts.cpp engine and ggml CPU source use **MIT**. Preserve their copyright and licence notices. Other native dependencies keep their own terms. GPU/vendor runtime libraries require the terms for that particular build; choosing Qwen weights does not license every library in a distribution.

In 1.1, Qwen speech uses named, ready **CustomVoice** speakers. Creating a voice from a description (VoiceDesign) is not part of this version. See Qwen speech for the model-mode boundary.

Whisper and text-model integration

Original Whisper tiny/base/small weights and whisper.unity source use **MIT**. LLMUnity's text compatibility sources use **Apache-2.0**. Preserve their licences and the third-party notices supplied with their exact native libraries. Install a compatible integration as a whole rather than mixing unrelated native builds.

Piper and individual voices

The supported archived rhasspy Piper source uses **MIT**; its phonemizer uses **GPL-3.0 espeak-ng**. The plugin communicates with Piper through a separate process. If you distribute GPL-covered components, retain their GPL licence and notices and provide the exact Corresponding Source through a GPL-compliant distribution method. Process separation does not remove these distribution obligations.

Each Piper voice has its own model and dataset terms. Choose a voice with terms suitable for your game and keep any required attribution. A compatible voice file or an engine's MIT licence does not grant rights to every voice. The package's voice records identify the checked files; for GPL components see the eSpeak boundary. These requirements do not mean that all Piper use is non-commercial.

Files to keep with your game

Keep the applicable LICENSE, NOTICE, copyright/attribution records, modification notices and any required source offer or source distribution. Check engines, native dependencies and voices as separate components. Third-party notices link the component and source records. Detailed development provenance remains in the source inventory.

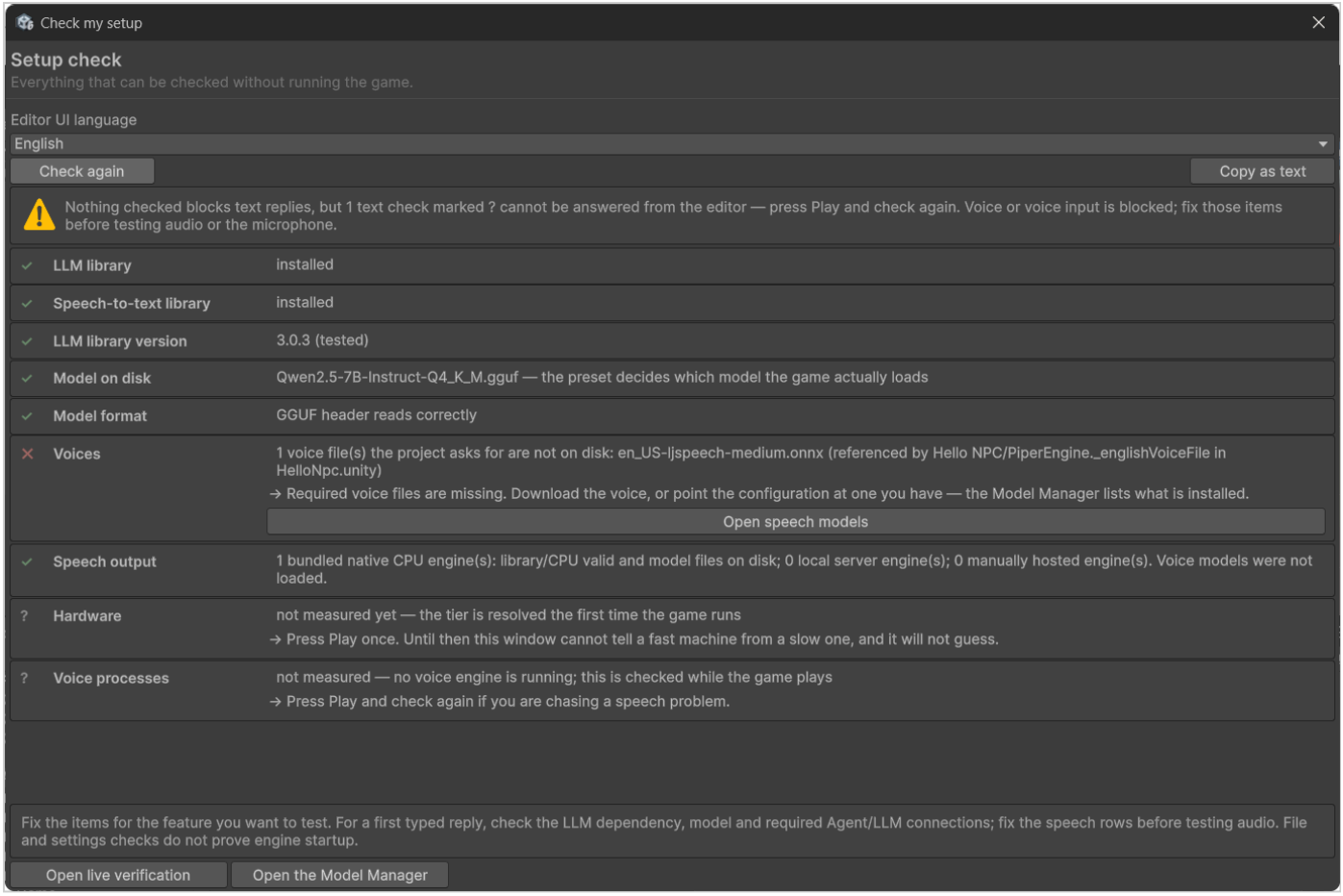
Configured local engines process dialogue on the player's computer. Explicit downloads contact their configured hosts; a remote speech URL sends the requested text to that endpoint. The plugin has no publisher telemetry.

- Bundled Qwen native speech
- The character speaks in the subtitle but there is no sound
- Piper is silent and every line ends in a synthesis timeout
- Local Piper import is refused
- The first sentence starts late, or a sentence has gaps
- A sentence is cut off, or never spoken: "did not finish the line within 60 s"
- No model weights found
- A model download stops, or fails after retries
- A download, import or build says a folder "is a link"
- The character never says anything, and push-to-talk stops working
- Holding the push-to-talk key does nothing, or the screen says "Not sent: …"
- Replies are much slower than the documented figures
- The engine is not ready, or the process died
- An engine you wrote against an external server says it is not ready
- A dependency is missing
- The character talks about furniture that is not there
- The character ignores "reply in one short sentence"
- A language switch produces the wrong voice, or an English reply
- The transcript comes back in a script nobody spoke, or as bracketed sound words
- Verification says a language is not configured
- Live verification cannot check a character outside Play
- Live verification played the line but did not mark the language verified
- An agent's tool call fails
- Removing the package
- Still stuck
- A custom speech engine stalls or throws inside a nested coroutine
- Scene wiring in the setup report

Troubleshooting

Ordered by how often it happens. Every message below is a string the plugin actually emits — search for it here before searching the internet.

First, always: `Tools ▸ Offline AI NPC ▸ Check my setup`. It reports what is missing and what to do about it, and it distinguishes "this is broken" from "this could not be determined" rather than guessing.



This is what a failure looks like: the check that failed is red, and the line under it is the fix rather than a restatement of the problem. Match your window to this one before reading further — most of what follows is the long form of a line in that list.

Four markers, and the fourth is the one worth knowing:

	Means
✓ green	checked, and fine
! amber	works, below the intended quality or speed
✕ red	cannot work until you fix it
? grey	**nobody has measured this yet** — not a pass

The grey ones are questions the editor cannot answer: your hardware tier before the game has ever run, the speech processes before one is up. Press Play once and check again. Grey is deliberately not green — "not measured" and "all good" must not look identical.

Bundled Qwen native speech

For the included Qwen route, check the native library, supported CPU and talker/codec model files. Read the engine's LastProblem/Console diagnostic. A successful file scan does not prove native generation or sound. For an explicitly configured HTTP URL, check that server and its model separately.

Windows x64 CPU is the bundled Qwen target. The bundled DLL works in Unity Editor and Windows players; its required CPU instructions are listed in the speech setup. macOS/Linux native libraries are not included. Mobile, web and consoles are not supported by this package. Linux and macOS are not verified.

The character speaks in the subtitle but there is no sound

For the included Qwen route, check the native library, supported CPU and talker/codec model files. Read the engine's LastProblem/Console diagnostic. A successful file scan does not prove native generation or sound. For an explicitly configured HTTP URL, check that server and its model separately.

If you deliberately use an external HTTP URL, inspect that server's own startup, model and backend diagnostics. See optional server configuration. These server installation errors do not apply to the ordinary DLL route.

For Piper, first check that both voice files are present:

> A Piper voice is a pair of files. Download both; the model alone loads and then says nothing.

A Piper voice is `name.onnx` **and** `name.onnx.json`. With the `.json` missing the engine used to start, report itself ready, and synthesise silence; it now refuses the voice and names the file, and the setup check flags an unpaired or half-written config.

Fix: download both files into `Assets/StreamingAssets/piper/voices/`.

Piper is silent and every line ends in a synthesis timeout

> `[PiperEngine] Synthesis timeout/failure after 45s ('…', first line on this voice), file '….wav' never completed`

If every line ends this way, look at Unity's temporary folder path: on Windows it contains your user name and the project's company and product names. Piper 1.2.0 takes the output path it is given as text in the system's legacy code page, so a path with letters outside plain ASCII — a Cyrillic or Chinese user name, say — can send the file elsewhere, and the file the engine waits for never appears. This was found by reading the code and has not been reproduced on a test machine.

Ways out: Qwen3-TTS, whose bundled native route writes no per-request WAV; Windows' *Use Unicode UTF-8 for worldwide language support* (Settings ▸ Time & language ▸ Language & region ▸ Administrative language settings); or company and product names in plain ASCII, which helps only when the user name is ASCII too.

Local Piper import is refused

Import file requires a nonempty `.onnx` and its readable, valid adjacent `.onnx.json`. Keep both originals in the same source folder. If either destination already exists, choose a different filename or inspect the existing pair before trying again; the import does not overwrite it. Symbolic links and reparse points are refused by this operation.

If the error says `Left ... untouched`, inspect the named path. Its contents changed or ownership could not be proved, so cleanup preserved it. Do not delete the named file without checking it. The original source files remain in place.

Cancel import stops copying or verification cooperatively. The window stays available while cleanup checks ownership before removing copies. A conflict message means another operation still owns this path; cancellation does not release a worker's files before it has actually finished. Wait for completion and retry. A download refuses a destination whose presence, file identity, size or recorded timestamps changed after its replacement intent was recorded; inspect the preserved destination and partial file before starting a new attempt. A successful import reports its original destination even if the configured models folder changed while the copy was running. On Windows, use ordinary drive or UNC paths without a device/extended-namespace prefix or components ending in a dot or space; these ambiguous path forms are refused.

The first sentence starts late, or a sentence has gaps

Qwen speech may produce audio slower than playback. **Lead seconds** controls the initial buffer; raising it can smooth a line but adds delay and does not increase generation speed. Keep replies short and reduce competing CPU work. The included DLL has a fixed CPU profile. GPU speech requires an optional external setup verified separately.

A sentence is cut off, or never spoken: "did not finish the line within 60 s"

If the line exceeds **Request Timeout Seconds**, inspect the engine diagnostic, shorten the reply and reduce CPU contention. The included DLL uses CPU; a longer timeout adds waiting, not speed. Stop or cancellation requests cooperative cancellation and still awaits the actual native worker before context free. A server GPU configuration is an optional, separately verified HTTP route.

No model weights found

> `[OfflineAINPC] No model weights found. Expected one of: …`

The tier system resolved a tier, asked the catalog for that tier's model, and the file is not on disk.

Fix: open `Tools ▸ Offline AI NPC ▸ Model Manager` and download the preset for your language and tier, or place the `.gguf` under the exact file name the message lists in one of the places the plugin looks — the Model Manager's folder, `persistentDataPath`, or `StreamingAssets`, in that order. A renamed file is not found — the name in the manifest is the name on disk.

A model download stops, or fails after retries

> `no data arrived for 60 s after 3 automatic retries. The partial file was kept; Resume continues from it.`

A stalled transfer, a dropped connection and a busy or failing server (408, 429 or 5xx) are retried by themselves three times, after 2, 4 and 8 seconds or after the server's `Retry-After`, up to two minutes. After that the Model Manager shows the reason, and **Resume** continues the partial file. A missing file or a refusal is not retried: the link or the server is wrong. The Console says "downloaded and verified" only when a SHA-256 was checked. Without a checksum it says only the size was checked, and "not verified" when the catalog has no size either.

A download, import or build says a folder "is a link"

> `'…' is a link (a junction or symbolic link). Model files are written, moved and deleted only along ordinary folders, …`

Model files are written, copied and deleted only along ordinary folders, so a junction or symbolic link cannot send them somewhere else. Downloads and imports refuse a models folder reached through one, and an import refuses a file picked through one too; a build that includes models refuses a project opened through one. A build that includes no model is not affected.

Fix: choose a models folder on an ordinary path in the Model Manager, pick a file to import at its real location, and open the project from its real folder rather than through the link — or leave the models out of that build.

The character never says anything, and push-to-talk stops working

Almost always the busy lock. If your controller sets a `_busy` flag and an early return, an exception or a cancelled coroutine skips clearing it, the character is locked out for the rest of the session, silently.

Fix: release it in a `finally`, on every path. See Dialogue. This is the single most common integration bug in this codebase's own history.

Holding the push-to-talk key does nothing, or the screen says "Not sent: …"

Nothing at all: is Push To Talk on the character, with *Listen To Key* ticked? The key is ignored while a text field has keyboard focus — click outside the text box first. If you drive it from your own input, call `Begin()` and `End()`.

"Not sent: …" gives the reason, also in `PushToTalk.LastDrop` and the `Dropped` event:

The screen says	Do
the press was too short	hold the key while you speak; a press under *Minimum Hold Seconds* (0.3) is discarded
nothing loud enough was heard	check the microphone, its level, and that Windows lets desktop apps use it (Settings ▸ Privacy & security ▸ Microphone)
no speech recognition component is on the character	install whisper.unity and add Whisper Speech Recognition, or your own `ISttEngine`
speech recognition is not ready: …	the detail names the model; download it in Model Manager ▸ **Speech recognition**, or set *Model File Name*. Just after entering Play it can say the tier is not settled yet: wait for the *Startup verdict* line in the Console
no words were recognized / only noise was recognized	speak closer to the microphone
"…" is what recognition invents from near-silence	the gate caught a typical Whisper hallucination; speak again
recognition failed: …	the detail is the recognizer's error, also in the Console. "English-only" there means an `.en` model with another language: download a multilingual one such as `ggml-base.bin`

Replies are much slower than the documented figures

Check, in this order:

1. **Is something else generating?** Background conversation costs about 5× — see Limitations.
2. **Is this the first turn?** The first reply of a session can take longer than later ones; in the recorded development session it took about 2.1 s (Limitations). A warm-up turn during a loading screen moves that work out of play.
3. **Is the prompt changing every turn?** A changed system prompt can need processing again instead of reusing the cached prefix. Build the stable part once and put changing facts in the turn.
4. **Which tier resolved?** The startup log says so: `Startup verdict: tier Standard — …; model qwen3-4b-q4km, 36 GPU layers`. A machine that fell to a lower tier is running a smaller model on fewer GPU layers, and will be slower per token than the reference figures.

The engine is not ready, or the process died

> A speech process died or failed to start. The plugin restarts one automatically within its budget; if this keeps happening, check the executable path…

Piper remains an external process: a dead voice is restarted on its next request and the default voice watchdog has a budget of three restarts per minute. The bundled Qwen DLL is an in-process context, not a watchdog child. Inspect LastProblem, model files and CPU support; await StopAsync before replacing its owner. If native cleanup cannot be confirmed, keep that owner and its diagnostic rather than treating the context as released. A server attached by URL is externally owned and is not restarted or stopped by this component.

What you lose: the reply in flight, not the session.

If it happens constantly: the voice file is likely corrupt or truncated. Re-download it and check the file size against the model card.

An engine you wrote against an external server says it is not ready

The `VoicevoxEngine` example on the Languages page talks to a separate application with an HTTP server, and such a server takes time to boot. The example retries a health check and flips to ready when the server answers; an engine of your own should do the same.

Fix: start the server before entering Play mode, or wait — the example connects on its own.

A dependency is missing

After an upgrade, errors about `LLMAgent` or `LLMClient`: update LLM for Unity to 3.0.3 or newer. If 1.0 set `LLMUNITY_PRESENT` for an older loose-folder install, that symbol can survive a broken compile: Unity cannot load the new detector yet. Remove only `LLMUNITY_PRESENT` from Player > Other Settings > Scripting Define Symbols for the affected target to recover compilation until you install the supported dependency. Details: Upgrading from 1.0.

> Install LLM for Unity (package id `ai.undream.llm`). Without it there is no model to talk to.
>
> Install whisper.unity (`com.whisper.unity`) to accept voice input. Typed input works without it.

Both are external and free. The plugin compiles without either — the features that need them are compiled out, so a missing dependency is a missing feature and never a compile error. For a package that is the assembly's version defines; for LLM for Unity installed as a folder under `Assets`, it is the `LLMUNITY_PRESENT` symbol the plugin's detector sets.

> (newer than the tested versions; expected to work)

You are on a version this plugin has not been tested against. Usually fine.

The character talks about furniture that is not there

Perception is off. Measured in the same scene: with it off the character mentioned a fireplace, a window seat and a cat, none of which existed; with it on the character spoke only about real objects.

Fix: see Perception. Line of sight is measured to the centre of the object's renderers, not to its pivot, so a floor-level pivot under a metre-high body no longer hides it. A marker with no renderer is aimed at its pivot — give it an Aim Point at the height a person would look at. `PerceptionQuery.Explain(poi)` says exactly why an object is or is not in context.

The character ignores "reply in one short sentence"

Expected. **Small models do not honour worded length limits.** The token ceiling and the sentence limit are what actually keep replies short.

Fix: lower the ceiling, not the wording in the persona. The default is the reply profile's 128 tokens. A language definition's **Max tokens** and **Max sentences** take effect on a Hello NPC Character whose **Reply Limits** is **Current Language**, and in your own dialogue code once it applies the language's length rule — see the *Reply length* rows in Languages.

A language switch produces the wrong voice, or an English reply

Two different causes:

- **Wrong voice, right language:** the language has no voice bound for its code in the engine binding table. The manager logs which codes it has: `No engine bound for 'ko'. Bound languages: ja, en, es, ru, zh-Hans`.
- **Right voice, wrong language of reply:** the voice was switched but the prompt was not. Switch through your language manager so the system prompt is rebuilt, not through the TTS manager alone. This one hides well — the voice is correct both times, so the switch looks like it worked and only the words are wrong.

The transcript comes back in a script nobody spoke, or as bracketed sound words

> the player said "do you hear me" and the box filled with the same katakana sound-word four times over

Two separate faults, and they arrive together often enough to look like one.

- **Wrong script:** recognition was told to expect a language the player is not speaking. Whisper Speech Recognition sets it from the project's current language on every press; with a `WhisperManager` of your own, check its `language` field. Recognition does not refuse a mismatch — it transcribes what it heard using the

language it was told to expect, so English audio under a Japanese setting comes back as Japanese sound-words. Set the language to what your players speak; if you copied your speech setup from another scene, check this field first, because everything else in that setup copies correctly and only this one is scene-specific.

- **Bracketed sound words:** recognition annotates non-speech — `(coughs)`, `[BLANK_AUDIO]`, `*sighs*`, ♪ — and a run of those is not short, not silent, and not a known hallucination, so `UtteranceGate` passes it. **The gate does not strip annotations, by design:** it judges whether somebody spoke, and someone who coughs before a real question did speak. Deciding what to do with the annotation is the caller's, because a game that shows a transcript and a game that feeds one to a model want opposite things.

Fix for the second: Push To Talk already does this. On your own input path, strip annotations before you call `Inspect(text, language)`, and treat a line that is *only* annotation as nothing said. Removing the brackets rather than rejecting the line matters — `(coughs) where were you on the fourteenth` is a real question with a cough in front of it. `TranscriptCleaner.Clean(raw, out cleaned)` in the plugin does exactly this: it strips `(...)`, `[...]`, `*...*` and note-marked runs, and returns `OnlyNoise` when nothing is left.

Verification says a language is not configured

> the live-verification window refuses a language code and lists the ones the catalog has

Correct and deliberate. Ask `LanguageRuntime.IsConfigured(code)`, which answers about the catalog. Do not use `Find`/`FindOrDefault` for that question — they fall back to the default language, so an unconfigured code comes back looking supported.

Live verification cannot check a character outside Play

Every check generates a reply before it voices it, and the language model's agent finishes starting inside the player loop, which does not run in edit mode. So Hello NPC and most characters can be checked only in Play. Outside it, Hello NPC refuses at once and says so; another character gets a few seconds, then the window says why, rather than waiting for a load that cannot complete.

Not a bug. Enter Play mode; the same window works there.

When a custom probe calls your normal dialogue controller, direct its conversation memory writes to a temporary `ConversationMemorySandbox(originalStore)` and restore the original adapter only after cancelled generation has settled. This keeps probe questions out of persistent memory without confusing an absent key with a saved empty value. Do not restore history while a native generation can still append to it.

Live verification played the line but did not mark the language verified

Expected. The window can tell that audio played, not which language it was in. After **Speak** it asks whether the reply was spoken in the selected language, and only **Yes, record it** marks the cell. **Advanced ▸ Live verification — verify everything (console)** prints `PLAYED` and records nothing. A language without a built-in test line gets an empty one for you to fill in, and the console sweep skips it.

An agent's tool call fails

> Port 47371 is already in use — another program, or another Unity editor running this server, holds it. Choose another port.

Two editors have the server switched on, or another program owns the port. Pick another port in `Tools ▸ Offline AI NPC ▸ Agent tools (MCP)` and give the client the new address.

> The editor did not start 'say' within 30 s. It is not updating: a modal dialog is open, it is importing or compiling, or a debugger has paused it.

Look at the Unity window. A dialog waiting for an answer holds every call; answer it and call again.

> The editor is not in Play mode, and 'guard' most likely generates only in Play.

Expected with LLM for Unity, for the reason in Live verification cannot check a character outside Play. Have the agent call `play_mode` with `action: "enter"` and wait until it reports `playing`.

A refused connection for a few seconds after entering or leaving Play is the script reload restarting the server. The call works again once it is back.

Removing the package

Delete `Assets/OfflineAINPC` from Unity's Project window: the plugin then removes the `LLMUNITY_PRESENT` symbol it set for LLM for Unity installed as a folder. If you delete the folder outside Unity, remove `LLMUNITY_PRESENT` yourself in Project Settings ▸ Player ▸ Other Settings ▸ Scripting Define Symbols, for every platform you build; left behind, it breaks the next install of the plugin into a project without LLM for Unity. Remove `AINPC_DEBUG` there too if you turned on verbose logging — deleting the package leaves it either way.

Deleting the folder leaves everything the plugin keeps outside it; remove what you no longer need:

- the models folder chosen in the Model Manager — on Windows `%LOCALAPPDATA%\OfflineAINPC\Models` unless you picked another;
- `ProjectSettings/OfflineAINPC/` and `ProjectSettings/OfflineAINPCModels.asset` — the required-models list and the build choices;
- `UserSettings/OfflineAINPC/verification.json` — Live verification results;
- the speech programs and voices you put in `StreamingAssets` (`qwen3-tts/`, `piper/`);
- the plugin's editor preferences, keys starting with `OfflineAINPC.`.

Still stuck

`Tools ▸ Offline AI NPC ▸ Check my setup` ▸ **Copy as text** puts a report on the clipboard you can paste into a support request: hardware, tier, model, dependency versions, what is installed and what is missing. It is not sent anywhere — the plugin has no telemetry of any kind — but file paths in it can contain your user name; look it over before you send it. For a fuller log, turn on **Advanced ▸ Logging ▸ Verbose logging (AINPC_DEBUG)** and choose **Advanced ▸ Logging ▸ Minimum level ▸ Debug**, reproduce the problem, and attach the Console; set both back afterwards.

A custom speech engine stalls or throws inside a nested coroutine

The speech driver owns nested iterators, including legacy one-buffer engines. Exceptions in their factory, iterator or cleanup are reported and the unit can continue as subtitles. Cancelling stops the nested work and ignores audio that arrives afterwards.

In Play mode the session's synthesis deadline runs in real time: a `WaitForSeconds` inside your engine keeps its scaled duration, so a paused game still reaches the deadline. Code after a fixed-update or end-of-frame wait stays in that Unity phase. Outside Play mode, drive an engine that yields Unity wait instructions through the editor driver; a standalone session does not guess how long they take.

If you drive `SpeechSynthesis.Run` yourself, stop its hosting coroutine and dispose its returned enumerator when abandoning a wait. If you own a `StreamingSpeechSession`, call `Cancel()`/`Dispose()` as part of your component's disable or interruption path.

Scene wiring in the setup report

Check my setup inspects the Hello NPC sample's required Agent and LLM references. Audio and Engine are optional connections for speech output; their absence does not block a configured language model from answering in text. The check also inspects the live-verification adapter's Agent/LLM wiring. It does not start a model, add components or change the scene. A custom character can opt in by implementing `OfflineAINPC.Diagnostics.ISetupDiagnosticSource` and reporting its required components and references through `SetupWiringIssues`. Keep that collector read-only. An exception is reported as an inspection failure and does not hide the other characters' findings. No scene row means no declared issue was found; it is not proof of native or player readiness.

- Layout
- The boundary
- External processes
- Dependencies

Offline AI NPC — architecture

A fully offline conversational-NPC stack for Unity: local language-model dialogue, local speech recognition, local speech synthesis, and per-character state and memory. To retain values across scenes and restarts, configure persistent state through `NpcStateComponent.Store` and conversation memory through `NPCMemory.Store`. Moods remain transient. No cloud, no API keys, no per-token cost.

This page is the map of the code. The other pages say what to do; this one says where things are and why the boundaries sit where they sit.

Layout

```
Runtime/
  Core/          language catalog and runtime, sentence rules, JSON, process lifecycle,
logging
  Speech/Tts/    ITTSEngine, the Qwen3-TTS engine (Qwen3Tts/), the Piper engine, the speech
                 session and the switching manager
  Speech/Stt/    Push To Talk, ISttEngine, the shared microphone loop, the push-to-talk
                 recorder, the utterance gate, the transcript cleaner
  Memory/        rolling per-character conversation memory over an injectable key/value
store
  State/         axes, bands, traits, needs, relationships, milestones, the state block
  Perception/    points of interest, the perception query, the context block
  Actions/       verbs, targets, the grammar, the dispatcher, the action turn
  Hardware/      the hardware probe, tiers, the model catalog, the startup cascade
  Models/        model files: locating, reading GGUF headers, downloads, the shipping
manifest
  Diagnostics/   the in-scene diagnostics component
  Verification/  what the live-verification window and the latency runner need from a
character
  Integrations/  LLMUnity/ — the host, the grammar binding and the verification adapter;
                 compiles only when LLM for Unity is installed. Whisper/ — Whisper Speech
                 Recognition; compiles only when whisper.unity is installed
Editor/          the windows (Welcome, Model Manager, Presets, Check my setup, Prompt
debugger,
                 Live verification, Agent tools), inspectors, build hooks, and the MCP
server
                 for coding agents (Mcp/)
Samples/         HelloNpc, LivingCharacter, WorldAware — three scenes, each its own
assembly
```

```
Documentation/   these pages
LICENSES/        third-party notices and dated copies of every licence and card relied on
```

The plugin's own tests live in the source repository, not in the package.

The boundary

`OfflineAINPC.Runtime` has an empty `references` list and an empty `precompiledReferences` list. Beyond the engine it takes **nothing at all** — not a host game's assembly, not LLMUnity, not whisper.unity, and no third-party library — not Newtonsoft.Json either: the JSON it needs is `Core/Serialization`, about four hundred lines that belong to the package. This assembly boundary separates integration dependencies. Successful C# compilation does not verify native backends or platform support; those need compatible binaries and a separate player test.

Everything the plugin cannot know is injected, with a working default where one exists:

Seam	Interface	Default	You supply
Language model	`ILlmHost`	`LLMUnityHost` (when LLM for Unity is installed)	a host for another inference library
Persistence	`IKeyValueStore`	`PlayerPrefsStore`	your save system
Current language	`ILanguageSource`	`LanguageRuntimeSource`	a wrapper over your own settings
Speech engine	`ITTSEngine`	`Qwen3TtsEngine`, which the built-in definitions select (`PiperEngine` second)	any engine, per language
Speech recognition	`ISttEngine`	`WhisperSpeechRecognition` (when whisper.unity is installed)	another recognizer for Push To Talk
Piper voice casting	`IPiperVoiceRouter`	the component's serialized fields	your own routing
Loading UI	`TTSEngineManager.OnSwitching` / `OnSwitched`	none	a listener that drives your overlay
A character, for verification	`ILiveVerifiable`	`LLMUnityLiveVerificationAdapter`	an adapter over your own controller

External processes

Model Manager downloads the speech model files. The package includes a **Windows x64 CPU Qwen native DLL**; leave **External server URL** empty to use it. The ordinary Qwen route needs no manually started HTTP server, executable or compiler setup. Keep an existing working Piper route. See the native speech setup and Licensing.

Piper and engines you add that spawn a separate program register their owned child with ExternalProcessRegistry so reload, Play exit and application quit can drain it. A Qwen server attached by URL remains externally owned. The ordinary Qwen DLL has no child server process; its native context is released only after actual workers settle.

Dependencies

LLM for Unity and whisper.unity are installed by you, not bundled — bundling either is a duplicate-assembly compile error for anyone who already has them. The code that touches them lives in assemblies that compile only when the dependency is present: `OfflineAINPC.LLMUnity` and the HelloNpc sample on `LLMUNITY_PRESENT`, `OfflineAINPC.Whisper` on `WHISPER_PRESENT`. For an installed package, the assemblies' `versionDefines` on `ai.undream.llm` and `com.whisper.unity` set those symbols. For LLM for Unity installed as a folder under `Assets`, the plugin's detector sets `LLMUNITY_PRESENT` project-wide, and takes it away when the plugin is deleted from inside Unity. A missing dependency is a missing feature, never a compile error.

- Bundled compatibility source text
- External Unity dependencies
- Included Qwen native speech library
- External speech engines
- Model and voice files
- Before you redistribute

Third-party notices — Offline AI NPC

This is a scoped inventory, not a licence grant or clearance of an entire native/model stack. The plugin export contains its own code, configuration and documentation, plus text compatibility sources with their original licences and modification notices. External packages, model weights, Piper and espeak data are installed separately. The exact Windows x64 CPU Qwen native DLL is included as described below. An actual game distribution can include components that the Asset Store package excludes.

The dated source revisions and the model files' expected checksums are in `STACK_SOURCE_INVENTORY_2026-10-02.md`. An observed source tag, expected weight hash and installed binary hash are different pins. Retain the notices of the exact artifacts you redistribute.

Bundled compatibility source text

`Editor/Dependencies/CompatibilityData/ai.undream.llm/` retains the LLMUnity 3.0.3 Apache-2.0 licence, original third-party notices, source revision, exact before/after hashes and modified source text. `Editor/Dependencies/CompatibilityData/com.whisper.unity/` retains the whisper.unity 529a MIT copyright/permission notice, source revision, exact before/after hashes and two modified managed source files. Payloads are data files; they do not compile a second SDK when this plugin is imported. Both directories include modification notices. These compatibility-source changes include no model files or SDK native binaries. The separately described Qwen DLL delivery is included in the package. The optional compatibility window applies them only after Preview and an explicit Apply.

External Unity dependencies

Component	Source licence	Evidence and distribution boundary
LLM for Unity v3.0.3, UndreamAI	Apache-2.0	Source. External dependency; its Editor setup can download LlamaLib v2.0.5.
LlamaLib v2.0.5, UndreamAI	Apache-2.0	Source. Native distribution includes patched llama.cpp and other libraries.

Component	Source licence	Evidence and distribution boundary
llama.cpp, ggml authors	MIT	LlamaLib pins 872646b30c342c5fa98efec002f94fdb4f1f1ea5 and applies patches; that source pin is not proof of every installed build.
FeatureDetector	CC0-1.0	LlamaLib submodule 077393260d5e0e19938582a33b9dd0e74baa8ff2.
USearch C# bindings and six native artifacts	Apache-2.0 source text	Included by LLMUnity, outside this plugin. Local bytes match its v3.0.3 tree; the revision of the original NuGet/native build is not stated by the package. Retain its actual copyright/NOTICE material.
whisper.unity v1.4.0, Macoron	MIT	Integrator. Optional recognition bindings and native libraries; typed input works without it.
whisper.cpp and its ggml libraries	MIT source	The integrator workflow defaults to v1.7.5 but permits a different build ref, so the integrator's native libraries do not name their exact source revision.
Newtonsoft.Json / Unity package 3.2.2	MIT notices	LLMUnity's dependency. The plugin uses its own JSON reader and does not itself require Newtonsoft.Json.

OpenSSL, cpp-httplib, nlohmann/json, OpenMP/compiler runtimes and backend libraries belong in the inventory of the native build you ship. Source licences do not identify the versions or notices of every installed runtime. A LlamaLib installation can contain NVIDIA CUDA/cuBLAS and Vulkan loader DLLs; the NVIDIA EULA and each runtime's own redistribution conditions apply to them.

Included Qwen native speech library

`Plugins/Windows/x86_64/OfflineAINPCQwenNative.dll` is the ordinary empty-URL Qwen route. Its delivery manifest identifies 5366978 bytes, SHA256 `4de29ccecbb2962452c2c9f2a1a4dbcc49f1c56e50f5aad74cb25a1739d0d9f3`, qwentts.cpp `6fae92914045cd83364d2845ceaa0f7969727319` and ggml `40e16e4a814f7fe851a0c486fb9e8c722e957830`. This is Windows x64 CPU, eight threads, static ggml, flash attention off and max batch one. It loads and speaks in the Unity Editor and in a built Windows player; no Linux/macOS binary is included.

qwentts.cpp and ggml are MIT. The static MinGW-w64 runtime has its own notices, including winpthreads; GCC runtime code is GPLv3 with Runtime Library Exception 3.1. Preserve all five exact files in `Plugins/Windows/x86_64/Notices/`: `qwentts-LICENSE.txt`, `ggml-LICENSE.txt`, `MinGW-w64-runtime.txt`, `GPL-3.0.txt` and `GCC-RUNTIME-EXCEPTION.txt`. The exception applies under its stated Eligible Compilation Process / Independent Module conditions; it is not a blanket MIT label or the Piper/eSpeak GPL distribution boundary. The talker and codec are separate Apache-2.0 weight downloads, not bytes in this DLL.

External speech engines

Component	Source licence	Current scope
qwentts.cpp, omnivoice.cpp authors	MIT	Original source pin, ggml 77cdef82aad64f141164fac02076e36d5848474c; cpp-httplib and yyjson are MIT. A server executable you install yourself is a separate build with its own source and notices.
Archived Piper, Michael Hansen	MIT source	2023.11.14-2. Supported separate process; preserve working user configurations. The newer OHF GPL project is a different distribution.
Archived piper-phonemize	MIT source	Piper's phonemizer. The archived master snapshot does not identify the build inside a 2023 Piper release.

Component	Source licence	Current scope
espeak-ng fork and dictionaries	GPL-3.0 source/data terms	Linked within Piper. Exact fork, generated dictionary inputs/build and Corresponding Source must match the distributed artifacts. See boundary.
ONNX Runtime	MIT + third-party notices	Existing Piper DLL resource metadata is consistent with a 1.14-era build; this does not replace a complete binary/source match or its ThirdPartyNotices.
fmt / spdlog	MIT	Archived Piper CMake requests fmt 10.0.0 and spdlog 1.12.0. Verify actual build inputs.
libtashkeel_model.ort	UNKNOWN weight/data provenance	Phonemizer distribution includes this Arabic preprocessing model. Code MIT does not establish its weight/data licence.
VOICEVOX ENGINE / character voices	Software and per-character terms	Documentation-only extension example; not shipped, integrated or cleared as a profile here. Review the exact software edition and each voice's terms.

Built-in presets use Qwen3-TTS CustomVoice. Preserve existing Piper selections and review the exact licence boundary of any engine or voice you add.

Model and voice files

Files	Declared terms and provenance boundary
Qwen3-1.7B Q8_0; Qwen3-4B / 8B Q4_K_M GGUF	Qwen repositories advertise Apache-2.0; full archived licence texts exist.
Qwen3-TTS 0.6B / 1.7B CustomVoice talkers and 12Hz tokenizer GGUF	Official base/codec and Serveurperso conversion repositories advertise Apache-2.0. Preserve the notices of both.

Files	Declared terms and provenance boundary
Whisper tiny.en, base.en, base, small ggml	MIT original models and separate conversion provenance. Their expected checksums are in the preset catalog and in the dated inventory.
Piper ljspeech medium/high	Card records public-domain LJ Speech and training from scratch. It does not waive engine/model distribution notices.
Piper libritts-high, 904 speakers	Card records CC BY 4.0 LibriTTS train-clean-360 and training from scratch. Preserve applicable dataset/model attribution.
Piper dmitri, sharvard and other lessac fine-tunes	Dataset label and fine-tune base are separate. The plugin recommends no profile built on a base its publisher restricts; existing user configurations are kept.
NonCommercial voices and voices of unknown terms	Not recommended and not supported by the plugin.

Weights are never bundled in this plugin. Model Manager downloads require an explicit user action; LLMUnity's Editor library bootstrap is a separate dependency action. The catalog's repository URLs may resolve a mutable `main`; configured SHA256 pins expected bytes where present, not the repository's complete licensing or training history.

Before you redistribute

Before redistributing a chosen stack, record exact binary and model hashes, source/build revisions and patches, licence/copyright/NOTICE copies, and dictionary and preprocessing data. GPL-covered components additionally need a compliant Corresponding Source distribution method.

Of native code this package contains only the named Qwen DLL; it contains no model weights and no espeak data. Keep the DLL's five notices with it in your game's distribution.

- [1.1]
  - Added
  - Compatibility
  - Default profiles
  - Fixed
- [1.0.0] — 2026-08-17
  - Added — the guide for someone who has done none of this
  - Added — the documentation a buyer reads
  - Changed — documentation
  - Measured — can a 4B defend a brief without contradicting itself?
  - Measured — does the state block change what the model says?
  - Added — internal state (mood, relationships, needs, traits)
  - Added — points of interest (perception with no wiring)
  - Added — structured actions via grammar-constrained decoding
  - Added — preset catalog
  - Added — hardware tiers, auto-tuning, degradation cascade
  - Added — streaming reply parser and sentence-level TTS handoff
  - Added — package skeleton and the assembly boundary
  - Moved (behaviour unchanged)
  - Known limitations

Changelog — Offline AI NPC

All notable changes to the plugin. The format follows Keep a Changelog.

[1.1]

Added

- First-dialogue setup: choose a Hello NPC and an existing catalog model, preview the assignment and apply it with Undo. Start with a typed reply; speech and microphone setup are optional.
- Qwen3-TTS's ordinary local route includes a Windows x64 CPU native DLL. With an empty External server URL it needs model downloads, not a manually built or launched tts-server executable. A nonempty URL / AttachTo keeps optional external HTTP support. Existing Piper selections, voice pairs and buffered speech engines remain supported.
- Native Qwen cancellation requests cooperative cancellation and awaits actual generation, callbacks and worker settlement before context free; a cancelled presentation is not proof of native release.
- Optional Push To Talk with Whisper Speech Recognition, including recording feedback and a recognition-language setting for each input control.
- Editor windows, inspectors and guides in English, Russian, Spanish and Simplified Chinese. Optional editor MCP tools start disabled and are excluded from player builds.
- Public turn, input and startup-state hooks for custom game UI; background model-file imports with progress and cancellation, plus assignment of an existing GGUF.

Compatibility

- Existing `Say`/`TrySay` calls, event bindings and working Piper setups remain supported. See Upgrading from 1.0 before replacing package assets.
- Service menu items move under **Tools ▸ Offline AI NPC ▸ Advanced**. Publisher build scripts using removed export/documentation helpers need the migration in that guide.
- The included Qwen DLL targets Windows x64 CPUs with AVX2, BMI2, FMA, F16C and SSE4.2. The DLL runs inside Unity Editor and Windows players. macOS/Linux libraries are not included. Validate the actual model, Unity backend and target Player.
- Empty-URL Qwen scenes now select the DLL route; existing Piper bindings are not switched automatically. Keep serialized settings and custom engine routes when updating.

Default profiles

- New Qwen speech presets use 0.6B CustomVoice Q4 with the Q8 codec across LLM tiers. New Qwen engines use the CPU DLL with External server URL and Style empty; existing serialized settings remain.
- Serena is the default speaker for every built-in Qwen language and preset.
- The non-thinking reply profile has a 128-token ceiling and a one-sentence delivery limit. Assign `Tiers.Sampling` to replace it; turn **Override agent sampling** off on the LLM host to keep the values typed on each agent. Larger talkers and GPU are manual options.
- Hello NPC **Reply Limits**: Token Ceiling Only (the default: every sentence within the token ceiling is spoken), Reply Profile (the profile's sentence limit ends the reply) or Current Language (the language definition's Max tokens and Max sentences).
- Whisper remains the optional speech-recognition integration. VoiceDesign is not part of this version.

Fixed

- Selected microphone names now control the shared recorder. Missing named devices report a capture error; changing inputs retires unfinished recordings and stale recognition.
- Borrowed LLM clients track their native owner. Calls after owner disposal are rejected before entering the native library; active calls settle before the context is released.
- Invalid or partially allocated LoRA paths release only initialized buffers; failed command-based model creation releases its native library.
- Native LLM replies, history and status strings release their owned buffers after copying, including failure paths, while borrowed callback strings remain untouched.
- Windows Editor builds release native file handles between transactions under Mono. Build journal updates retry transient access conflicts and preserve changed files.
- Interrupted Hello NPC replies stop their presentation and keep the character busy until generation settles. Subtitles follow buffered or paused speech.
- External HTTP Qwen speech requests settle on cancellation, including unfinished HTTP error bodies; truncated fixed-length PCM responses report failure.
- Conversation memory and character state preserve unread data and ownership during retries and verification; failed state saves remain pending for retry.
- Model imports, downloads and build staging check file ownership and selected model/ voice inputs. Setup reports distinguish file readiness from native startup.

- Speech sessions advance with `AudioSource.pitch`, so subtitles and the end of a line follow a voice played faster or slower.
- Reply splitting waits for the next visible character after an ellipsis, and after "No.", "Fig." and "Vol.", before deciding that a sentence ended.
- A Windows build whose output is not a Player `.exe` (Create Visual Studio Solution, for one) finishes with a warning naming the notice files to copy by hand. A project or build folder under a junction no longer fails the build.
- Piper removes its own output WAV files older than a day when it starts.

Detailed 1.1 history and historical measurements preserves the full development record. See Licensing for the current dependency and model records.

[1.0.0] — 2026-08-17

First public release. Everything below was built before it, and is listed rather than summarised because a first changelog is the only one a buyer can read as a description of what they are getting.

Added — the guide for someone who has done none of this

- **`BeginnerGuide.md`**. The existing pages assume a reader who has installed a Unity package with external dependencies before; this one assumes nothing, and every step says how to tell it worked rather than only what to do.
- It closes two gaps found by measuring rather than by reading: **the required Unity version appeared nowhere** in the documentation, and there was **no chapter on making a player build** — even though models are excluded from builds by default, an included one is staged into `StreamingAssets` only for the duration of the build, and a build with broken model references is refused rather than shipped dead.
- A map of every runtime area with the problem it solves, all six editor windows by the question each answers, and what the plugin deliberately does **not** do.
- An API reference organised by what you do with a type — put on a GameObject, implement, call, author as an asset — because a concept you cannot name is one you cannot look up. Public types named in the docs went from 34% to 57%; the rest are enums and internals, and the guide says so instead of implying completeness.

Added — the documentation a buyer reads

- **`IsThisForYourGame.md`**, deliberately first and deliberately willing to talk a wrong-fit buyer out of the purchase. Names the games this is wrong for — mobile, crowds of talking NPCs, combat barks, tightly authored narrative, low-spec audiences — and says a dialogue tree and this plugin are complements rather than competitors.
- **`Limitations.md`** as its own page, not a footnote. Every limit carries its mitigation where one exists and says so plainly where none does.
- **`Languages.md`** answering the question in three tiers instead of one number: ready out of the box, works-bring-your-own-voice (roughly a hundred languages), and not viable. The plugin does not gate on that list and the page says so.
- **`Dialogue.md`** — the complete controller a buyer otherwise writes from scratch. Extracted and compiled against the public API rather than written from memory.
- **`Recipes.md`**, `GettingStarted.md`, `HardwareTiers.md`, `Licensing.md`, `Troubleshooting.md` (mapped from the error strings the plugin actually emits), and an index.

- `DocumentationSnippetTests` — the code printed in the documentation, compiled and run, because a snippet that does not build fails in the buyer's project rather than in ours.

Changed — documentation

- **`State.md` now opens with the ten-line version** and reaches the full apparatus afterwards. Building the demo against the public API needed exactly "the player pressed, subtract eighteen"; it met the catalog-plus-traits-plus-persistence apparatus first, concluded the system was too heavy, and wrote forty lines of its own rules. The short path always existed and was not shown.

Measured — can a 4B defend a brief without contradicting itself?

- 20 sessions x 16 turns = 320 turns, neutral questions only, Qwen3-4B: **zero departures from stated facts**. The model is a reliable witness to what it was told.
- **One derived fact was wrong in 19 sessions out of 20 — identically wrong every time.** A consistently wrong answer reads as a designed tell. Never ask a small model to compute.
- Uncovered questions: 65% declined honestly, 35% borrowed a nearby fact, 0% freely invented — after adding an explicit "say you do not remember" instruction. Cover what players will ask, or the gaps fill themselves.

Measured — does the state block change what the model says?

- An earlier pass reported no effect. That was a MEASUREMENT error, not a result: it counted "did the reply differ" (which scores the same with the feature off) and asked indirect questions. Counting explicit references to the state, n = 10 per condition, with a control arm for every condition:
- sleepiness at 95/100: **6/10** mentions, against **0/10** at 5/100 and **0/10** with the block disabled;
- trust at 16/20: **6/10**, against **2/10** at 0/20 and **1/10** disabled.
- Tested and REVERTED: addressing the character in second person ("you are tired") — it collapsed the trust effect to 1/10 and made tiredness score 10/10 at BOTH ends of the axis. Also reverted: moving the block after the player's line, which flattened every condition to the control.
- Tested and NOT adopted: raising the reply limit to three sentences. It strengthened the relationship effect (8/10 against a 0/10 control) but lifted the sleepiness control to 5/10, because a longer reply mentions more things by chance. Mixed, and the limit exists for latency, so the default stays at one sentence.
- Honest framing for the docs and the listing: state **colours** replies, it does not make the character act.

Added — internal state (mood, relationships, needs, traits)

- Four axis KINDS, kept apart because they live on different timescales: a mood eases back to a baseline, a need grows until something satisfies it, a relationship only moves when something happens, and a trait never moves at all. Merging any two of them produces the classic bug where either the mood stops decaying or the need starts.
- `AxisDefinition` carries range, baseline, rate per hour, hysteresis, the model's write allowance and the descriptor bands — no threshold or rate is hardcoded in logic. `AxisCatalogAsset` / `TraitPresetAsset` are the authored form, with `schemaVersion` and a migration hook from the first version.
- Relationships are keyed by PAIR, so NPC↔NPC opinions work from day one.
- Descriptors, not numbers: each band maps to a short clause per language through `LocalizedText`. A small model given `trust: 7` either ignores it or over-dramatises it. The block states only what has actually moved, ranked by how far from baseline, and is capped at two clauses.

- `StateVerbGate` filters the VERB list from state the way perception filters targets: an exhausted character is not asked to decline to look around — the verb is absent from the character's grammar. Verbs by state, targets by perception, composing without knowing about each other.
- The model's own way in is the `feel` action, whose `axis` parameter is a grammar CHOICE built from the axes a designer opened. Needs are unreachable rather than merely rejected, and open axes are magnitude-clamped per reply.
- `ThresholdCrossed` fires once per band crossing, with hysteresis, and never on load — so a milestone is celebrated the moment it happens and not again at every launch.
- Save/load through `IStateStore` with `Export()`/`Import()` for projects that own their save format; mood is deliberately session-only.
- `Log`: the plugin's single logger, with `Debug` compiled out unless `AINPC_DEBUG` and a redirectable handler for projects with their own logging.

Added — points of interest (perception with no wiring)

- `AIPointOfInterest`: drop it on an object and every NPC can notice it. It registers itself into a static `PoiRegistry` while enabled, so pooling, additive scenes and scene unloads are correct for free — there is no list to maintain and therefore none to forget. Competing products ask the developer to hand-register every object.
- `PerceptionQuery` per character: a recompute a few times a second (never per frame) scoring `proximity + priority + facing + recency of interaction` with serialized weights. Occlusion is a hard exclusion — something behind a wall is not "low scoring", it is not seen. Only the top N (default 6, hard cap 15) reach the prompt.
- Candidates come from the registry rather than `Physics.OverlapSphere`, because a sphere cast would force every point of interest to carry a collider and that would break the one-component promise.
- The perception result feeds TWO things from one snapshot: the "Nearby" line in the prompt and the target enum of the action grammar. A character is therefore unable to name an object it cannot see, and the description can never disagree with what is nameable.
- Freshness without prompt rebuilds: the persona snapshot keeps the VERBS (stable, so the prompt cache survives) while the object ids arrive per turn with the query and are never written to the transcript. An object that appeared an instant ago is nameable this turn; one that left is not.
- `LocalizedText`: verb, parameter, target and point-of-interest descriptions are now keyed by language code with fallback, so a Japanese conversation is no longer steered by English instruction fragments. A sixth language is data, not code.
- Each point of interest carries a `UnityEvent<NpcActionContext>`, so "when an NPC uses this mug, do X" is inspector wiring. The payload carries the resolved component.
- `PoiBulkAttach`: make an existing scene perceivable by layer or tag in one call.
- `PerceptionQuery.Explain` and selection gizmos for debugging by eye.

Added — structured actions via grammar-constrained decoding

- `Actions/ActionSchema`: registered verbs, their parameters and their target requirements, producing BOTH a GBNF grammar (what is *expressible*) and a prompt description (what is *appropriate*). Neither alone is enough — a grammar is invisible to the model, and a prompt is not a constraint.

- Output format: an OPTIONAL leading `{"verb":…}` line, then free prose. Leading, so the action is known before the first word is spoken; optional, so ordinary conversation stays ordinary. The grammar forbids prose from starting with `{` and forbids an action with no speech after it — the character can never act and then fall silent.
- `GbnfWriter` builds the grammar from the schema. Unregistered targets and values outside a `Choice` list are not merely discouraged, they are unreachable.
- `ActionParser` validates and resolves the block, and NEVER throws — the character is already talking by the time it runs. `ActionRepair` supplies the single corrective round, which asks for the action alone and then gives up silently.
- `ActionDispatcher` delivers a resolved action three ways: a UnityEvent for inspector wiring, a C# event / `IActionHandler` for code, and per-verb subscriptions. Every callback is contained: a handler that throws loses its action, not the conversation.
- `IGrammarSink` is the seam. The core never references LLMUnity; the shipped adapter lives in `OfflineAINPC.LLMUnity`, an assembly excluded by `defineConstraints` when LLMUnity is absent, and is found through the core interface via `GetComponent`.
- `LLMUnityPresenceDetector` sets `LLMUNITY_PRESENT` when LLMUnity is installed as loose files under `Assets/` — the Asset Store case, where `versionDefines` cannot fire. It leaves a package installation alone.
- 57 EditMode tests, including a small GBNF matcher that checks what the grammar accepts and rejects without loading a model.

Added — preset catalog

- `PresetCatalog` / `PresetCatalogJson`: a (language x tier) -> everything-needed record, read from JSON with an integer `schemaVersion`. An UNKNOWN FUTURE schemaVersion is a warning, never an error — a customer on an older plugin keeps working on the copy they have.
- Every asset in a preset carries `license`, `licenseUrl` and `attribution`. Attribution is a field the plugin can surface, because a preset that quietly obliges a buyer to display "Built with Llama" and does not say so is a licence violation waiting to happen.
- Distribution: an embedded copy ships with the plugin and works offline forever; a remote copy is fetched only by a MANUAL editor command, and nothing is ever uploaded. Any failure (offline, 404, malformed, future schema) silently keeps the last good copy.
- Merge rule: `official` entries are replaced wholesale, `user` entries are never touched, and on an id collision the user entry wins. An update cannot deliver `user` entries.
- Seed content: Qwen3 8B/4B/1.7B across the tiers for en, es, ja, zh-Hans, ru — all Apache-2.0, none gated. TTS slots are left empty with an explanation where no licence-clean voice could be confirmed.

Added — hardware tiers, auto-tuning, degradation cascade

- Four tiers (`Fallback`/`Light`/`Standard`/`Premium`) resolved from a probed `HardwareProfile` against `TierThresholds`. Tier 0 is a working state, not an error.
- `TierPolicy` and `ModelManifest` ScriptableObjects hold the data; `TierThresholds` and `ModelCatalog` are their pure-C# counterparts, which is what the logic actually reads — so every threshold and every model rule is testable without the engine.
- Warm-up benchmark with automatic GPU-layer selection, cached per GPU + model + plugin version. An unfinished attempt is recorded under its own key BEFORE the attempt, so a launch that never came back is detected on the next one and stepped down from.

- Degradation cascade: model start failures retry with fewer layers, then a lighter tier, then Tier 0. `Tiers.OnTierResolved` / `OnTierDowngraded` report every step.
- Public API: `Tiers.Current`, `Tiers.Profile`, `Tiers.Reason`, `Tiers.Verdict`, `Tiers.ActiveModel`, `Tiers.ActiveGpuLayers`, `Tiers.MeasuredTokensPerSecond`, `Tiers.Override(tier?)`, and the two events.
- `schemaVersion` + `OnAfterDeserialize` migration on both config assets from their first version, so a customer's tuned config survives a plugin update.
- Model entries carry licence id, licence URL and a required-attribution field; the default catalog is Qwen3 (Apache-2.0) because it is the cleanest to redistribute.

Added — streaming reply parser and sentence-level TTS handoff

- `Core/StreamingReplyParser` — cuts a reply into speakable units while the model is still writing it. Pure C#. Terminators cover `.!?`, their full-width twins, `。` , and the ellipsis forms; decimals, abbreviations and initials do not split; a terminator arriving in its own chunk is handled (that is what the real stream does); an ellipsis only ends a unit when a new sentence starts after it; short units merge forward; a leading `<...>` / `{...}` metadata block is held back rather than spoken.
- `Speech/Tts/StreamingSpeechSession` — synthesis and playback as two coroutines over a shared queue, so unit N is audible while unit N+1 is in the synthesizer. Ordered, never overlapping, cancellable mid-flight, and degrades to subtitle-only when the synthesizer dies rather than hanging the conversation.
- The demo's dialogue controller routes its reply through both, keeping its busy lock in a `finally` across the whole multi-sentence session. Push-to-talk may interrupt a character mid-reply.
- 34 EditMode tests for the parser, including a captured real token stream replayed verbatim.

Added — package skeleton and the assembly boundary

- `Assets/OfflineAINPC/` package layout: `Runtime/` (Core, Speech/Tts, Speech/Stt, Memory, Hardware), `Editor/`, `Tests/Editor`, `Tests/Runtime`, `Documentation/`, `LICENSES/`.
- Four assembly definitions. `OfflineAINPC.Runtime` has an **empty `references` list** — no game assembly, no LLMUnity, no whisper.unity. It briefly also carried `Newtonsoft.Json.dll` as a `precompiledReference`; that dependency was removed before release, and `precompiledReferences` is empty too. Beyond the engine the runtime takes nothing, which is what makes the decoupling a compiler-enforced fact rather than a claim.
- `ILanguageSource` + `FixedLanguageSource`: the seam through which a host tells the TTS manager the current language.
- `MicChunk`: the recorder's own PCM carrier, replacing whisper.unity's `AudioChunk` on the mic event, so the STT package is no longer a compile-time dependency of the mic.
- `ExternalProcessRegistry` + `ExternalProcessLifecycle` (+ the Editor-side hooks): one teardown point for OS child processes, wired to `beforeAssemblyReload`, `ExitingPlayMode` and `Application.quitting`. Piper's process pool registers itself.
- The hardware probe: GPU, RAM and thread count, with a pure tier resolve for testing.
- `DependencyGuard`: version-define state (`LLMUNITY_PRESENT`, `WHISPER_PRESENT`) plus `Require*` helpers that fail with a readable message naming the package and version.
- EditMode tests for conversation memory, the hardware tier table, the mic ring-buffer slice arithmetic, the process registry, and the language seam. PlayMode tests for recorder/mic-loop lifecycle and PCM↔AudioClip round-tripping. Nothing in the suite needs a model, a microphone, or a network.

Moved (behaviour unchanged)

`ITTSEngine`, `ILanguageAwareEngine`, `AudioData`, `AudioClipConversion`, `VoiceConfig` / `IPiperVoiceRouter`, `PiperProcessPool`, `PiperEngine`, `TTSEngineManager`, `SharedMicLoop`, `PersistentMicRecorder`, `NPCMemory`, `IKeyValueStore`, and the `Language` enum. Namespaces became `OfflineAINPC.*`; every file kept its `.meta`, so scene and prefab references are untouched.

Known limitations

- `LLMUNITY_PRESENT` is resolved from the package registry, so LLM for Unity installed as an Asset Store folder under `Assets/` rather than as a UPM package is not seen by `versionDefines`; the LLMUnity integration and the HelloNpc sample then do not compile in. `Check my setup` detects that case and says so.
- Delivered as a `.unitypackage` folder, not a UPM package; there is no `package.json`.

- Qwen3-TTS models and preset voices
- Piper: recorded dataset and fine-tune base
- Redistributing a voice

Voice licences and provenance

Voice models have separate terms from the plugin and speech engine. Check the model publisher's licence, required notices and model card for the version you distribute. Training-data and fine-tune-base records provide additional context; a permissive weight/dataset label does not describe every recording or recorded person's rights.

Qwen3-TTS models and preset voices

The catalog uses Qwen3-TTS CustomVoice GGUF models and a shared codec. The official checkpoints and the Serveurperso GGUF repository advertise Apache-2.0. Nine preset timbres are embedded in the model, not separate voice downloads. The technical report describes training scale, languages and stages, but not a complete recording-permission inventory for these timbres. Keep the required redistribution notices; see Licensing.

Piper: recorded dataset and fine-tune base

Archived cards are in `voice-cards-2026-07-30/` and `voice-cards-2026-09-23/`. The ljspeech-medium, dmitri and sharvard records below also use Piper voice repository revision c10ece1aade47bb51c153c893d14e5bf8e5b7117.

Voice	Recorded dataset terms / training	Redistribution notes
en_US-ljspeech-medium / high	Public-domain LJ Speech; cards record training from scratch.	Engine/runtime notices still apply.
en_US-libritts-high	CC BY 4.0 LibriTTS train-clean-360; from scratch; 904 speakers.	Retain applicable attribution and notices.
en_US-libritts_r-medium	Related LibriTTS-R corpus, CC BY 4.0; card records a lessac fine-tune base. LibriTTS-R is distinct from the original LibriTTS corpus.	Check the exact voice publisher's terms and recorded fine-tune-base terms.
ru_RU-dmitri-medium / es_ES-davefx-medium	Cards label their datasets CC0 and identify a lessac fine-tune base.	Check the exact voice publisher's terms and recorded fine-tune-base terms.
es_ES-sharvard-medium	Dataset CC BY 3.0, two speakers; card identifies a lessac fine-tune base.	Check the exact voice publisher's terms and recorded fine-tune-base terms; retain applicable attribution and notices.

Voice	Recorded dataset terms / training	Redistribution notes
en_US-amy-medium	Dataset URL leads to a repository-wide CC BY-SA label; the voice has no statement of its own; lessac base recorded.	Check the per-voice publisher's terms and recorded fine-tune-base terms.
en_US-hfc_female-medium	NonCommercial dataset terms; lessac base recorded.	Check the stated NonCommercial scope and the exact weight publisher's terms before commercial distribution.
zh_CN-chaowen-medium	Card records a Xiao Ya fine-tune base; archived Xiao Ya card names NonCommercial BZNSYP.	Check the stated NonCommercial scope and the exact weight publisher's terms before commercial distribution.

The lessac dataset licence grants research-only use and excludes commercial use of those materials. A permissive dataset used for fine-tuning does not establish permission to use the recorded base. Check the exact voice publisher's terms and recorded fine-tune-base terms before redistribution; dataset terms do not automatically determine the terms of every derived weight file.

Dataset labels also do not establish every weight publisher's terms. Preserve the exact card, model revision/hash and required credits. LJ Speech's public-domain recording claim is useful evidence; it is not a waiver of Piper/espeak/ONNX notices or a statement that the whole speech stack is obligation-free.

Redistributing a voice

For each voice you distribute, retain its model version/hash, source URL, model card and required notices. Check the engine/build, runtime libraries, preprocessing and dictionaries, and any reference recordings separately. Review the terms again when changing a model or voice.

native-build-manifest.json


```
{
  "schema": "offline-ai-qwen-native-delivery-v1",
  "library": "OfflineAINPCQwenNative",
  "platform": "Windows x64",
  "cpu_minimum": [
    "AVX2",
    "BMI2",
    "FMA",
    "F16C",
    "SSE4.2"
  ],
  "cpu_threads": 8,
  "cpu_profile": "static AVX2 (different from previously measured icelake HTTP server)",
  "flash_attention": false,
  "max_batch": 1,
  "upstream_commit": "6fae92914045cd83364d2845ceaa0f7969727319",
  "ggml_commit": "40e16e4a814f7fe851a0c486fb9e8c722e957830",
  "private_patches": [
    "backend.h fixed CPU threads 8 and no process-wide GGML_BACKEND selector"
  ],
  "public_abi": 5,
  "private_bridge_abi": 1,
  "binary": {
    "file": "OfflineAINPCQwenNative.dll",
    "bytes": 5366978,
    "sha256": "4de29ccecbb2962452c2c9f2a1a4dbcc49f1c56e50f5aad74cb25a1739d0d9f3"
  },
  "dependencies": [
    "ADVAPI32.dll",
    "KERNEL32.dll",
    "msvcrt.dll"
  ],
  "dynamic_ggml": false,
  "runtime_executables": [],
  "actual_native_model_acceptance": "unverified",
  "actual_unity_player_acceptance": "unverified",
  "licenses": [
    "qwentts MIT",
    "GGML MIT",
    "MinGW-w64 runtime notices (including winpthreads)",
    "GCC runtime GPLv3 with Runtime Library Exception 3.1"
  ],
  "notices": [
    {
      "file": "Notices/GCC-RUNTIME-EXCEPTION.txt",
```

```
      "bytes": 3324,
      "sha256": "9d6b43ce4d8de0c878bf16b54d8e7a10d9bd42b75178153e3af6a815bdc90f74"
    },
    {
      "file": "Notices/ggml-LICENSE.txt",
      "bytes": 1078,
      "sha256": "94f29bbed6a22c35b992c5c6ebf0e7c92f13b836b90f36f461c9cf2f0f1d010d"
    },
    {
      "file": "Notices/GPL-3.0.txt",
      "bytes": 35149,
      "sha256": "3972dc9744f6499f0f9b2dbf76696f2ae7ad8af9b23dde66d6af86c9dfb36986"
    },
    {
      "file": "Notices/MinGW-w64-runtime.txt",
      "bytes": 15076,
      "sha256": "ecff91ddb5799c8b79a4bdb576921b9c3e2989783fd1944527e8eb9b41a5ff5f"
    },
    {
      "file": "Notices/qwentts-LICENSE.txt",
      "bytes": 1087,
      "sha256": "cddbecd5db98ec5bd44af3e7221b6d44c3e9fed1c92940a3be5b0080c9b86475"
    }
  ]
}
```

- The current package boundary
- If distributing the Piper stack
- Voices, phoneme routes and output
- When to review again

The espeak boundary

Piper is an optional external process. Its archived engine source is MIT, while its phonemizer links GPL-3.0 espeak-ng. A top-level MIT label does not cover the whole native distribution, its dictionaries, ONNX Runtime's notices or voice models.

The current package boundary

The plugin invokes Piper over stdin/stdout JSON, does not load its libraries into Unity, and can operate with another engine or typed input. Piper, espeak, their data and voice weights are installed outside `Assets/OfflineAINPC/`. The export preflight excludes native/model payloads and espeak data from that folder. This is a source/export rule; review the actual release package and game distribution separately.

The GNU aggregation FAQ considers both the communication mechanism and its semantics. Separate processes and pipes support the documented boundary, but they are not an unconditional legal guarantee for every future integration. Distribution location alone also does not settle whether programs form one work. Keep the boundary and re-review changes that alter it.

If distributing the Piper stack

Artifact	What must be identified
Piper executable and piper-phonemize	Exact source revisions, patches, build/install scripts and dependencies.
espeak-ng library	GPL-covered source matching the actual binary, including the fork revision.
espeak-ng-data	Exact dictionary sources, generated data and build inputs; individual notices where applicable.
ONNX Runtime and other libraries	Actual versions, licence/copyright texts and ThirdPartyNotices.
Voice weights / libtashkeel model	Their independent weight, dataset and fine-tune provenance.

GPL-covered object-code distribution requires a method allowed by GPL-3.0 sections 1 and 6, licence/notices and no additional restrictions on the covered components. For section 6(d), equivalent no-charge Corresponding Source access and clear directions accompany the binary offering. Other section 6 methods have their own conditions. A

generic upstream URL or written offer is not automatic compliance.

Archived Piper's 2023.11.14-2 CMake can fetch a moving phonemizer branch. Its tag alone therefore does not identify the entire installed stack's Corresponding Source. A version string is supporting evidence, not a substitute for matching the binaries and dictionary files you ship to their source.

The current process boundary is designed to keep the plugin separate. Do not infer that every proprietary game, modified protocol or combined distribution is automatically cleared. The actual combination and redistribution conditions need their own review.

Voices, phoneme routes and output

Voice files are a separate question: the dataset licence, fine-tune base and independent weight terms need review. A Chinese pinyin route or an unused espeak backend does not remove GPL-covered files that are still present in a distributed Piper folder.

The GNU output FAQ generally distinguishes output from copyright in program code, with exceptions where protected content is copied. Independent recording, voice, model and other rights still apply. The process boundary does not establish that all generated audio owes no obligations.

When to review again

- When changing process/library integration or the communication protocol.
- When adding an engine installer, changing its destination or exporting the package.
- Before distributing an engine, native library, dictionary or model with a game.
- When selecting a different Piper fork, phonemizer, voice or preprocessing model.

Existing working Piper configurations remain supported. A new engine choice needs its own exact inventory; this document does not grant blanket clearance to Qwen3-TTS or any other alternative. See `THIRD_PARTY_NOTICES.md`.

- What comes from where
- Sources
- Model files: expected checksums
- Before you redistribute

Source pins — recorded 2 October 2026

The exact revisions this version of the plugin was written against, and the checksums the Model Manager expects of the model files it downloads. A source revision, the expected checksum of a weight file and the hash of an installed binary are three different pins: none of them stands in for the others, and none of them is a licence grant. Keep the notices of the exact artifacts you redistribute.

What comes from where

Part	Where it comes from
The plugin	Its own code, configuration and documentation under `Assets/OfflineAINPC`, the Qwen native DLL named in the notices, and text copies of the compatibility sources. No model weights and no espeak data.
LLM for Unity	Installed by you. Its editor setup downloads `LlamaLib-v2.0.5.zip` with its checksum and unpacks the runtimes into StreamingAssets.
Model files	Downloaded by the Model Manager when you ask for one and accept its licence. The catalog addresses a repository's `main` branch, not the revisions listed below.
Piper, Whisper models, your own engines	Installed by you, outside the plugin's folder.

Sources

Component	Revision
LLM for Unity v3.0.3	`9931a9ea46906b1f43adaf9498c05ccf227b5d0b`

Component	Revision
LlamaLib v2.0.5	`21d3b9dcd1c05ad13f94f198e59462960019a057`; llama.cpp `872646b30c342c5fa98efec002f94fdb4f1f1ea5` plus the repository's patches; FeatureDetector `077393260d5e0e19938582a33b9dd0e74baa8ff2`
USearch	As pinned by LLM for Unity v3.0.3. The revision of its native build is not stated by the package; a README citation of 2.8.16 is not that statement.
whisper.unity	`529a628a915a97799e89e061af9cb7c71407124d`. Its build workflow defaults to whisper.cpp v1.7.5 and allows another ref.
Piper, archived	2023.11.14-2 = `38917ffd8c0e219c6581d73e07b30ef1d572fce1`; MIT. Its build asks for fmt 10.0.0, spdlog 1.12.0 and the phonemizer's `master`.
piper-phonemize, archived snapshot	`ba3cc06c5248215928821f1393b2b854a936991a`; ONNX Runtime 1.14.1 by default; espeak-ng fork `0f65aa301e0d6bae5e172cc74197d32a6182200f`. A snapshot of `master` does not identify the build inside an older Piper release.
qwentts.cpp, the bundled DLL's source	`6fae92914045cd83364d2845ceaa0f7969727319`; ggml `40e16e4a814f7fe851a0c486fb9e8c722e957830`
qwentts.cpp, the earlier server-era source	`71ad93d591a2811f35db77e27c02acba091c9e9b`; ggml `77cdef82aad64f141164fac02076e36d5848474c`

Copyright holders of the qwentts.cpp source: omnivoice.cpp authors (2023–2026), ggml authors (2023–2026), cpp-httplib yhirose (2017), yyjson YaoYuan (2020) — each MIT. That covers the source, not runtime DLLs beside it and not model or data rights. LlamaLib's FeatureDetector is CC0-1.0; its other dependencies carry their own notices.

The CUDA and cuBLAS libraries a LlamaLib installation can contain are under NVIDIA's terms; the CUDA 12.4.1 EULA is the relevant primary reference. Check the version you actually redistribute.

Model files: expected checksums

The checksums the repositories publish in their metadata. The plugin's catalog carries the same eleven, and the Model Manager compares a finished download against its own.

File	Repository revision	SHA256
Qwen3-1.7B-Q8_0.gguf	90862c4b9d2787eaed51d12237eafdfe7c5f6077	061b54daade076b5d3362dac252678d17da8c68f07560be70818cace6590cb1a
Qwen3-4B-Q4_K_M.gguf	bc640142c66e1fdd12af0bd68f40445458f3869b	7485fe6f11af29433bc51cab58009521f205840f5b4ae3a32fa7f92e8534fdf5
Qwen3-8B-Q4_K_M.gguf	7c41481f57cb95916b40956ab2f0b139b296d974	d98cdcbd03e17ce47681435b5150e34c1417f50b5c0019dd560e4882c5745785
qwen-talker-0.6b-customvoice-Q4_K_M.gguf	b7ee2e8c7459c3bea99da23e3d178125a7d1713c	b3a7e6613d80f8a703c06267fc1e94d48ce91932ab82ab6e31c50f4ca4868e1e
qwen-talker-0.6b-customvoice-Q8_0.gguf	same GGUF repository	4eb38675c736ed6ac72012846ac8d6ef80e5af8bc05726870f0b3a6569588519
qwen-talker-1.7b-customvoice-Q4_K_M.gguf	same GGUF repository	cc328834a631bc08bf9f43e62fa23f8a1383d9b429864ce6690cfb172077fc4a
qwen-tokenizer-12hz-Q8_0.gguf	same GGUF repository	1883beeed99348fc35e23dd225e9082f93f6f8c109330a33d935baa8acdbfd94
ggml-tiny.en.bin	5359861c739e955e79d9a303bcbc70fb988958b1	921e4cf8686fdd993dcd081a5da5b6c365bfde1162e72b08d75ac75289920b1f
ggml-base.en.bin	same Whisper repository	a03779c86df3323075f5e796cb2ce5029f00ec8869eee3fdfb897afe36c6d002

File	Repository revision	SHA256
ggml-base.bin	same Whisper repository	60ed5bc3dd14eea856493d334349b405782ddcaf0028d4b5df4088345fba2efe
ggml-small.bin	same Whisper repository	1be3a9b2063867b937e64e2ec7483364a79917e157fa98c5d94b5c1fffea987b

Source repositories: Qwen 1.7B, 4B, 8B, TTS GGUF, Whisper GGUF/ggml.

The official Qwen3-TTS 0.6B and 1.7B CustomVoice repositories, at revisions `85e237c12c027371202489a0ec509ded67b5e4b5` and `0c0e3051f131929182e2c023b9537f8b1c68adfe`, declare Apache-2.0. Keep the notices of both the original and the conversion repository.

Before you redistribute

- Match the native libraries you ship to their source, patches and toolchain, and keep their full licence, copyright and NOTICE material with your distribution.
- Piper links espeak-ng, which is GPL-3.0: shipping Piper means shipping the Corresponding Source of the exact build you ship, dictionaries included. See the boundary.
- The phonemizer's `libtashkeel_model.ort` states no licence for its weights or data.

GPL-3.0.txt

```
                    GNU GENERAL PUBLIC LICENSE
                       Version 3, 29 June 2007

 Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
 Everyone is permitted to copy and distribute verbatim copies
 of this license document, but changing it is not allowed.

                            Preamble

  The GNU General Public License is a free, copyleft license for
software and other kinds of works.

  The licenses for most software and other practical works are designed
to take away your freedom to share and change the works.  By contrast,
the GNU General Public License is intended to guarantee your freedom to
share and change all versions of a program--to make sure it remains free
software for all its users.  We, the Free Software Foundation, use the
GNU General Public License for most of our software; it applies also to
any other work released this way by its authors.  You can apply it to
your programs, too.

  When we speak of free software, we are referring to freedom, not
price.  Our General Public Licenses are designed to make sure that you
have the freedom to distribute copies of free software (and charge for
them if you wish), that you receive source code or can get it if you
want it, that you can change the software or use pieces of it in new
free programs, and that you know you can do these things.

  To protect your rights, we need to prevent others from denying you
these rights or asking you to surrender the rights.  Therefore, you have
certain responsibilities if you distribute copies of the software, or if
you modify it: responsibilities to respect the freedom of others.

  For example, if you distribute copies of such a program, whether
gratis or for a fee, you must pass on to the recipients the same
freedoms that you received.  You must make sure that they, too, receive
or can get the source code.  And you must show them these terms so they
know their rights.

  Developers that use the GNU GPL protect your rights with two steps:
(1) assert copyright on the software, and (2) offer you this License
giving you legal permission to copy, distribute and/or modify it.

  For the developers' and authors' protection, the GPL clearly explains
that there is no warranty for this free software.  For both users' and
```

authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying,

distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

  The Corresponding Source need not include anything that users
can regenerate automatically from other parts of the Corresponding
Source.

  The Corresponding Source for a work in source code form is that
same work.

  2. Basic Permissions.

  All rights granted under this License are granted for the term of
copyright on the Program, and are irrevocable provided the stated
conditions are met. This License explicitly affirms your unlimited
permission to run the unmodified Program. The output from running a
covered work is covered by this License only if the output, given its
content, constitutes a covered work. This License acknowledges your
rights of fair use or other equivalent, as provided by copyright law.

  You may make, run and propagate covered works that you do not
convey, without conditions so long as your license otherwise remains
in force. You may convey covered works to others for the sole purpose
of having them make modifications exclusively for you, or provide you
with facilities for running those works, provided that you comply with
the terms of this License in conveying all material for which you do
not control copyright. Those thus making or running the covered works
for you must do so exclusively on your behalf, under your direction
and control, on terms that prohibit them from making any copies of
your copyrighted material outside their relationship with you.

  Conveying under any other circumstances is permitted solely under
the conditions stated below. Sublicensing is not allowed; section 10
makes it unnecessary.

  3. Protecting Users' Legal Rights From Anti-Circumvention Law.

  No covered work shall be deemed part of an effective technological
measure under any applicable law fulfilling obligations under article
11 of the WIPO copyright treaty adopted on 20 December 1996, or
similar laws prohibiting or restricting circumvention of such
measures.

  When you convey a covered work, you waive any legal power to forbid
circumvention of technological measures to the extent such circumvention
is effected by exercising rights under this License with respect to
the covered work, and you disclaim any intention to limit operation or
modification of the work as a means of enforcing, against the work's
users, your or third parties' legal rights to forbid circumvention of
technological measures.

  4. Conveying Verbatim Copies.

  You may convey verbatim copies of the Program's source code as you
receive it, in any medium, provided that you conspicuously and
appropriately publish on each copy an appropriate copyright notice;
keep intact all notices stating that this License and any
non-permissive terms added in accord with section 7 apply to the code;
keep intact all notices of the absence of any warranty; and give all
recipients a copy of this License along with the Program.

  You may charge any price or no price for each copy that you convey,
and you may offer support or warranty protection for a fee.

  5. Conveying Modified Source Versions.

  You may convey a work based on the Program, or the modifications to
produce it from the Program, in the form of source code under the
terms of section 4, provided that you also meet all of these conditions:

    a) The work must carry prominent notices stating that you modified
    it, and giving a relevant date.

    b) The work must carry prominent notices stating that it is
    released under this License and any conditions added under section
    7. This requirement modifies the requirement in section 4 to
    "keep intact all notices".

    c) You must license the entire work, as a whole, under this
    License to anyone who comes into possession of a copy. This
    License will therefore apply, along with any applicable section 7
    additional terms, to the whole of the work, and all its parts,
    regardless of how they are packaged. This License gives no
    permission to license the work in any other way, but it does not
    invalidate such permission if you have separately received it.

    d) If the work has interactive user interfaces, each must display
    Appropriate Legal Notices; however, if the Program has interactive
    interfaces that do not display Appropriate Legal Notices, your
    work need not make them do so.

  A compilation of a covered work with other separate and independent
works, which are not by their nature extensions of the covered work,
and which are not combined with it such as to form a larger program,
in or on a volume of a storage or distribution medium, is called an
"aggregate" if the compilation and its resulting copyright are not
used to limit the access or legal rights of the compilation's users
beyond what the individual works permit. Inclusion of a covered work
in an aggregate does not cause this License to apply to the other
parts of the aggregate.

  6. Conveying Non-Source Forms.

  You may convey a covered work in object code form under the terms
of sections 4 and 5, provided that you also convey the
machine-readable Corresponding Source under the terms of this License,
in one of these ways:

    a) Convey the object code in, or embodied in, a physical product
    (including a physical distribution medium), accompanied by the
    Corresponding Source fixed on a durable physical medium
    customarily used for software interchange.

    b) Convey the object code in, or embodied in, a physical product
    (including a physical distribution medium), accompanied by a
    written offer, valid for at least three years and valid for as
    long as you offer spare parts or customer support for that product
    model, to give anyone who possesses the object code either (1) a
    copy of the Corresponding Source for all the software in the
    product that is covered by this License, on a durable physical
    medium customarily used for software interchange, for a price no
    more than your reasonable cost of physically performing this
    conveying of source, or (2) access to copy the
    Corresponding Source from a network server at no charge.

    c) Convey individual copies of the object code with a copy of the
    written offer to provide the Corresponding Source. This
    alternative is allowed only occasionally and noncommercially, and
    only if you received the object code with such an offer, in accord
    with subsection 6b.

    d) Convey the object code by offering access from a designated
    place (gratis or for a charge), and offer equivalent access to the
    Corresponding Source in the same way through the same place at no
    further charge. You need not require recipients to copy the
    Corresponding Source along with the object code. If the place to
    copy the object code is a network server, the Corresponding Source
    may be on a different server (operated by you or a third party)
    that supports equivalent copying facilities, provided you maintain
    clear directions next to the object code saying where to find the
    Corresponding Source. Regardless of what server hosts the
    Corresponding Source, you remain obligated to ensure that it is
    available for as long as needed to satisfy these requirements.

    e) Convey the object code using peer-to-peer transmission, provided
    you inform other peers where the object code and Corresponding
    Source of the work are being offered to the general public at no
    charge under subsection 6d.

  A separable portion of the object code, whose source code is excluded
from the Corresponding Source as a System Library, need not be
included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this

License by making exceptions from one or more of its conditions.
Additional permissions that are applicable to the entire Program shall
be treated as though they were included in this License, to the extent
that they are valid under applicable law. If additional permissions
apply only to part of the Program, that part may be used separately
under those permissions, but the entire Program remains governed by
this License without regard to the additional permissions.

  When you convey a copy of a covered work, you may at your option
remove any additional permissions from that copy, or from any part of
it. (Additional permissions may be written to require their own
removal in certain cases when you modify the work.) You may place
additional permissions on material, added by you to a covered work,
for which you have or can give appropriate copyright permission.

  Notwithstanding any other provision of this License, for material you
add to a covered work, you may (if authorized by the copyright holders of
that material) supplement the terms of this License with terms:

    a) Disclaiming warranty or limiting liability differently from the
    terms of sections 15 and 16 of this License; or

    b) Requiring preservation of specified reasonable legal notices or
    author attributions in that material or in the Appropriate Legal
    Notices displayed by works containing it; or

    c) Prohibiting misrepresentation of the origin of that material, or
    requiring that modified versions of such material be marked in
    reasonable ways as different from the original version; or

    d) Limiting the use for publicity purposes of names of licensors or
    authors of the material; or

    e) Declining to grant rights under trademark law for use of some
    trade names, trademarks, or service marks; or

    f) Requiring indemnification of licensors and authors of that
    material by anyone who conveys the material (or modified versions of
    it) with contractual assumptions of liability to the recipient, for
    any liability that these contractual assumptions directly impose on
    those licensors and authors.

  All other non-permissive additional terms are considered "further
restrictions" within the meaning of section 10. If the Program as you
received it, or any part of it, contains a notice stating that it is
governed by this License along with a term that is a further
restriction, you may remove that term. If a license document contains
a further restriction but permits relicensing or conveying under this
License, you may add to a covered work material governed by the terms
of that license document, provided that the further restriction does

not survive such relicensing or conveying.

  If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

  Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

  8. Termination.

  You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

  However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

  Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

  Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

  9. Acceptance Not Required for Having Copies.

  You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

  10. Automatic Licensing of Downstream Recipients.

  Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

  An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

  You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

  11. Patents.

  A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

  A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

  Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

  In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a

party means to make such an agreement or commitment not to enforce a patent against the party.

  If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

  If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

  A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

  Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

  12. No Surrender of Others' Freedom.

  If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this

License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

  13. Use with the GNU Affero General Public License.

  Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

  14. Revised Versions of this License.

  The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

  Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

  If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

  Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

  15. Disclaimer of Warranty.

  THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR

PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM
IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF
ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

  16. Limitation of Liability.

  IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING
WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS
THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY
GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE
USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF
DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD
PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS),
EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF
SUCH DAMAGES.

  17. Interpretation of Sections 15 and 16.

  If the disclaimer of warranty and limitation of liability provided
above cannot be given local legal effect according to their terms,
reviewing courts shall apply local law that most closely approximates
an absolute waiver of all civil liability in connection with the
Program, unless a warranty or assumption of liability accompanies a
copy of the Program in return for a fee.

                     END OF TERMS AND CONDITIONS

            How to Apply These Terms to Your New Programs

  If you develop a new program, and you want it to be of the greatest
possible use to the public, the best way to achieve this is to make it
free software which everyone can redistribute and change under these terms.

  To do so, attach the following notices to the program. It is safest
to attach them to the start of each source file to most effectively
state the exclusion of warranty; and each file should have at least
the "copyright" line and a pointer to where the full notice is found.

    <one line to give the program's name and a brief idea of what it does.>
    Copyright (C) <year> <name of author>

    This program is free software: you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
    GNU General Public License for more details.

```
    You should have received a copy of the GNU General Public License
    along with this program.  If not, see <https://www.gnu.org/licenses/>.

Also add information on how to contact you by electronic and paper mail.

  If the program does terminal interaction, make it output a short
notice like this when it starts in an interactive mode:

    <program>  Copyright (C) <year>  <name of author>
    This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
    This is free software, and you are welcome to redistribute it
    under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate
parts of the General Public License.  Of course, your program's commands
might be different; for a GUI interface, you would use an "about box".

  You should also get your employer (if you work as a programmer) or school,
if any, to sign a "copyright disclaimer" for the program, if necessary.
For more information on this, and how to apply and follow the GNU GPL, see
<https://www.gnu.org/licenses/>.

  The GNU General Public License does not permit incorporating your program
into proprietary programs.  If your program is a subroutine library, you
may consider it more useful to permit linking proprietary applications with
the library.  If this is what you want to do, use the GNU Lesser General
Public License instead of this License.  But first, please read
<https://www.gnu.org/licenses/why-not-lgpl.html>.
```

GCC-RUNTIME-EXCEPTION.txt

```
GCC RUNTIME LIBRARY EXCEPTION

Version 3.1, 31 March 2009

Copyright (C) 2009 Free Software Foundation, Inc. <http://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this
license document, but changing it is not allowed.

This GCC Runtime Library Exception ("Exception") is an additional
permission under section 7 of the GNU General Public License, version
3 ("GPLv3"). It applies to a given file (the "Runtime Library") that
bears a notice placed by the copyright holder of the file stating that
the file is governed by GPLv3 along with this Exception.

When you use GCC to compile a program, GCC may combine portions of
certain GCC header files and runtime libraries with the compiled
program. The purpose of this Exception is to allow compilation of
non-GPL (including proprietary) programs to use, in this way, the
header files and runtime libraries covered by this Exception.

0. Definitions.

A file is an "Independent Module" if it either requires the Runtime
Library for execution after a Compilation Process, or makes use of an
interface provided by the Runtime Library, but is not otherwise based
on the Runtime Library.

"GCC" means a version of the GNU Compiler Collection, with or without
modifications, governed by version 3 (or a specified later version) of
the GNU General Public License (GPL) with the option of using any
subsequent versions published by the FSF.

"GPL-compatible Software" is software whose conditions of propagation,
modification and use would permit combination with GCC in accord with
the license of GCC.

"Target Code" refers to output from any compiler for a real or virtual
target processor architecture, in executable form or suitable for
input to an assembler, loader, linker and/or execution
phase. Notwithstanding that, Target Code does not include data in any
format that is used as a compiler intermediate representation, or used
for producing a compiler intermediate representation.

The "Compilation Process" transforms code entirely represented in
```

non-intermediate languages designed for human-written code, and/or in Java Virtual Machine byte code, into Target Code. Thus, for example, use of source code generators and preprocessors need not be considered part of the Compilation Process, since the Compilation Process can be understood as starting with the output of the generators or preprocessors.

A Compilation Process is "Eligible" if it is done using GCC, alone or with other GPL-compatible software, or if it is done without using any work based on GCC. For example, using non-GPL-compatible Software to optimize any GCC intermediate representations would not qualify as an Eligible Compilation Process.

1. Grant of Additional Permission.

You have permission to propagate a work of Target Code formed by combining the Runtime Library with Independent Modules, even if such propagation would otherwise violate the terms of GPLv3, provided that all Target Code was generated by Eligible Compilation Processes. You may then convey such a combination under terms of your choice, consistent with the licensing of the Independent Modules.

2. No Weakening of GCC Copyleft.

The availability of this Exception does not imply any general presumption that third-party software is unaffected by the copyleft requirements of the license of GCC.

ggml-LICENSE.txt

```
MIT License

Copyright (c) 2023-2026 The ggml authors

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

GPL-3.0.txt

```
                    GNU GENERAL PUBLIC LICENSE
                       Version 3, 29 June 2007

 Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
 Everyone is permitted to copy and distribute verbatim copies
 of this license document, but changing it is not allowed.

                            Preamble

  The GNU General Public License is a free, copyleft license for
software and other kinds of works.

  The licenses for most software and other practical works are designed
to take away your freedom to share and change the works.  By contrast,
the GNU General Public License is intended to guarantee your freedom to
share and change all versions of a program--to make sure it remains free
software for all its users.  We, the Free Software Foundation, use the
GNU General Public License for most of our software; it applies also to
any other work released this way by its authors.  You can apply it to
your programs, too.

  When we speak of free software, we are referring to freedom, not
price.  Our General Public Licenses are designed to make sure that you
have the freedom to distribute copies of free software (and charge for
them if you wish), that you receive source code or can get it if you
want it, that you can change the software or use pieces of it in new
free programs, and that you know you can do these things.

  To protect your rights, we need to prevent others from denying you
these rights or asking you to surrender the rights.  Therefore, you have
certain responsibilities if you distribute copies of the software, or if
you modify it: responsibilities to respect the freedom of others.

  For example, if you distribute copies of such a program, whether
gratis or for a fee, you must pass on to the recipients the same
freedoms that you received.  You must make sure that they, too, receive
or can get the source code.  And you must show them these terms so they
know their rights.

  Developers that use the GNU GPL protect your rights with two steps:
(1) assert copyright on the software, and (2) offer you this License
giving you legal permission to copy, distribute and/or modify it.

  For the developers' and authors' protection, the GPL clearly explains
that there is no warranty for this free software.  For both users' and
```

authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying,

distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

  The Corresponding Source need not include anything that users
can regenerate automatically from other parts of the Corresponding
Source.

  The Corresponding Source for a work in source code form is that
same work.

  2. Basic Permissions.

  All rights granted under this License are granted for the term of
copyright on the Program, and are irrevocable provided the stated
conditions are met. This License explicitly affirms your unlimited
permission to run the unmodified Program. The output from running a
covered work is covered by this License only if the output, given its
content, constitutes a covered work. This License acknowledges your
rights of fair use or other equivalent, as provided by copyright law.

  You may make, run and propagate covered works that you do not
convey, without conditions so long as your license otherwise remains
in force. You may convey covered works to others for the sole purpose
of having them make modifications exclusively for you, or provide you
with facilities for running those works, provided that you comply with
the terms of this License in conveying all material for which you do
not control copyright. Those thus making or running the covered works
for you must do so exclusively on your behalf, under your direction
and control, on terms that prohibit them from making any copies of
your copyrighted material outside their relationship with you.

  Conveying under any other circumstances is permitted solely under
the conditions stated below. Sublicensing is not allowed; section 10
makes it unnecessary.

  3. Protecting Users' Legal Rights From Anti-Circumvention Law.

  No covered work shall be deemed part of an effective technological
measure under any applicable law fulfilling obligations under article
11 of the WIPO copyright treaty adopted on 20 December 1996, or
similar laws prohibiting or restricting circumvention of such
measures.

  When you convey a covered work, you waive any legal power to forbid
circumvention of technological measures to the extent such circumvention
is effected by exercising rights under this License with respect to
the covered work, and you disclaim any intention to limit operation or
modification of the work as a means of enforcing, against the work's
users, your or third parties' legal rights to forbid circumvention of
technological measures.

  4. Conveying Verbatim Copies.

  You may convey verbatim copies of the Program's source code as you
receive it, in any medium, provided that you conspicuously and
appropriately publish on each copy an appropriate copyright notice;
keep intact all notices stating that this License and any
non-permissive terms added in accord with section 7 apply to the code;
keep intact all notices of the absence of any warranty; and give all
recipients a copy of this License along with the Program.

  You may charge any price or no price for each copy that you convey,
and you may offer support or warranty protection for a fee.

  5. Conveying Modified Source Versions.

  You may convey a work based on the Program, or the modifications to
produce it from the Program, in the form of source code under the
terms of section 4, provided that you also meet all of these conditions:

    a) The work must carry prominent notices stating that you modified
    it, and giving a relevant date.

    b) The work must carry prominent notices stating that it is
    released under this License and any conditions added under section
    7. This requirement modifies the requirement in section 4 to
    "keep intact all notices".

    c) You must license the entire work, as a whole, under this
    License to anyone who comes into possession of a copy. This
    License will therefore apply, along with any applicable section 7
    additional terms, to the whole of the work, and all its parts,
    regardless of how they are packaged. This License gives no
    permission to license the work in any other way, but it does not
    invalidate such permission if you have separately received it.

    d) If the work has interactive user interfaces, each must display
    Appropriate Legal Notices; however, if the Program has interactive
    interfaces that do not display Appropriate Legal Notices, your
    work need not make them do so.

  A compilation of a covered work with other separate and independent
works, which are not by their nature extensions of the covered work,
and which are not combined with it such as to form a larger program,
in or on a volume of a storage or distribution medium, is called an
"aggregate" if the compilation and its resulting copyright are not
used to limit the access or legal rights of the compilation's users
beyond what the individual works permit. Inclusion of a covered work
in an aggregate does not cause this License to apply to the other
parts of the aggregate.

  6. Conveying Non-Source Forms.

  You may convey a covered work in object code form under the terms
of sections 4 and 5, provided that you also convey the
machine-readable Corresponding Source under the terms of this License,
in one of these ways:

    a) Convey the object code in, or embodied in, a physical product
    (including a physical distribution medium), accompanied by the
    Corresponding Source fixed on a durable physical medium
    customarily used for software interchange.

    b) Convey the object code in, or embodied in, a physical product
    (including a physical distribution medium), accompanied by a
    written offer, valid for at least three years and valid for as
    long as you offer spare parts or customer support for that product
    model, to give anyone who possesses the object code either (1) a
    copy of the Corresponding Source for all the software in the
    product that is covered by this License, on a durable physical
    medium customarily used for software interchange, for a price no
    more than your reasonable cost of physically performing this
    conveying of source, or (2) access to copy the
    Corresponding Source from a network server at no charge.

    c) Convey individual copies of the object code with a copy of the
    written offer to provide the Corresponding Source. This
    alternative is allowed only occasionally and noncommercially, and
    only if you received the object code with such an offer, in accord
    with subsection 6b.

    d) Convey the object code by offering access from a designated
    place (gratis or for a charge), and offer equivalent access to the
    Corresponding Source in the same way through the same place at no
    further charge. You need not require recipients to copy the
    Corresponding Source along with the object code. If the place to
    copy the object code is a network server, the Corresponding Source
    may be on a different server (operated by you or a third party)
    that supports equivalent copying facilities, provided you maintain
    clear directions next to the object code saying where to find the
    Corresponding Source. Regardless of what server hosts the
    Corresponding Source, you remain obligated to ensure that it is
    available for as long as needed to satisfy these requirements.

    e) Convey the object code using peer-to-peer transmission, provided
    you inform other peers where the object code and Corresponding
    Source of the work are being offered to the general public at no
    charge under subsection 6d.

  A separable portion of the object code, whose source code is excluded
from the Corresponding Source as a System Library, need not be
included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this

License by making exceptions from one or more of its conditions.
Additional permissions that are applicable to the entire Program shall
be treated as though they were included in this License, to the extent
that they are valid under applicable law. If additional permissions
apply only to part of the Program, that part may be used separately
under those permissions, but the entire Program remains governed by
this License without regard to the additional permissions.

  When you convey a copy of a covered work, you may at your option
remove any additional permissions from that copy, or from any part of
it. (Additional permissions may be written to require their own
removal in certain cases when you modify the work.) You may place
additional permissions on material, added by you to a covered work,
for which you have or can give appropriate copyright permission.

  Notwithstanding any other provision of this License, for material you
add to a covered work, you may (if authorized by the copyright holders of
that material) supplement the terms of this License with terms:

    a) Disclaiming warranty or limiting liability differently from the
    terms of sections 15 and 16 of this License; or

    b) Requiring preservation of specified reasonable legal notices or
    author attributions in that material or in the Appropriate Legal
    Notices displayed by works containing it; or

    c) Prohibiting misrepresentation of the origin of that material, or
    requiring that modified versions of such material be marked in
    reasonable ways as different from the original version; or

    d) Limiting the use for publicity purposes of names of licensors or
    authors of the material; or

    e) Declining to grant rights under trademark law for use of some
    trade names, trademarks, or service marks; or

    f) Requiring indemnification of licensors and authors of that
    material by anyone who conveys the material (or modified versions of
    it) with contractual assumptions of liability to the recipient, for
    any liability that these contractual assumptions directly impose on
    those licensors and authors.

  All other non-permissive additional terms are considered "further
restrictions" within the meaning of section 10. If the Program as you
received it, or any part of it, contains a notice stating that it is
governed by this License along with a term that is a further
restriction, you may remove that term. If a license document contains
a further restriction but permits relicensing or conveying under this
License, you may add to a covered work material governed by the terms
of that license document, provided that the further restriction does

not survive such relicensing or conveying.

  If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

  Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

  8. Termination.

  You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

  However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

  Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

  Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

  9. Acceptance Not Required for Having Copies.

  You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

  10. Automatic Licensing of Downstream Recipients.

  Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

  An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

  You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

  11. Patents.

  A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

  A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

  Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

  In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a

party means to make such an agreement or commitment not to enforce a patent against the party.

  If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

  If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

  A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

  Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

  12. No Surrender of Others' Freedom.

  If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this

License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

  13. Use with the GNU Affero General Public License.

  Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

  14. Revised Versions of this License.

  The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

  Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

  If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

  Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

  15. Disclaimer of Warranty.

  THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR

PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM
IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF
ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

  16. Limitation of Liability.

  IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING
WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS
THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY
GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE
USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF
DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD
PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS),
EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF
SUCH DAMAGES.

  17. Interpretation of Sections 15 and 16.

  If the disclaimer of warranty and limitation of liability provided
above cannot be given local legal effect according to their terms,
reviewing courts shall apply local law that most closely approximates
an absolute waiver of all civil liability in connection with the
Program, unless a warranty or assumption of liability accompanies a
copy of the Program in return for a fee.

                     END OF TERMS AND CONDITIONS

            How to Apply These Terms to Your New Programs

  If you develop a new program, and you want it to be of the greatest
possible use to the public, the best way to achieve this is to make it
free software which everyone can redistribute and change under these terms.

  To do so, attach the following notices to the program. It is safest
to attach them to the start of each source file to most effectively
state the exclusion of warranty; and each file should have at least
the "copyright" line and a pointer to where the full notice is found.

    <one line to give the program's name and a brief idea of what it does.>
    Copyright (C) <year> <name of author>

    This program is free software: you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
    GNU General Public License for more details.

```
    You should have received a copy of the GNU General Public License
    along with this program.  If not, see <https://www.gnu.org/licenses/>.

Also add information on how to contact you by electronic and paper mail.

  If the program does terminal interaction, make it output a short
notice like this when it starts in an interactive mode:

    <program>  Copyright (C) <year>  <name of author>
    This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
    This is free software, and you are welcome to redistribute it
    under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate
parts of the General Public License.  Of course, your program's commands
might be different; for a GUI interface, you would use an "about box".

  You should also get your employer (if you work as a programmer) or school,
if any, to sign a "copyright disclaimer" for the program, if necessary.
For more information on this, and how to apply and follow the GNU GPL, see
<https://www.gnu.org/licenses/>.

  The GNU General Public License does not permit incorporating your program
into proprietary programs.  If your program is a subroutine library, you
may consider it more useful to permit linking proprietary applications with
the library.  If this is what you want to do, use the GNU Lesser General
Public License instead of this License.  But first, please read
<https://www.gnu.org/licenses/why-not-lgpl.html>.
```

MinGW-w64-runtime.txt

```
MinGW-w64 runtime licensing
***************************

This program or library was built using MinGW-w64 and statically
linked against the MinGW-w64 runtime. Some parts of the runtime
are under licenses which require that the copyright and license
notices are included when distributing the code in binary form.
These notices are listed below.


========================
Overall copyright notice
========================

Copyright (c) 2009, 2010, 2011, 2012, 2013 by the mingw-w64 project

This license has been certified as open source. It has also been designated
as GPL compatible by the Free Software Foundation (FSF).

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

   1. Redistributions in source code must retain the accompanying copyright
      notice, this list of conditions, and the following disclaimer.
   2. Redistributions in binary form must reproduce the accompanying
      copyright notice, this list of conditions, and the following disclaimer
      in the documentation and/or other materials provided with the
      distribution.
   3. Names of the copyright holders must not be used to endorse or promote
      products derived from this software without prior written permission
      from the copyright holders.
   4. The right to distribute this software or to use it for any purpose does
      not give you the right to use Servicemarks (sm) or Trademarks (tm) of
      the copyright holders.  Use of them is covered by separate agreement
      with the copyright holders.
   5. If any files are modified, you must cause the modified files to carry
      prominent notices stating that you changed the files and the date of
      any change.

Disclaimer

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS ``AS IS'' AND ANY EXPRESSED
OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES
OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO
EVENT SHALL THE COPYRIGHT HOLDERS BE LIABLE FOR ANY DIRECT, INDIRECT,
```

```
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA,
OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE,
EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

========================================
getopt, getopt_long, and getop_long_only
========================================

Copyright (c) 2002 Todd C. Miller <Todd.Miller@courtesan.com>

Permission to use, copy, modify, and distribute this software for any
purpose with or without fee is hereby granted, provided that the above
copyright notice and this permission notice appear in all copies.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES
WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF
MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR
ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN
ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF
OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.

Sponsored in part by the Defense Advanced Research Projects
Agency (DARPA) and Air Force Research Laboratory, Air Force
Materiel Command, USAF, under agreement number F39502-99-1-0512.

        *       *       *       *       *       *       *

Copyright (c) 2000 The NetBSD Foundation, Inc.
All rights reserved.

This code is derived from software contributed to The NetBSD Foundation
by Dieter Baron and Thomas Klausner.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:
 1. Redistributions of source code must retain the above copyright
    notice, this list of conditions and the following disclaimer.
 2. Redistributions in binary form must reproduce the above copyright
    notice, this list of conditions and the following disclaimer in the
    documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE NETBSD FOUNDATION, INC. AND CONTRIBUTORS
``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED
TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE ARE DISCLAIMED.  IN NO EVENT SHALL THE FOUNDATION OR CONTRIBUTORS
```

BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
POSSIBILITY OF SUCH DAMAGE.

---

gdtoa: Converting between IEEE floating point numbers and ASCII

The author of this software is David M. Gay.

Copyright (C) 1997, 1998, 1999, 2000, 2001 by Lucent Technologies
All Rights Reserved

Permission to use, copy, modify, and distribute this software and
its documentation for any purpose and without fee is hereby
granted, provided that the above copyright notice appear in all
copies and that both that the copyright notice and this
permission notice and warranty disclaimer appear in supporting
documentation, and that the name of Lucent or any of its entities
not be used in advertising or publicity pertaining to
distribution of the software without specific, written prior
permission.

LUCENT DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE,
INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS.
IN NO EVENT SHALL LUCENT OR ANY OF ITS ENTITIES BE LIABLE FOR ANY
SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER
IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION,
ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF
THIS SOFTWARE.

* * * * * * *

The author of this software is David M. Gay.

Copyright (C) 2005 by David M. Gay
All Rights Reserved

Permission to use, copy, modify, and distribute this software and its
documentation for any purpose and without fee is hereby granted,
provided that the above copyright notice appear in all copies and that
both that the copyright notice and this permission notice and warranty
disclaimer appear in supporting documentation, and that the name of
the author or any of his current or former employers not be used in

```
advertising or publicity pertaining to distribution of the software
without specific, written prior permission.

THE AUTHOR DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE,
INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS.  IN
NO EVENT SHALL THE AUTHOR OR ANY OF HIS CURRENT OR FORMER EMPLOYERS BE
LIABLE FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY
DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION,
ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS
SOFTWARE.

        *       *       *       *       *       *       *

The author of this software is David M. Gay.

Copyright (C) 2004 by David M. Gay.
All Rights Reserved
Based on material in the rest of /netlib/fp/gdota.tar.gz,
which is copyright (C) 1998, 2000 by Lucent Technologies.

Permission to use, copy, modify, and distribute this software and
its documentation for any purpose and without fee is hereby
granted, provided that the above copyright notice appear in all
copies and that both that the copyright notice and this
permission notice and warranty disclaimer appear in supporting
documentation, and that the name of Lucent or any of its entities
not be used in advertising or publicity pertaining to
distribution of the software without specific, written prior
permission.

LUCENT DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE,
INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS.
IN NO EVENT SHALL LUCENT OR ANY OF ITS ENTITIES BE LIABLE FOR ANY
SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER
IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION,
ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF
THIS SOFTWARE.


========================
Parts of the math library
========================

Copyright (C) 1993 by Sun Microsystems, Inc. All rights reserved.

Developed at SunSoft, a Sun Microsystems, Inc. business.
Permission to use, copy, modify, and distribute this
software is freely granted, provided that this notice
```

```
is preserved.

        *       *       *       *       *       *       *

Copyright (C) 1993 by Sun Microsystems, Inc. All rights reserved.

Developed at SunPro, a Sun Microsystems, Inc. business.
Permission to use, copy, modify, and distribute this
software is freely granted, provided that this notice
is preserved.

        *       *       *       *       *       *       *

FIXME: Cephes math lib
Copyright (C) 1984-1998 Stephen L. Moshier

It sounds vague, but as to be found at
<http://lists.debian.org/debian-legal/2004/12/msg00295.html>, it gives an
impression that the author could be willing to give an explicit
permission to distribute those files e.g. under a BSD style license. So
probably there is no problem here, although it could be good to get a
permission from the author and then add a license into the Cephes files
in MinGW runtime. At least on follow-up it is marked that debian sees the
version a-like BSD one. As MinGW.org (where those cephes parts are coming
from) distributes them now over 6 years, it should be fine.

================================================
Some string, memory and time conversion functions
================================================

Copyright © 2005-2020 Rich Felker, et al.

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

```
==================================
Headers and IDLs imported from Wine
==================================

Some header and IDL files were imported from the Wine project. These files
are prominent maked in source. Their copyright belongs to contributors and
they are distributed under LGPL license.

Disclaimer

This library is free software; you can redistribute it and/or
modify it under the terms of the GNU Lesser General Public
License as published by the Free Software Foundation; either
version 2.1 of the License, or (at your option) any later version.

This library is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the GNU
Lesser General Public License for more details.

===========
winpthreads
===========

Copyright (c) 2011 mingw-w64 project

Permission is hereby granted, free of charge, to any person obtaining a
copy of this software and associated documentation files (the "Software"),
to deal in the Software without restriction, including without limitation
the rights to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell copies of the Software, and to permit persons to whom the
Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN THE SOFTWARE.


/*
 * Parts of this library are derived by:
 *
 * Posix Threads library for Microsoft Windows
```

```
*
* Use at own risk, there is no implied warranty to this code.
* It uses undocumented features of Microsoft Windows that can change
* at any time in the future.
*
* (C) 2010 Lockless Inc.
* All rights reserved.
*
* Redistribution and use in source and binary forms, with or without modification,
* are permitted provided that the following conditions are met:
*
*
*  * Redistributions of source code must retain the above copyright notice,
*    this list of conditions and the following disclaimer.
*  * Redistributions in binary form must reproduce the above copyright notice,
*    this list of conditions and the following disclaimer in the documentation
*    and/or other materials provided with the distribution.
*  * Neither the name of Lockless Inc. nor the names of its contributors may be
*    used to endorse or promote products derived from this software without
*    specific prior written permission.
*
* THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AN
* ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
* WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED.
* IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
* INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING,
* BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
* DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
* LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE
* OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED
* OF THE POSSIBILITY OF SUCH DAMAGE.
*/
```

qwentts-LICENSE.txt

```
MIT License

Copyright (c) 2023-2026 The omnivoice.cpp authors

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```